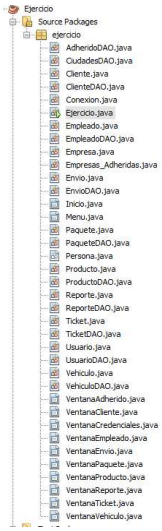
Una vez terminado todo se lo mandé a mi compañero Benjamin para que lo revisara, el código estaba bastante bien, pero tuve algunas correcciones. Una de estas fue borrar el main de la ventana que me olvide por completo y otra fue que me olvide de agregar el método de actualizar tabla, ya que mi código solamente actualizaba la tabla cuando tocaba mostrar.

9/10

ROSATO:Se concluyó de forma exitosa la programación, testeo local y finalización de la totalidad de los procedimientos almacenados que habían sido solicitados por el líder del equipo de desarrollo, logrando la estabilización de las consultas requeridas para la persistencia del sistema antes de pasar a las fases de integración con la interfaz gráfica.

SARAGUSTI:Entre el 8 al 9 de septiembre estuve haciendo todas las ventanas con conexion a sql como un seguro por si los chicos no entregaban sus ventanas a tiempo, esto previamente acordado con el líder Goldfarb, debido a que no nos podemos retrasar con los tiempos acordados debido al poco tiempo restante remanente.



(No puedo mostrar todo el código debido a que es mucho pero para tener una comparación el archivo general tenía partes rojas en todas las clases excepto que en el seguro no esta por la debida modificación de mi parte)

En estos días también estuve haciendo las modificaciones de los procedimientos de mis compañeros debido a que algunos de estos tiraban errores y yo no podía continuar trabajando con esos errores.

Además colabore con mi parte de triggers, agregando algunos a la tabla ciudades y otros a la tabla clientes, creando las tablas auxiliares de cliente histórico y ciudad histórica correspondientemente debido a que si se borra se guardarán con la colaboración del trigger,Además de crear un auxiliar de envios que cada vez que se modifica el origen del envío se guarda en una tabla auxiliar para saber porqué sitios recorrió este, para luego tener en cuenta como fue viajando y donde está actualmente.

10/10

PEDRAZA:Comencé con la actividad el día **10/10**, finalizando las clases DAO ese día, pero se me dificultaba mucho entender las condiciones de las ventanas. Por un lado, consulté al líder Goldfarb si tenía que incluir los demás procedimientos extras (además de los del CRUD) en las ventanas, pero me dijo que no. Por otro lado, le pedí a Saragusti (quien puso

las condiciones de las ventanas) que me lo explique bien. Me mandó un audio explicando todo y por lo tanto, el dia **11/10** pude realizar las ventanas correctamente .

GOLDFARB:

Actividades Realizadas:

Hoy logré terminar mi claseDAO + mi ventana de Producto (ProductoDAO + VentanaProducto). Verifique junto con Benjamin, y después de arreglar unos errores mínimos, la ventana funcionaba correctamente.

11/10

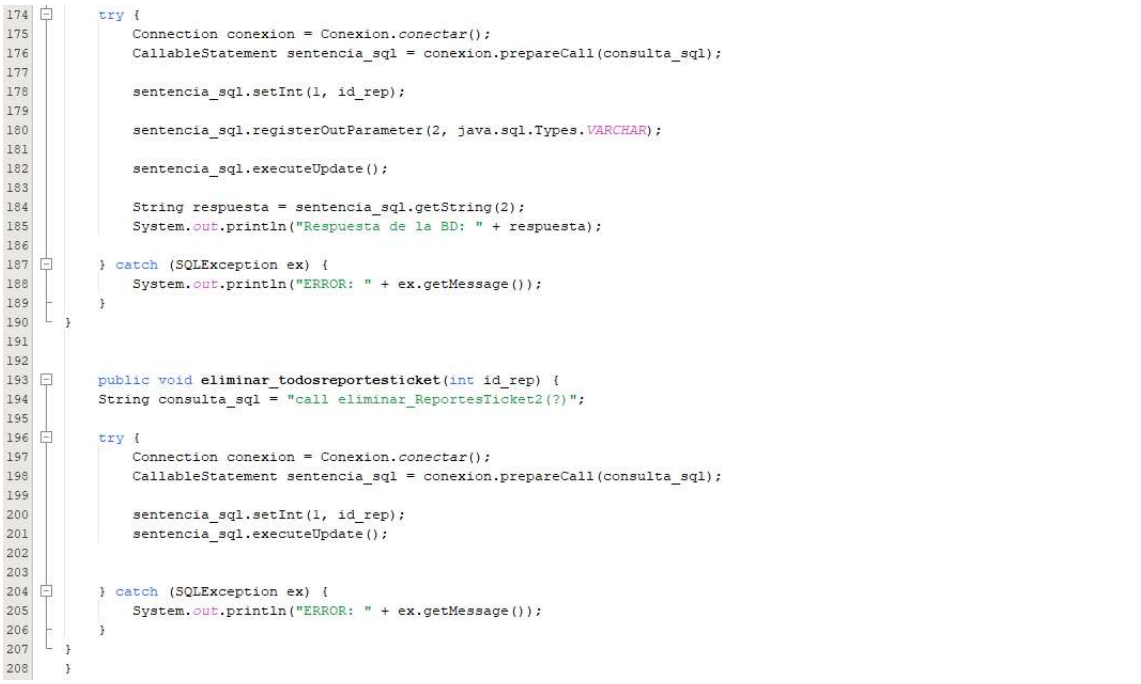
PEDRAZA:Clase ReporteDAO :

```
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import javax.swing.JOptionPane;
/**
 *
 * @author Redes-20
 */
public class ReporteDAO {
    public void insertar(Reporte r) throws SQLException {
    String consulta_sql = "CALL agregar_reportes(?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setString(1, r.getDetalle());

        sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);

        sentencia_sql.executeUpdate();

        String mensajeRespuesta = sentencia_sql.getString(2);
        System.out.println("Respuesta del servidor: " + mensajeRespuesta);
        System.out.println("Reporte guardado exitosamente");

    } catch (SQLException ex) {
        System.out.println("ERROR :" + ex.getMessage());
    }
}
    public  ArrayList<Reporte> listar(){
        ArrayList<Reporte> reportes=new ArrayList<>();
        String consulta_sql="call mostrar_reportes()";

            Connection conexion=Conexion.conectar();
            try {
                CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
                ResultSet resultado=sentencia_sql.executeQuery();
                while(resultado.next()){
                    Reporte rep=new Reporte();
```

```
                    rep.setIdReporte(resultado.getInt("ID_Reporte"));
                    rep.setDetalle(resultado.getString("detalle"));

                    reportes.add(rep);
                    System.out.println("------------------");
                }
            } catch (SQLException ex) {
                System.out.println("ERROR "+ex.getMessage());
            }
            return reportes;


        }
    public Reporte buscar(int id) {
     String consulta_sql = "call buscar_reportess(?)";
     Connection conexion = Conexion.conectar();

     try {
         CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
         sentencia_sql.setInt(1, id);
         ResultSet resultado = sentencia_sql.executeQuery();

         if (resultado.next()) {
             Reporte rep=new Reporte();
                    rep.setIdReporte(resultado.getInt("ID_Reporte"));
                    rep.setDetalle(resultado.getString("detalle"));

             System.out.println("------------------");
             return rep;
         } else {
             System.out.println("No se encontró ningún reporte con el ID: " + id);
         }
     } catch (SQLException ex) {
         System.out.println("ERROR " + ex.getMessage());
     }
     return null;
 }
     public void eliminar(int id) {
     String consulta_sql = "call eliminarReporte(?, ?)";

     try {
         Connection conexion = Conexion.conectar();
```

```
         CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

         sentencia_sql.setInt(1, id);

         sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);

         sentencia_sql.executeUpdate();

         String respuesta = sentencia_sql.getString(2);
         System.out.println("Respuesta de la BD: " + respuesta);
         System.out.println("Registro eliminado");

     } catch (SQLException ex) {
         System.out.println("ERROR: " + ex.getMessage());
     }
 }

    public void actualizar(Reporte rep) {
     String consulta_sql = "call editar_reporte(?, ?, ?)";

     try {
         Connection conexion = Conexion.conectar();
         CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

         sentencia_sql.setInt(1, rep.getIdReporte());
         sentencia_sql.setString(2, rep.getDetalle());

         sentencia_sql.registerOutParameter(3, java.sql.Types.VARCHAR);

         sentencia_sql.executeUpdate();

         String respuesta = sentencia_sql.getString(3);
         System.out.println("Respuesta de la BD: " + respuesta);
         System.out.println("Datos actualizados correctamente");

     } catch (SQLException ex) {
         System.out.println("Error " + ex.getMessage());
     }
 }

     public void mostrar_tickets(int id){
         String consulta_sql="call Tickets_reporte(?)";

             Connection conexion=Conexion.conectar();
```

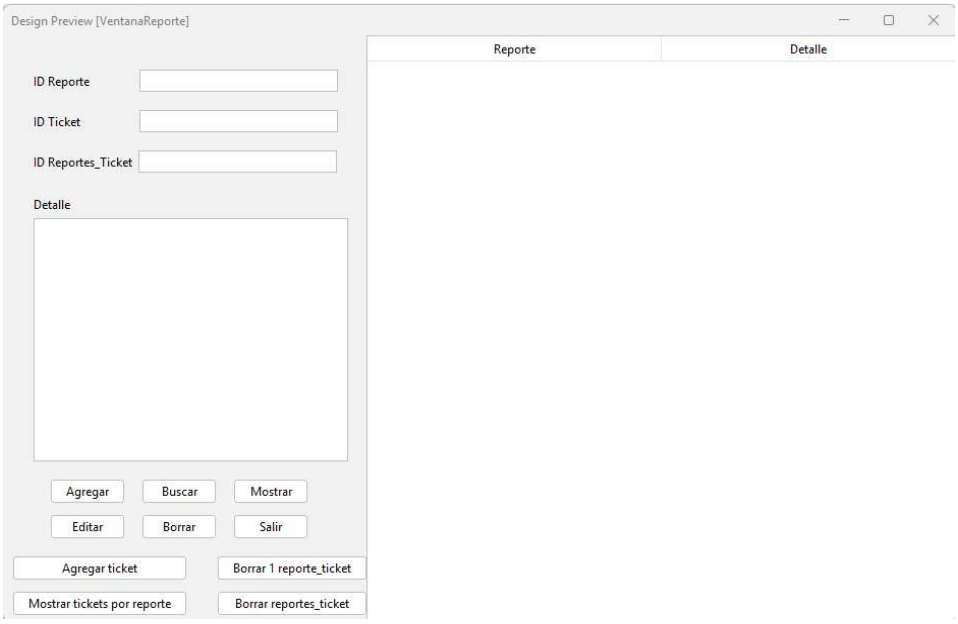
```
        try {
            CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
            sentencia_sql.setInt(1, id);
            ResultSet resultado=sentencia_sql.executeQuery();
            while(resultado.next()){
                int id_ticket=resultado.getInt("ID_Ticket");
                int id_cliente=resultado.getInt("ID_Cliente");
                String fecha=resultado.getString("fecha");
                String detalle=resultado.getString("detalle");
                int total=resultado.getInt("total");
                String pago=resultado.getString("metodospago");
                JOptionPane.showMessageDialog(null, "ID del ticket: "+id_ticket+"\n ID del cliente: "+id_cliente+"\n Fecha: " + fecha+"\n Detalle: " + detalle+"\n Total: "+total+"\n Metodo de pago: " +
                System.out.println("--------------------");
            }
        } catch (SQLException ex) {
            System.out.println("ERROR "+ex.getMessage());
        }
    }
    public void agregar_ticket(int id_reporte, int id_ticket){
        String consulta_sql = "CALL agregarReportes_Ticket(?, ?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setInt(1, id_reporte);
        sentencia_sql.setInt(2, id_ticket);

        sentencia_sql.registerOutParameter(3, java.sql.Types.VARCHAR);

        sentencia_sql.executeUpdate();

        String mensajeRespuesta = sentencia_sql.getString(8);
        System.out.println("Respuesta del servidor: " + mensajeRespuesta);

    } catch (SQLException ex) {
        System.out.println("ERROR :" + ex.getMessage());
    }
    }

    public void eliminar_reporteticket(int id_rep) {
    String consulta_sql = "call eliminar_ReportesTicket(?, ?)";

    try {
```



```
        try {
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

            sentencia_sql.setInt(1, id_rep);

            sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);

            sentencia_sql.executeUpdate();

            String respuesta = sentencia_sql.getString(2);
            System.out.println("Respuesta de la BD: " + respuesta);

        } catch (SQLException ex) {
            System.out.println("ERROR: " + ex.getMessage());
        }
    }


    public void eliminar_todosreportesticket(int id_rep) {
    String consulta_sql = "call eliminar_ReportesTicket2(?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setInt(1, id_rep);
        sentencia_sql.executeUpdate();


    } catch (SQLException ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
    }
    }
```

La clase TicketDAO tiene el mismo formato, por lo que no incluiré capturas de ese código para no irradiar la carpeta con más capturas.

Ventana Reporte (Preview):



Código para agregar ticket:

```
410     private void btnAddTicketActionPerformed(java.awt.event.ActionEvent evt) {
411         if (contador >= 10){
412             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
413             System.exit(0);
414         }
415         String id_ticket = txtTicket.getText();
416         String id_reporte = txtReporte.getText();
417          if("".equals(id_ticket)|| "".equals(id_reporte) ){
418              JOptionPane.showMessageDialog(null, "ERROR - Los campos de 'ID Reporte' e 'ID Ticket' no pueden estar vacios.");
419              contador++;
420             return;
421         }
422         if(!id_ticket.matches("[0-9]+")){
423             JOptionPane.showMessageDialog(null, "ERROR - El ID del ticket solo puede contener numeros.");
424             contador++;
425             return;
426         }
427         if(!id_reporte.matches("[0-9]+")){
428             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
429             contador++;
430             return;
431         }
432         int idTicket=Integer.parseInt(id_ticket);
433         int idReporte=Integer.parseInt(id_reporte);
434 
435         dao_reporte.agregar_ticket(idReporte, idTicket);
436     }
```

Código para mostrar ticket mediante ID_Reporte:

```
387     private void btnMostrarTicketsActionPerformed(java.awt.event.ActionEvent evt) {
388         if (contador >= 10){
389             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
390             System.exit(0);
391         }
392         String id = txtReporte.getText();
393 
394          if("".equals(id)){
395              JOptionPane.showMessageDialog(null, "ERROR - El campo 'ID Reporte' no puede estar vacio.");
396              contador++;
397             return;
398         }
399         if(!id.matches("[0-9]+")){
400             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
401             contador++;
402             return;
403         }
404 
405         int id_reporte=Integer.parseInt(id);
406 
407         dao_reporte.mostrar_tickets(id_reporte);
408     }
```

Código para borrar un reporte_ticket:

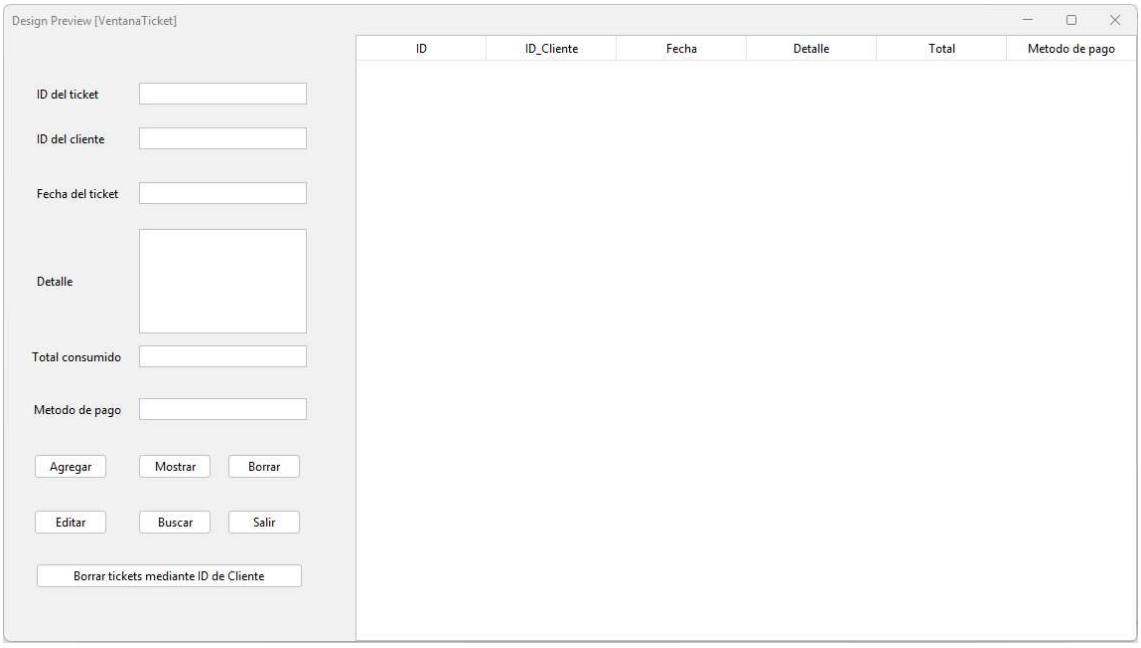
```
460     private void btnBorrar1ReporteActionPerformed(java.awt.event.ActionEvent evt) {
461         if (contador >= 10){
462             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
463             System.exit(0);
464         }
465         String id = txtReporte.getText();
466          if("".equals(id)){
467              JOptionPane.showMessageDialog(null, "ERROR - El campo 'ID Reporte' no puede estar vacio.");
468              contador++;
469             return;
470         }
471         if(!id.matches("[0-9]+")){
472             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
473             contador++;
474             return;
475         }
476 
477         int id_reporte=Integer.parseInt(id);
478 
479         dao_reporte.eliminar_reporteticket(id_reporte);
480     }
```

Código para borrar todos los reportes_tickets de 1 ID_Reporte:

```
438     private void btnBorrarReportesActionPerformed(java.awt.event.ActionEvent evt) {
439         if (contador >= 10){
440             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
441             System.exit(0);
442         }
443         String id = txtReporte.getText();
444          if("".equals(id)){
445              JOptionPane.showMessageDialog(null, "ERROR - El campo 'ID Reporte' no puede estar vacio.");
446              contador++;
447             return;
448         }
449         if(!id.matches("[0-9]+")){
450             JOptionPane.showMessageDialog(null, "ERROR - El ID del reporte solo puede contener numeros.");
451             contador++;
452             return;
453         }
454 
455         int id_reporte=Integer.parseInt(id);
456 
457         dao_reporte.eliminar_todosreportesticket(id_reporte);
458     }
```

Los códigos CRUD son como los que hice para las validaciones el **30/9.**

Ventana Ticket (Preview):



Código para borrar Tickets mediante ID_Cliente:

```
420     private void btnBorrarPorClienteActionPerformed(java.awt.event.ActionEvent evt) {
421         if (contador >= 10){
422             JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
423             System.exit(0);
424         }
425 
426         String id = txtCliente.getText();
427         if("".equals(id)){
428             JOptionPane.showMessageDialog(null, "ERROR - No pueden estar vacios los campos");
429             contador++;
430             return;
431         }
432         if(!id.matches("[0-9]+")){
433             JOptionPane.showMessageDialog(null, "ERROR - El ID del cliente solo puede contener numeros.");
434             contador++;
435             return;
436         }
437 
438         int id_cliente=Integer.parseInt(id);
439 
440         Cliente c = dao_cliente.buscar(id_cliente);
441 
442         if (c == null) {
443             JOptionPane.showMessageDialog(null,
444                 "No existe un cliente con ese ID.");
445             return;
446         }
447 
448         dao.eliminar_porCliente(id_cliente);
449         actualizarTabla();
450     }
```

Los códigos CRUD son como los que hice para las validaciones el **30/9.**

12/10
ROSATO:11/10- 17/10

Se dio inicio al desarrollo e implementación de las interfaces gráficas de usuario correspondientes a los módulos asignados, específicamente las ventanas para la gestión de Empresas Adheridas. Durante este período, las tareas se centraron en la codificación de la lógica visual y de eventos en el entorno de desarrollo de Java.

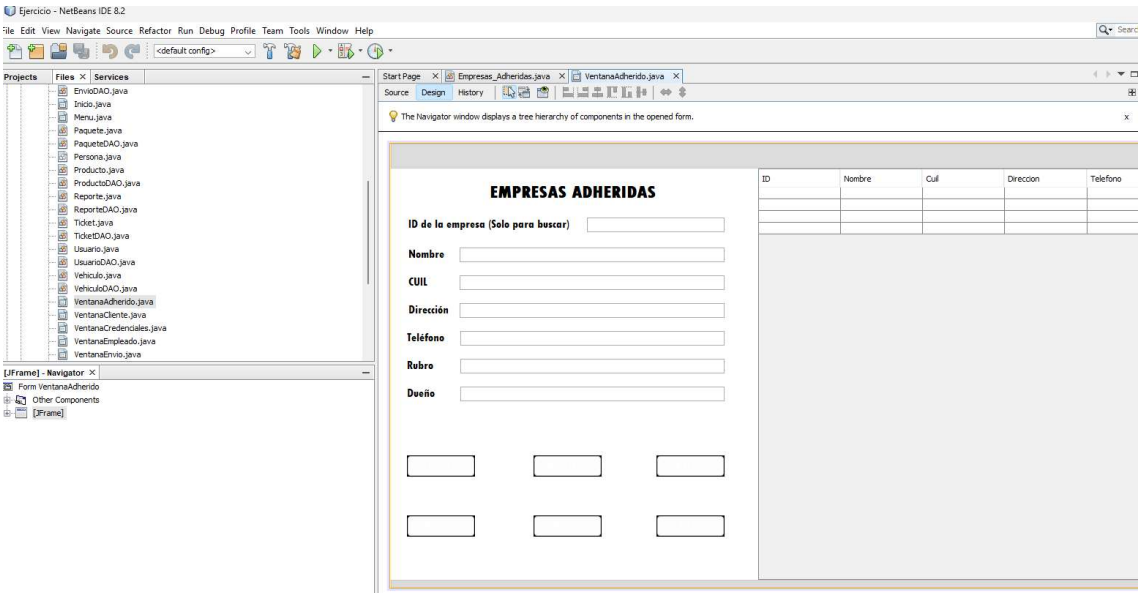
En paralelo, se trabajó en el modelado tridimensional de la infraestructura logística utilizando la herramienta SketchUp. Debido a la complejidad técnica del diseño tridimensional de los almacenes y flotas, se recurrió al uso de la inteligencia artificial Gemini como soporte pedagógico y técnico. Esto permitió agilizar la curva de aprendizaje, optimizar el uso de las herramientas de geometría y lograr un prototipo a escala que representa de manera fiel la distribución espacial de la empresa.

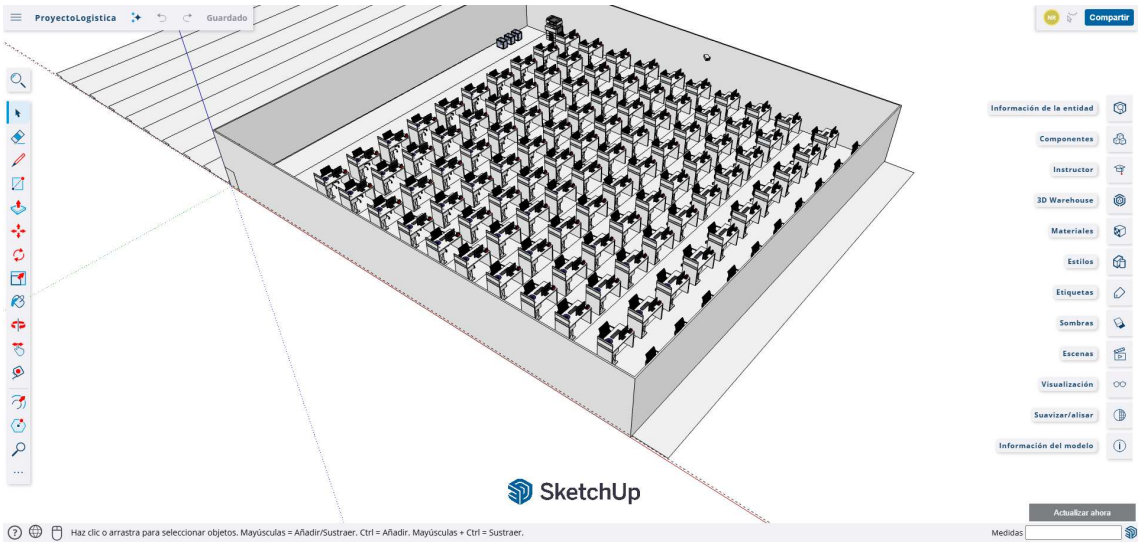
```java
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
/**
 *
 * @author Benjamin
 */
public class AdheridoDAO {
    public void insertar(Empresas_Adheridas c) throws SQLException {
    String consulta_sql = "CALL agregar_empresa_adherida(?, ?, ?, ?, ?, ?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setString(1, c.getNombre());
        sentencia_sql.setString(2, c.getCuil());
        sentencia_sql.setString(3, c.getDireccion());
        sentencia_sql.setInt(4, c.getTelefono());
        sentencia_sql.setString(5, c.getRubro());
        sentencia_sql.setString(6, c.getDuenio());

        sentencia_sql.registerOutParameter(7, java.sql.Types.VARCHAR);

        sentencia_sql.executeUpdate();

        String mensajeRespuesta = sentencia_sql.getString(7);
        System.out.println("Respuesta del servidor: " + mensajeRespuesta);
        System.out.println("Empleado guardado exitosamente");

    } catch (SQLException ex) {
        System.out.println("ERROR :" + ex.getMessage());
```

Start Page | Empresas_Adheridas.java | VentanaAdherido.java | AdheridoDAO.java | VentanaVehiculo.java

Source | History

```
40      } catch (SQLException ex) {
41          System.out.println("ERROR :" + ex.getMessage());
42      }
43  }
44      public  ArrayList<Empresas_Adheridas> listar(){
45          ArrayList<Empresas_Adheridas> adherido=new ArrayList<>();
46          String consulta_sql="call mostrar_adheridos()";
47
48              Connection conexion=Conexion.conectar();
49              try {
50                  CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
51                  ResultSet resultado=sentencia_sql.executeQuery();
52                  while(resultado.next()){
53                      Empresas_Adheridas p=new Empresas_Adheridas();
54                      p.setId_empresa(resultado.getInt("ID_empresa"));
55                      p.setNombre(resultado.getString("nombre"));
56                      p.setCuil(resultado.getString("cuit"));
57                      p.setDireccion(resultado.getString("direccion"));
58                      p.setDuenio(resultado.getString("duenio"));
59                      p.setRubro(resultado.getString("rubro"));
60                      p.setTelefono(resultado.getInt("telefono"));
61                      adherido.add(p);
62                      System.out.println("------------------");
63                  }
64              } catch (SQLException ex) {
65                  System.out.println("ERROR "+ex.getMessage());
66              }
67              return adherido;
68
69
70          }
71      public Empresas_Adheridas buscar(int id) {
72       String consulta_sql = "call buscar_Adheridos(?)";
73       Connection conexion = Conexion.conectar();
74
75       try {
76           CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
77           sentencia_sql.setInt(1, id);
78           ResultSet resultado = sentencia_sql.executeQuery();
79
80           if (resultado.next()) {
```





GOLDFARB:Actividades Realizadas:

Ayer, decidimos con el grupo que toda actividad restante del proyecto se le daría a HTML, ya que esta sería la parte más complicada de todo el proyecto. Por esto decidí plantear que paginas usaremos. Falta ver cuales usamos y cuales descartamos.

1- INDEX
2-CREAR CUENTA
3- INICIAR SESIÓN
4- VER ENVIO
5- VER TODOS LOS ENVÍOS
6-CREAR ENVÍO
7-REGISTRAR PERSONA
8- REGISTRAR CLIENTE
9-CONSULTAR CLIENTES
10-REGISTRAR TRANSPORTISTA
11-CONSULTAR TRANSPORTISTA
12-AGREGAR PAQUETE A UN ENVIO
13-ASOCIAR VEHÍCULOS A UN ENVÍO (puede ser parte de CrearEnvio)
14-REGISTRAR PAQUETES
15-AGREGAR PRODUCTOS A LOS PAQUETES
16-CONSULTAR PAQUETES
17-REGISTRAR PRODUCTOS
18-CONSULTAR PRODUCTOS
19-CONTROLAR EL STOCK (editar producto digamos)
20-REGISTRAR VEHICULOS
21-REGISTRAR EMPRESAS ADHERIDAS
22-CONSULTA GENERAL

SARAGUSTI:El 12 de septiembre estuve corrigiendo las entregas de Xiomara, Pedraza y Goldfarb, en el caso de Golfarb lo ayude con la realización de su ventana y las clases Dao debido a que no entendía mucho la metodología de estas, las entregas de Xiomara y Pedraza estaban bien,excepto con algunos problemas de refrescarse despues de instrucciones o de fallos con algunos nombres de variables que no tardaron en corregirse.
Además mientras estaba ayudando a goldfarb con sus ventanas agregue el modo de inicio de sesión actualizado solicitado en el cual puedes crear tu cuenta e iniciar sesión, eso si la contraseña para crear si o si tiene que ser Logistica 32 para corroborar que si sos alguien fiable, esta se puede cambiar igualmente después de iniciar sesión.

(Visualización de la nueva ventana de inicio de sesión con el dinámico, pudiendo crear nuevas cuentas rápidamente)

En el caso de la ventana para ver las credenciales puse un JLAbel sin texto que se le cambia el valor al número de usuario que es, en este caso el id, y después las barras de texto que aparecen en blanco se rellenan con los campos de la clase usuario, pudiendo ver los datos que ingresaste y hasta modificarlos también en la base de datos.

(La primera es como se ve sin ejecutarse la segunda cuando se ejecuta y se entra con un usuario ya iniciado)

```
delimiter //
create procedure validar_login(in p_nombre varchar(100), in p_contra varchar(100))
        BEGIN
                select * from usuarios where nombre=p_nombre and
contrasenia=p_contra;
        end//
        delimiter;
DELIMITER //

CREATE PROCEDURE editar_usuario(
    IN p_usuario INT,
    IN p_nombre VARCHAR(100),
    IN p_apellido VARCHAR(100),
    IN p_email VARCHAR(100),
    IN p_contrasenia VARCHAR(500)
)
BEGIN
    UPDATE usuarios
    SET nombre = p_nombre,
        apellido = p_apellido,
        email = p_email,
        contrasenia = p_contrasenia
    WHERE id_usuario = p_usuario;
END//

        delimiter;
DELIMITER //

CREATE PROCEDURE crear_usuario(
    IN p_nombre VARCHAR(100),
    IN p_apellido VARCHAR(100),
    IN p_email VARCHAR(100),
    IN p_contrasenia VARCHAR(500)
)
BEGIN
    insert into
usuarios(nombre,apellido,email,contrasenia)values(p_nombre,p_apellido,p_email,p_contras
```

(Como se veían los procedimientos que realice para no tener que hacer consultas o updates, debido a que eso puede generar una brecha de seguridad en el sistema)

13/10

UREÑA:Ese día Benjamin se encargó de armar el reparto de las pantallas que nos tocaba diseñadas en HTML, con el objetivo de empezar a conectarlas de una vez por todas al servidor web. En la distribución de tareas , a mi me asignaron la responsabilidad de armar dos páginas, “Registrar productos” y ”Consultar Productos”.

La primera pantalla, la de registro, la tengo que diseñar pensando en un formulario limpio donde el usuario pueda cargar toda la información.

La segunda pantalla, la de consulta, viene con un poco más de complejidad y trabajo de lógica de mi parte. No se trata solo de mostrar el listado de lo que hay en stock, sino que dentro de esa misma página tengo que meter los botones y las funcionalidades para poder “Editar” y “Eliminar” cualquier producto directamente desde ahí.

GOLDFARB:Actividades Realizadas:

Ya decidimos que paginas haremos y quien hará cada piso de la Maqueta.

1-INDEX
2-INDEX ADMIN
2-CREAR CUENTA
3- INICIAR SESIÓN
4- VER ENVIO
5- VER TODOS LOS ENVÍOS
6-CREAR ENVÍO
7-REGISTRAR PERSONA

Agustin:

8- REGISTRAR CLIENTE
9-CONSULTAR CLIENTES

Pedraza:
10-REGISTRAR TRANSPORTISTA
11-CONSULTAR TRANSPORTISTA
12-AGREGAR PAQUETE A UN ENVIO

Goldfarb:
13-ASOCIAR VEHÍCULOS A UN ENVÍO (puede ser parte de CrearEnvio)
14-REGISTRAR PAQUETES
15-AGREGAR PRODUCTOS A LOS PAQUETES

Xiomara:
16-CONSULTAR PAQUETES
17-REGISTRAR PRODUCTOS
18-CONSULTAR PRODUCTOS

Rosato:
19-CONTROLAR EL STOCK (editar producto digamos)
20-REGISTRAR VEHICULOS
21-REGISTRAR EMPRESAS ADHERIDAS

Benjamin:

22-CONSULTA GENERAL

Maqueta Redes:
Nico:
Las 5 primeras
Benjamin:

Benjamin:

22-CONSULTA GENERAL

Maqueta Redes:
Nico:
Las 5 primeras

Zowmir:
6,7,8

Pedraza:
9,10,11

Xiomara:
12,13,14

Goldfarb:
15,17,19

Benjamin:
16,18,20

Fecha: 13/10/26

Ubicación: Escuela : Aula

Hora: 11:05

Actividades Realizadas:

Al estar viendo un rato las páginas html a hacer, me di cuenta que varias eran redundantes y por ende, innecesarias, marque en rojo las ventanas que deberían ser parte de un html, no un html aparte.

1-INDEX (SE HACE ENTRE TODOS)
2-INDEX ADMIN (SE HACE ENTRE TODOS)
2-CREAR CUENTA
3- INICIAR SESIÓN
4- VER ENVIO
5- VER TODOS LOS ENVÍOS
6-CREAR ENVÍO
7-REGISTRAR PERSONA

Agustín:

8- REGISTRAR CLIENTE
9-CONSULTAR CLIENTES

Pedraza:
10-REGISTRAR TRANSPORTISTA
11-CONSULTAR TRANSPORTISTA
12-AGREGAR PAQUETE A UN ENVÍO. (Se hace desde CrearEnvio)

Goldfarb:
13-ASOCIAR VEHÍCULOS A UN ENVÍO (Se hace desde CrearEnvio)
14-REGISTRAR PAQUETES
15-AGREGAR PRODUCTOS A LOS PAQUETES (Se hace desde CrearPaquete)
16-CONSULTAR PAQUETES

Xiomara:

17-REGISTRAR PRODUCTOS
18-CONSULTAR PRODUCTOS

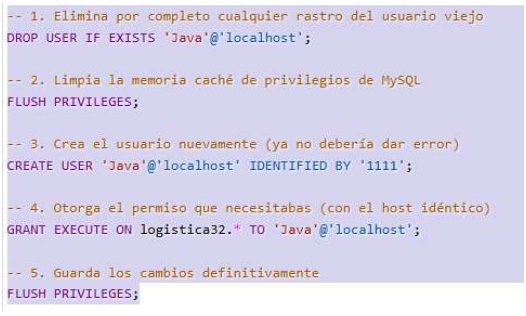
Rosato:
19-CONTROLAR EL STOCK (editar producto digamos)
20-REGISTRAR VEHÍCULOS
21-REGISTRAR EMPRESAS ADHERIDAS

Benjamin:

22-CONSULTA GENERAL

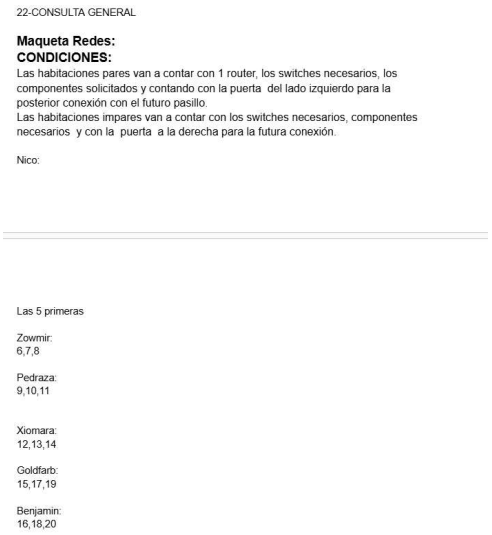
Ahora, me pondré a hacer mis páginas HTML asignadas.

SARAGUSTI:El 13 de septiembre le hice la devolución a Rosato de su ventana, que tenía el tema de no limpiar bien los campos, de no poder editar/agregar clientes debido al número telefónico y se debía a que era un string no un int, por lo que antes de insertarlo en el procedure se lo vuelve int para no tener que modificar la clase Persona, además de esto actualice para que no se use el usuario root, si no que se use el nuevo usuario llamado JAVA el cual solo tiene el permiso del execute.



(La primera foto siendo como se ve el perfil seleccionado desde JAVA con su contraseña y con el AccesO Procedure Bodies, debido a que no puede acceder a las tablas ni a los datos del procedimiento, sólo pudiendo tomar el resultado, y en cierto caso un mensaje de si se hizo correctamente o si hubo un error)

Se pone esta última parte de acces bodies debido a que si no al intentar comunicarse va a tirar un error y no va a poder acceder a los datos de mysql, pero de resto funciona igual que con el root.


22-CONSULTA GENERAL

Maqueta Redes:
CONDICIONES:
Las habitaciones pares van a contar con 1 router, los switches necesarios, los componentes solicitados y contando con la puerta del lado izquierdo para la posterior conexión con el futuro pasillo.
Las habitaciones impares van a contar con los switches necesarios, componentes necesarios y con la puerta a la derecha para la futura conexión.

Nico:

Las 5 primeras

Zowmir:
6,7,8

Pedraza:
9,10,11

Xiomara:
12,13,14

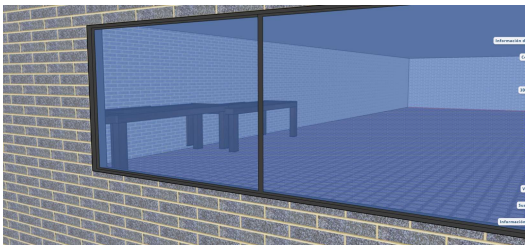
Goldfarb:
15,17,19

Benjamin:
16,18,20

(Me asignaron las habitaciones para la maqueta y una página de html, no más debido a que como hice bastante previamente , en el caso de darme más le sacaría la parte a los demás)

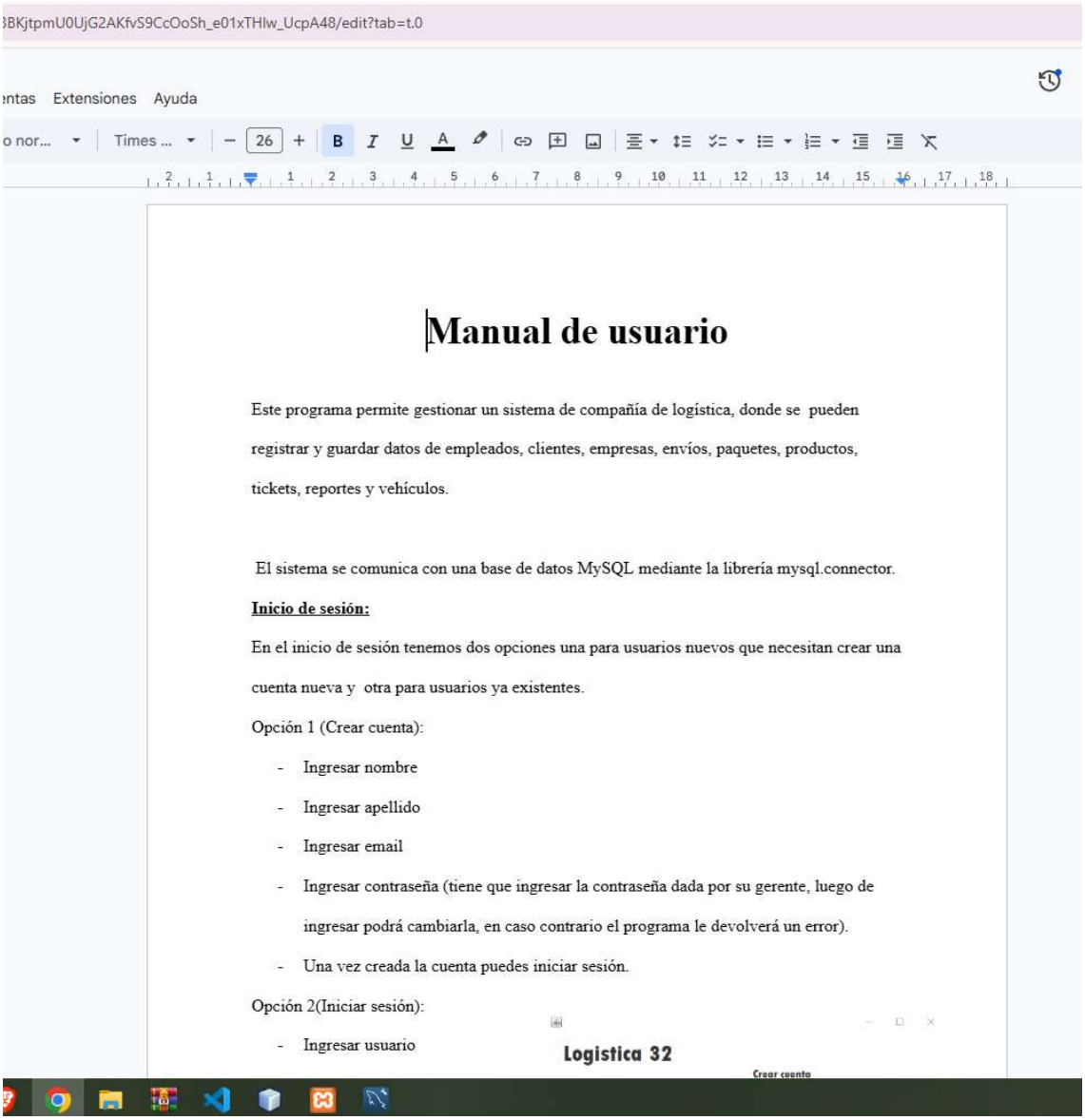
De la parte de la maqueta puse en práctica la organización, debido a que va a ser todo un pasillo ancho, las habitaciones no pueden tener la puerta donde quieran, por eso es que moldee un orden para así hacerlo, además de que no todas las habitaciones van a tener un router, solo las pares, debido a que en el presupuesto los routers que usamos tienen dos puertos(o 4 si contamos los seriales), entonces no necesitamos 20 si no que 10 routers en total.

Ese mismo dia empeze a practicar con la maqueta, haciendo una parte robusta de cómo se vería el piso 16, siendo que todavia ni esta a la mitad pero la estructura base está fija y hecha, con las paredes y las dimensiones cautelosamente medidas, coloque 3 mesas pero traspasan medio el suelo, lo que genera inconvenientes.

 (Como se ve la habitación desde la única ventana que hay por el momento, con las tres mesas,que una no se ve, atravesando las paredes y rompiendo las leyes de la física).

14/10

UREÑA: No recuerdo bien las fechas pero durante las siguientes semanas estuve realizando varias cosas, lo primero que realicé fue el manual de usuario, lo hice con un poco de ayuda de mi compañero Benjamin Saragusti. Estuve dos dias con esto ya que también tenía que hacer otro proyecto para la materia Proyecto Informático 2.



Otras de las cosas que estuve realizando estas semanas fueron los triggers que me faltaban y el diseño (css) de las paginas que me tocaron (“mostrarProductos”, “registrarProductos”).

TRIGGERS XIOMARA:

Asigna automáticamente la fecha del ticket si esta es nula.

```
DELIMITER //

CREATE TRIGGER ticketFecha
BEFORE INSERT ON Tickets
FOR EACH ROW
BEGIN
   IF NEW.fecha IS NULL THEN
      SET NEW.fecha = CURDATE();
   END IF;


   END IF;
END//

DELIMITER ;
```

antes de insertar verifica que el monto total no sea negativo.

```
CREATE TRIGGER montoNegativo
```

```
BEFORE INSERT ON Tickets
FOR EACH ROW
BEGIN
   IF NEW.total < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Error: El total del ticket no puede ser un valor negativo.';
```

```
        SET MESSAGE_TEXT = 'Error: El total del ticket no puede ser un valor negativo.';
    END IF;
END//

DELIMITER ;
```

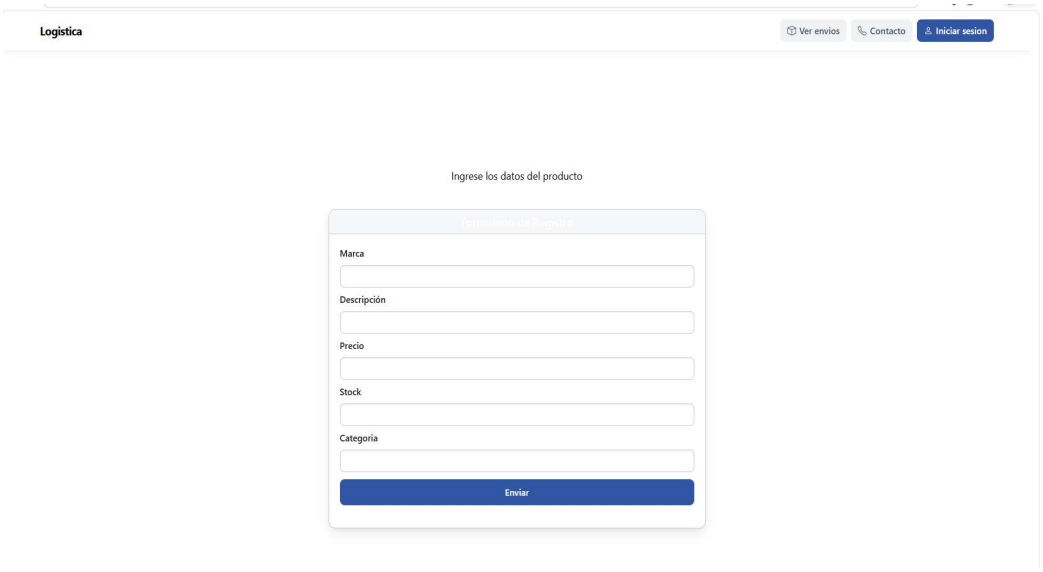
Limpiar la tabla intermedia al eliminar un ticket

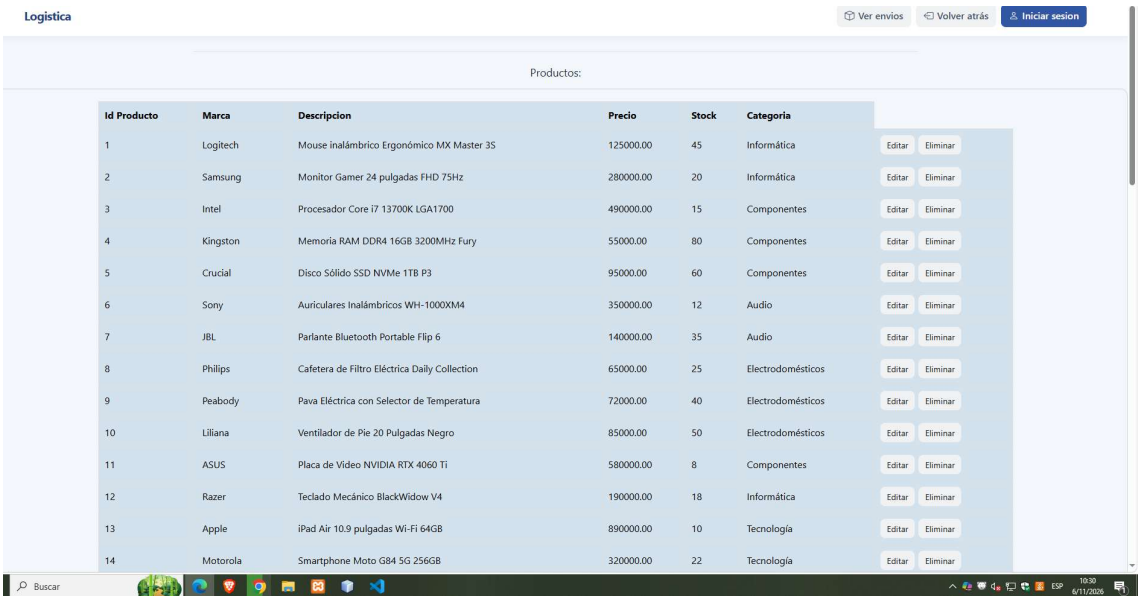
```
DELIMITER//
CREATE TRIGGER limpiarTabla
AFTER DELETE ON Tickets
FOR EACH ROW
BEGIN
    DELETE FROM Reportes_Ticket WHERE ID_Ticket = OLD.ID_Ticket;
END//

DELIMITER ;
```

Antes de insertar valida que ambos id existan(ticket y reporte)

```
CREATE TRIGGER verificarExist
BEFORE INSERT ON Reportes_Ticket
FOR EACH ROW
BEGIN
    DECLARE v_existe_ticket INT;

    SELECT COUNT(*) INTO v_existe_ticket
    FROM Tickets
    WHERE ID_Ticket = NEW.ID_Ticket;

    IF v_existe_ticket = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: No se puede asociar el reporte porque el Ticket
indicado no existe.';
    END IF;
END//

DELIMITER ;
```

Despues comence con la conexión con el servidor web. Se me complicó mucho la parte de la conexión por lo tanto no llegue a terminarlo. Me tuve que juntar con mis compañeros Nicolás Rosato, Santiago Goldfarb y Benjamin para que me ayuden con eso. Estuve mucho tiempo con esto ya que no llegaba a entenderlo, también porque se me frizo la computadora varias veces.

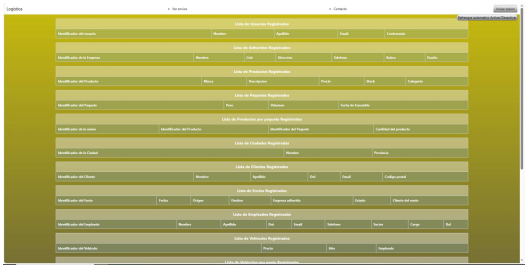




15/10

SARAGUSTI:El jueves 15 de octubre hice la pagina que me correspondía de html, siendo la vista general con todas las tablas de cada una de las tablas de la base de datos, pero solo pude hacer el apartado visual debido al hecho de que no tengo todavía los medios para realizar la conexión a la base de datos. Pero en ambitos artisticos use un modelo de tabla basado en otro que encontre en internet y ademas de eso le agregue un fondo de color degradado para una mejor vision artistica, la pestaña ademas cuenta con la barra de navegacion, copiada de el indice, con un fondo blanco para diferenciarse de todo el body, no solo eso si no que los botones ahora cuentan con un redondeo y ademas de eso tienen un

degradado entre blanco y gris para darles un efecto de que cuentan con un aspecto tridimensional.
Hablando de botones, sin contar el heredado del index, contamos con 2 siendo el de activar un refresco automático que permite a la máquina auto actualizarse cada 5 minutos, en el debido caso de que no la quieras refrescar constantemente para ver como cambio la base de datos, y un botón que te la refresca al pulsar. La metodología detrás del refresco automático es que el botón por si solo hace que un valor de JSON pase a estar de “No” a “Yes”, después dentro del script cada vez que se abre la página se verifica si este dato es positivo , y en tal caso se inicia un contador que al llegar a cero genera el reinicio instantáneo, esto no borra ningun dato si no que permite a la página mostrar los nuevos registros agregados/eliminados o editados de la BD.


(Como se ven las tablas con todos los degradados de color, con los datos de transparencia de las tablas, siendo que los renglones de abajo poseen una mayor transparencia y los titulos cuentan con un tono blanco semi transparente pero visible)

(Vista de como es el boton para refrescar cambios con la estetica de color que le da un toque de tridimencionalidad, para ser mas llamativo al espectador)

(Visualización de la fracción de código de javascript que funciona para la activación del refresco automático, que recibe los datos del archivo JSON llamado estado, este puede ser NO or UNDEFINED debido a que puede no estar inicializado en el momento, y en la parte de abajo está la porción que se ejecuta al iniciar siempre la pagina que revisa la lectura del archivo JSON en el localStorage)

16/10

ZOWNIR:El día de hoy me dispuse a realizar los triggers correspondientes a mi tabla almacenes. En total desarrollé cinco triggers, con el objetivo de controlar los datos que se ingresan o modifican y evitar que se guarde información incorrecta en la base de datos.

Los triggers son instrucciones que se ejecutan automáticamente cuando ocurre una determinada acción en una tabla. En este caso, los utilicé para establecer condiciones que deben cumplirse antes de permitir determinadas operaciones.

Los cinco triggers que realicé fueron los siguientes:

1)EVITAR NOMBRES DE ALMACEN VACIOS AL INSERTAR

Este trigger controla que, al intentar registrar un nuevo almacén, el campo correspondiente al nombre no se encuentre vacío. De esta manera se evita guardar un almacén sin identificación.

2)EVITAR DIRECCIONES VACIAS

Este trigger comprueba que el campo de dirección tenga información al momento de insertar un almacén. Esto permite evitar registros que no tengan una ubicación especificada.

3)EVITAR NOMBRES VACIOS AL MODIFICAR

Este trigger cumple una función similar al primero, pero se aplica cuando se intenta modificar un almacén que ya existe. Su objetivo es impedir que se actualice un registro dejando su nombre vacío.

4)EVITAR DIRECCIONES VACIAS AL MODIFICAR

Este trigger controla que, al modificar los datos de un almacén, no se deje el campo de dirección sin completar.

5)Evitar que el almacen se mueva a una ciudad inexistente

Este trigger verifica que la ciudad asignada al almacén exista dentro de la base de datos. Esto permite evitar que un almacén quede relacionado con una ciudad que no se encuentra registrada.

La realización de estos triggers me permitió comprender cómo se pueden utilizar las condiciones automáticas dentro de una base de datos para controlar la información. También aprendí que es importante validar los datos antes de guardarlos, ya que un registro incorrecto puede generar problemas en otras partes del sistema.

los triggers quedaron tal que así:
1)

```
DELIMITER //

CREATE TRIGGER validar_nombre_almacen
BEFORE INSERT ON Almacenes
FOR EACH ROW
BEGIN
    IF NEW.nombre IS NULL OR TRIM(NEW.nombre) = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El nombre del almacen no puede estar vacío';
    END IF;
END //
DELIMITER;

DELIMITER //
```

2)

```

DELIMITER //

CREATE TRIGGER validar_direccion_almacen
BEFORE INSERT ON Almacenes
FOR EACH ROW
BEGIN
    IF NEW.direccion IS NULL OR TRIM(NEW.direccion) = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La direccion no puede estar vacía';
    END IF;
END //

DELIMITER ;
```

3)

```
DELIMITER //

CREATE TRIGGER validar_nombre_almacen_update
BEFORE UPDATE ON Almacenes
FOR EACH ROW
BEGIN
    IF NEW.nombre IS NULL OR TRIM(NEW.nombre) = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El nombre del almacen no puede estar vacio';
    END IF;
END //

DELIMITER ;
```

4)

```
DELIMITER //

CREATE TRIGGER validar_direccion_almacen_update
BEFORE UPDATE ON Almacenes
FOR EACH ROW
BEGIN
    IF NEW.direccion IS NULL OR TRIM(NEW.direccion) = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La direccion no puede estar vacia';
    END IF;
END //

DELIMITER ;
```

5)

```
DELIMITER //

CREATE TRIGGER validar_ciudad_almacen
BEFORE INSERT ON Almacenes
FOR EACH ROW
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM Ciudades
        WHERE ID_Ciudad = NEW.ciudad
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La ciudad indicada no existe';
    END IF;
END //

DELIMITER ;
```

17/10

SARAGUSTI:El 17 de octubre continúe con el trabajo de la maqueta, Habiendo terminado todas las habitaciones que me correspondía, que eran la 16, la 18 y la 20. Tuve que usar distintos modelos por cada una debido a que la mayoría de modelos que encontraba después no los encontraba o si no necesitaba otro estilo debido a que era otro tipo de sala.
Las habitaciones están hechas a medida solicitada por los profesores, con las puertas ya

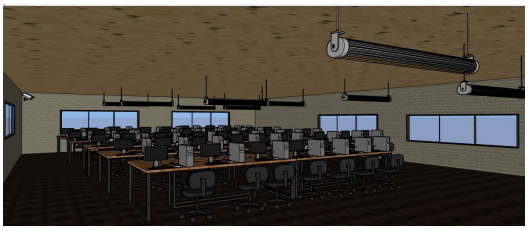
determinadas en su lugar, con las ventanas colocadas para ver el exterior, y con algunos

artículos adicionales para que el espacio no se sintiera vacío.

(Visualización de la habitación N° 16, como fue

mi primera habitación, era bastante grande y tenía pocos equipos tuve que colocar más

elementos de oficina para no sentir el espacio muerto, pero a su vez no tantos para no causar fatiga visual)

 (Visualización desde una esquina de la sala N° 18, que cuenta con 43 computadoras, 43 teléfonos, dos impresoras, una 3d y una común, y las tres cámaras, no se ven pero también están los switches y el router requerido).

(Visualización desde una esquina de la sala N°20, se ven las computadoras, con un diseño de menor cantidad de polígonos, la impresora 3d y la de red, las cámaras en las esquinas, pero lo que no se ve en esta vista son los routers, el switch y unos detalles específicos de la sala que puse para que no se vea vacía la habitación)

18/10
ROSATO:Culminacion del objetivo planteado por el lider del grupo, finalizacion de la ventana Vehiculos

codigo:

```
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
```

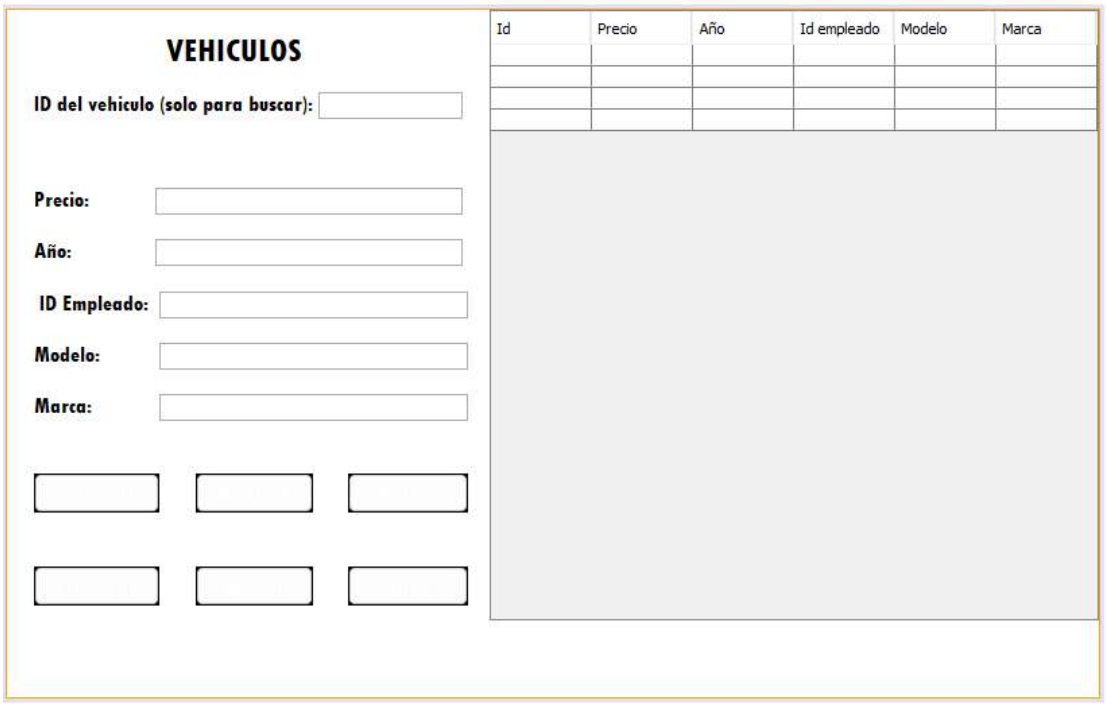
```
public class VehiculoDAO {

    public void insertar(Vehiculo v)throws SQLException {
        String consulta_sql = "CALL crear_vehiculo(?, ?, ?, ?, ?, ?)";

        try{
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);



            sentencia_sql.setInt(1, v.getPrecio());
            sentencia_sql.setInt(2, v.getAnio());
            sentencia_sql.setInt(3, v.getIdEmpleado());
            sentencia_sql.setString(4, v.getModelo());
            sentencia_sql.setString(5, v.getMarca());

            sentencia_sql.registerOutParameter(6, java.sql.Types.VARCHAR);
            sentencia_sql.executeUpdate();
            String mensajeRespuesta = sentencia_sql.getString(6);
            System.out.println("Respuesta del servidor: " +
mensajeRespuesta);
            System.out.println("Vehiculo guardado correctamente");
        }catch(SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }
    }

    public ArrayList <Vehiculo> listar(){
        ArrayList <Vehiculo> lista_vehiculo = new ArrayList<>();
        String consulta_sql="call mostrar_vehiculos()";
        try{
            Connection conexion = Conexion.conectar();
```

```
            CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);
            ResultSet resultado=sentencia_sql.executeQuery();

            while (resultado.next()){
                Vehiculo v = new Vehiculo();

                v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
                v.setPrecio(resultado.getInt("Precio"));
                v.setAnio(resultado.getInt("anio"));
                v.setIdEmpleado(resultado.getInt("ID_Empleado"));
                v.setModelo(resultado.getString("Modelo"));
                v.setMarca(resultado.getString("Marca"));




                lista_vehiculo.add(v);
                System.out.println("~~~~~~~~~~~~~~~~~~~~");
            }
        }catch (SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }
        return lista_vehiculo;
    }

    public void actualizar (Vehiculo v){
         String consulta_sql = "call editar_vehiculos (?, ?, ?, ?, ?, ?, ?)";

        try {
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);

            sentencia_sql.setInt(1, v.getIdvehiculo());
```

```
            sentencia_sql.setInt(2, v.getPrecio());
            sentencia_sql.setInt(3, v.getAnio());
            sentencia_sql.setInt(4, v.getIdEmpleado());
            sentencia_sql.setString(5, v.getModelo());
            sentencia_sql.setString(6, v.getMarca());

            sentencia_sql.registerOutParameter(7, java.sql.Types.VARCHAR);
            sentencia_sql.executeUpdate();
            String respuesta = sentencia_sql.getString(7);
            System.out.println("Respuesta de la BD: " + respuesta);
            System.out.println("Datos actualizados exitoamnete");

        }catch (SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }

    }

    public Vehiculo buscar(int id) {
    String consulta_sql = "call buscar_vehiculo(?)";
    Connection conexion = Conexion.conectar();

    try {
        CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);
        sentencia_sql.setInt(1, id);
        ResultSet resultado = sentencia_sql.executeQuery();

        if (resultado.next()) {
            Vehiculo v =new Vehiculo();
                v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
                v.setPrecio(resultado.getInt("Precio"));
                v.setAnio(resultado.getInt("anio"));
                v.setIdEmpleado(resultado.getInt("ID_Empleado"));
                v.setModelo(resultado.getString("Modelo"));
```

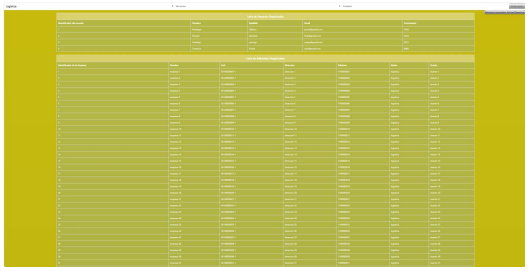
```
                v.setMarca(resultado.getString("Marca"));

            System.out.println("------------------");
            return v;
        } else {
            System.out.println("No se encontró ningún vehiculo con el ID: " +
id);
        }
    } catch (SQLException ex) {
        System.out.println("ERROR " + ex.getMessage());
    }
    return null;
}

    public void eliminar(int id) {
    String consulta_sql = "call eliminar_vehiculo (?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql =
conexion.prepareCall(consulta_sql);

        sentencia_sql.setInt(1, id);

        sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);

        sentencia_sql.executeUpdate();

        String respuesta = sentencia_sql.getString(2);
        System.out.println("Respuesta de la BD: " + respuesta);
        System.out.println("Vehiculo eliminado");

    } catch (SQLException ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
```

}
}



19/10

SARAGUSTI:El 19 de octubre conecte mi ventana, en este caso la gestión general, que tiene la visualización de todas las tablas de la base de datos, usando el servidor http de java, pero no lo pude terminar faltando las tablas de envío vehiculo, envio paquetes, reportes, tickets , reportes por ticket y almacenes, en la configuración hice que mi pagina fuera el índice debido a que las demás todavía no se terminaron/realizaron, en un futuro se reemplazará esa linea por el indice html.

 (Como se ven las tablas con los 50 inserts por tabla, excepto los usuarios que solo tienen 4 usuarios debido a la creación tardía de esta clase).

GOLDFARB:**Fecha: 19/10/26**
Ubicación: Escuela : Aula

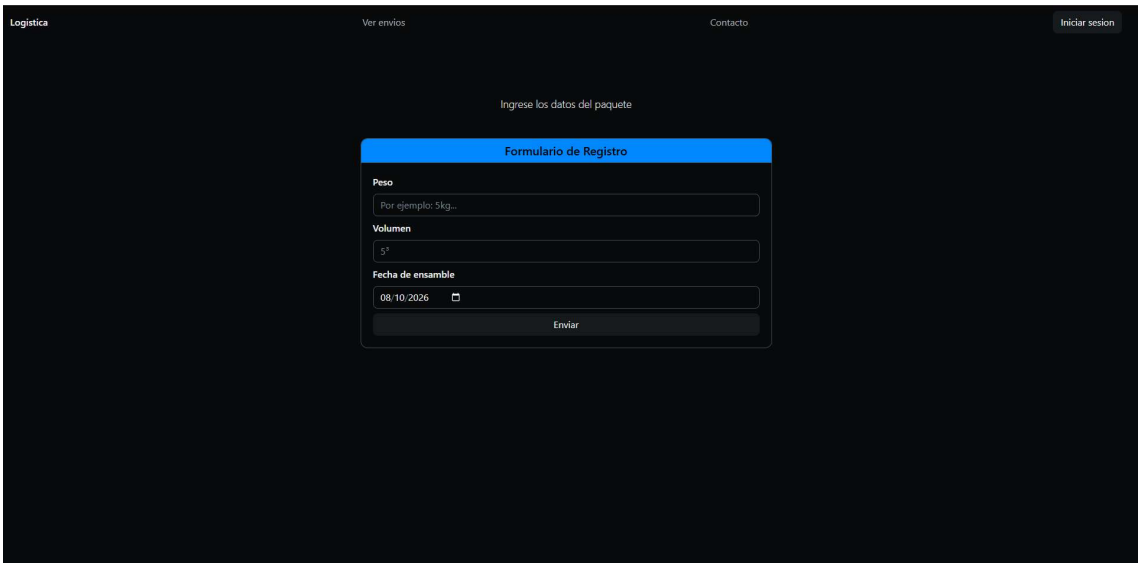
Hora: 11:50

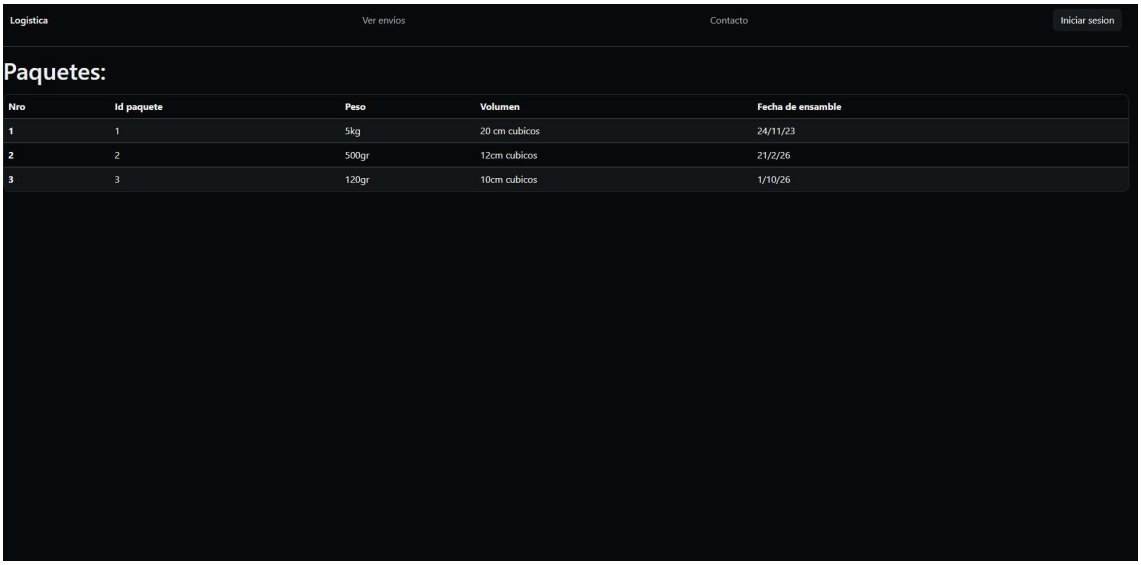
Actividades Realizadas:

Ya avance con el diseño de la página, a la tarde, Nahuel Acosta nos va a enseñar como hacer la conexión entera de HTML a base de datos.

Nahuel me dijo que cambie algunas cosas del diseño como la fuente del texto, añadir iconos, inclusive añadir un mapa para el consultador de envíos del cliente.

Avance con la pagina “MostrarPaquete” y “RegistrarPaquete”.


Logistica
Ver envios
Contacto
Iniciar sesion
Ingrese los datos del paquete
Formulario de Registro
Peso
Por ejemplo: 5kg...
Volumen
5°
Fecha de ensamble
08/10/2026
Enviar


Logistica
Ver envios
Contacto
Iniciar sesion
Paquetes:
Nro
Id paquete
Peso
Volumen
Fecha de ensamble
1
1
5kg
20 cm cubicos
24/11/23
2
2
500gr
12cm cubicos
21/2/26
3
3
120gr
10cm cubicos
1/10/26

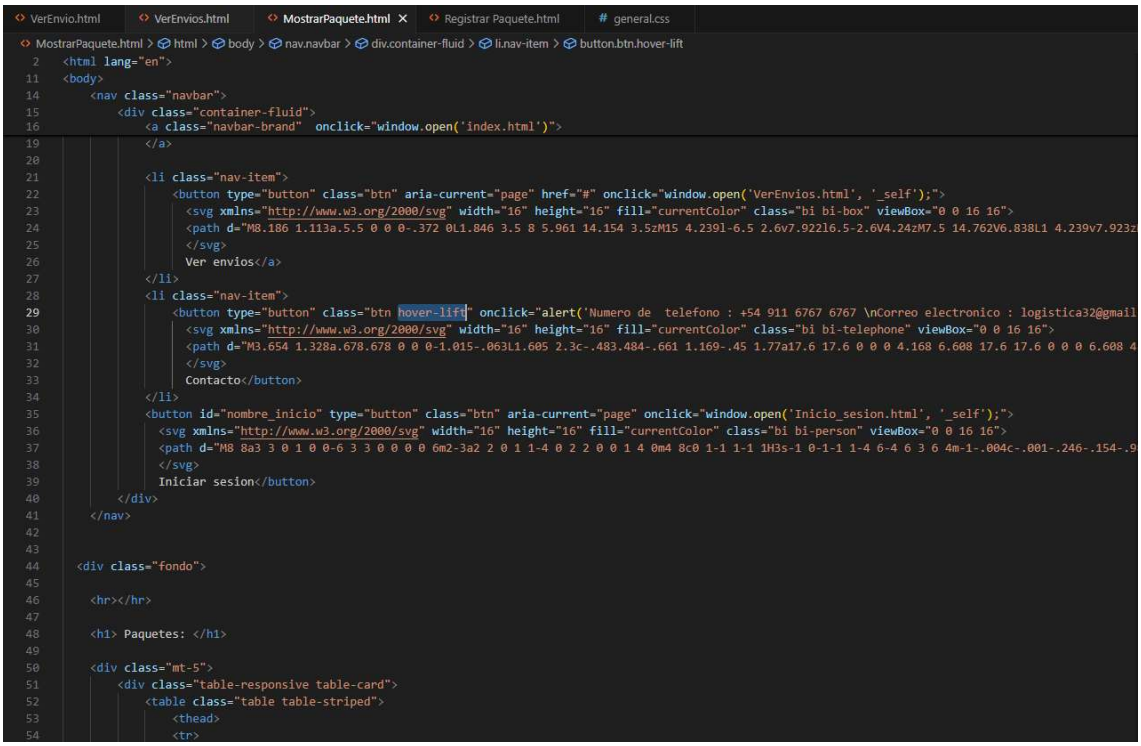
Fecha: 19/10/26

Ubicación: Escuela : Aula

Hora: 11:50

Actividades Realizadas:

Avance de vuelta con el diseño, ya que ayer enseñaron la conexión de HTML a base de datos, pero al no haber explicación, no entendí nada. Al diseño le agregue los “hover” a todos los botones.



Asi quedo:

PEDRAZA:**ACLARACIÓN DE SALTO DE FECHA:** aquí se ve un salto importante de fechas (**10/10 - 19/10**). Esto se debió a que me enfoque todos esos días al proyecto que mandó aparte el profesor Gabriel Tuinyla para la materia Proyecto Informático 2.
Durante esa semana, Saragusti nos asigno hacer cada uno partes de la pagina HTML, del codigo JAVA (ya conectado a HTTP, no me ofrecieron participar en eso y, de un dia para el otro, ya estaba todo hecho sin que se me informe), y pisos de la maqueta virtual.

A mi se me asigno realizar esto:

Pedraza:
10-REGISTRAR TRANSPORTISTA (Admin)
11-CONSULTAR TRANSPORTISTA (Admin)
-CONSULTAR TRANSPORTISTA (Cliente) (incluye JAVA y HTML)

Pedraza:
9,10,11 (pisos de la maqueta)

Departamento	Dimensiones (ancho x largo, alto 2.8m)	PCs	Teléfonos IP	Impresoras red	Plotters	Impresoras 3D	Cámaras IP
9. Sistemas y Trazabilidad (RFID/GPS	17.6m x 11.7m (alto 2.8m)	28	28	1	0	2	3
10. Seguridad Patrimonial	5.0m x 5.0m (alto 2.8m)	23	23	1	2	1	3
11. Recursos Humanos	16.1m x 27.0m (alto 2.8m)	115	115	1	0	0	3

20/10
ROSATO:Se llevaron a cabo las tareas de integración y establecimiento de conexiones entre la lógica de la aplicación en Java y las interfaces

desarrolladas en HTML, trabajando de manera conjunta con la compañera Xiomara Ureña. Durante esta jornada de desarrollo colaborativo, se contó además con aportes técnicos y revisiones por parte de los compañeros Santiago Goldfard y Benjamin Saragusti, quienes colaboraron en la optimización del flujo de datos y la verificación de la comunicación.

codigo :

```
html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Editar paquete</title>

   <link
href="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/css/bootstr
ap.min.css" rel="stylesheet"
integrity="sha384-B/GM4XqrwHnWXNOWMbloTmrYXZg10cakYGmpf
sR/bbzQ6JAJI4ihuyADKLnBgrCe" crossorigin="anonymous">

   <script type="module"
src="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/js/bootstrap
.bundle.min.js"
integrity="sha384-1a/pXj49ZQ1aHEmrJ+gMw1otqoVsYwlEnlD8mIfY2T
V03r20Y0CN7uqx1tQogjPL" crossorigin="anonymous"></script>
    <link rel="stylesheet" href="general.css">
</head>

<body>
```

```
    <div class="fondo-blanco">

    <!-- BARRA DE BUSQUEDA -->
    <nav class="navbar fondo-blanco">
        <div class="container-fluid">
            <a class="navbar-brand texto-negro"
onclick="window.open('index.html')">

            Logistica
            </a>

            <li class="nav-item">
                <button type="button" class="btn hover-lift" aria-current="page"
href="#" onclick="window.open('VerEnvios.html', '_self');">
                <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-box" viewBox="0 0 16 16">
                <path d="M8.186 1.113a.5.5 0 0 0-.372 0L1.846 3.5 8 5.961
14.154 3.5zM15 4.239l-6.5 2.6v7.922l6.5-2.6V4.24zM7.5
14.762V6.838L1 4.239v7.923zM7.443.184a1.5 1.5 0 0 1 1.114 0l7.129
2.852A.5.5 0 0 1 16 3.5v8.662a1 1 0 0 1-.629.928l-7.185 2.874a.5.5 0 0
1-.372 0L.63 13.09a1 1 0 0 1-.63-.928V3.5a.5.5 0 0 1 .314-.464z"/>
                </svg>
                Ver envios</a>
            </li>
            <li class="nav-item">
                <button type="button" class="btn hover-lift"
onclick="alert('Numero de  telefono : +54 911 6767 6767 \nCorreo
electronico : logistica32@gmail.com  \nNombre de contacto:Logistica 32
Equipo 6')">
```

```
                <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-telephone" viewBox="0 0 16
16">
                <path d="M3.654 1.328a.678.678 0 0 0-1.015-.063L1.605
2.3c-.483.484-.661 1.169-.45 1.77a17.6 17.6 0 0 0 4.168 6.608 17.6 17.6 0
0 0 6.608 4.168c.601.211 1.286.033 1.77-.45l1.034-1.034a.678.678 0 0
0-.063-1.015l-2.307-1.794a.68.68 0 0 0-.58-.122l-2.19.547a1.75 1.75 0 0
1-1.657-.459L5.482 8.062a1.75 1.75 0 0 1-.46-1.657l.548-2.19a.68.68 0 0
0-.122-.58zM1.884.511a1.745 1.745 0 0 1 2.612.163L6.29
2.98c.329.423.445.974.315 1.494l-.547 2.19a.68.68 0 0 0 .178.643l2.457
2.457a.68.68 0 0 0 .644.178l2.189-.547a1.75 1.75 0 0 1 1.494.315l2.306
1.794c.829.645.905 1.87.163 2.611l-1.034 1.034c-.74.74-1.846
1.065-2.877.702a18.6 18.6 0 0 1-7.01-4.42 18.6 18.6 0 0
1-4.42-7.009c-.362-1.03-.037-2.137.703-2.877z"/>
                </svg>
                Contacto</button>
            </li>
            <button id="nombre_inicio" type="button" class="btn hover-lift"
aria-current="page" onclick="window.open('Inicio_sesion.html', '_self');">
              <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-person" viewBox="0 0 16
16">
              <path d="M8 8a3 3 0 1 0 0-6 3 3 0 0 0 0 6m2-3a2 2 0 1 1-4 0 2 2
0 0 1 4 0m4 8c0 1-1 1-1 1H3s-1 0-1-1 1-4 6-4 6 3 6
4m-1-.004c-.001-.246-.154-.986-.832-1.664C11.516 10.68 10.289 10 8
10s-3.516.68-4.168 1.332c-.678.678-.83 1.418-.832 1.664z"/>
              </svg>
              Iniciar sesion</button>
        </div>
    </nav>
```

```
  <div class="separador">

  </div>

  <h1 class="texto-negro"> Ingrese los datos del vehiculo </h1>

  <br>

    <div class="container mt-5">
    <div class="row justify-content-center">
      <div class="col-md-5 col-lg-3 test_form">
        <div class="card shadow">
          <div class="card-header bg-primary text-white">
            <h4 class="texto-formulario mb-0 text-center" >Formulario de
Registro</h4>
          </div>
          <div class="card-body">
            <form id="formulario vehiculo">

              <input type="hidden" id="id" name="id">
  </div>

  <h1 class="texto-negro"> Ingrese los datos del Vehiculo </h1>

  <br>

    <div class="container mt-5 ">
    <div class="row justify-content-center">
      <div class="col-md-5 col-lg-3 test_form">
```

```
        <div class="card shadow">
          <div class="card-header ">
            <h4 class="texto-formulario mb-0 text-center texto-blanco"
>Formulario de Registro</h4>
          </div>
          <div class="card-body">

            <div class="mb-3">
              <label for="precio" class="form-label">precio</label>
              <input type="number" class="form-control" id="precio"
placeholder="100$" required>
            </div>

            <div class="mb-3">
              <label for="año" class="form-label">año</label>
              <input type="number" class="form-control" id="año"
required>
            </div>

            <div class="mb-3">
              <label for="modelo" class="form-label">modelo</label>
              <input type="text" class="form-control" id="modelo"
required>
            </div>
              <div class="mb-3">
              <label for="id_empleado"
class="form-label">id_Empleado</label>
              <input type="text" class="form-control" id="id_empleado"
required>
            </div>
```

```
              <div class="mb-3">
               <label for="Marca" class="form-label">Marca</label>
               <input type="text" class="form-control" id="Marca" required>
              </div>



              <button type="submit" class="btn btn-primary w-100
hover-lift">Enviar</button>
              <p id="mensaje"></p>
             </form>
            </div>
           </div>
          </div>
         </div>

        </div>



        <script>
        const formulario = document.getElementById("formulario vehiculo");
        const mensaje = document.getElementById("mensaje");

        const parametros = new URLSearchParams(window.location.search);
        const id = parametros.get("id");

        if (!id || !/^[0-9]+$/.test(id)) {
            mensaje.className = "alert alert-danger";
            mensaje.textContent = "El ID del vehiculo  no es válido.";
            formulario.style.display = "none";
```

```
    } else {
        fetch("/api/Vehiculos")
            .then(function (respuesta) {
                if (!respuesta.ok) {
                    throw new Error("No se pudo consultar la vehiculo .");
                }
                return respuesta.json();
            })
            .then(function (vehiculos) {
                // Buscamos el elemento actual por ID
                const vehiculo = vehiculos.find(function (elemento) {
                    return String(elemento.id) === id;
                });

                // CORREGIDO: Antes evaluabas '!paquete' cuando tu variable se llamaba 'Adheridos'
                if (!vehiculo) {
                    throw new Error("No se encontró el Vehiculo .");
                }

                // CORREGIDO: Mapeo manual directo a los IDs reales de tus inputs en el HTML
                document.getElementById("precio").value = vehiculo.precio || "";
                document.getElementById("año").value = vehiculo.anio || "";
                document.getElementById("modelo").value = vehiculo.modelo || "";
                document.getElementById("id_empleado").value = vehiculo.empleado || "";
```

```
            document.getElementById("Marca").value = vehiculo.marca ||
"";
        })
        .catch(function (error) {
            mensaje.className = "alert alert-danger";
            mensaje.textContent = error.message;
            formulario.style.display = "none";
        });
    }

    formulario.addEventListener("submit", function (evento) {
        evento.preventDefault();

        // CORREGIDO: Los nombres de las propiedades deben coincidir
con lo que espera tu API de edición
        const vehiculo = {
            id_vehiculo: id,
            precio: document.getElementById("precio").value,
            año: document.getElementById("año").value,
            modelo : document.getElementById("modelo").value,
            id_empleado : document.getElementById("id_empleado").value,
            marca : document.getElementById("Marca").value
        };

        fetch("/api/Vehiculos/editar", {
            method: "POST",
            headers: {
                "Content-Type": "application/x-www-form-urlencoded"
            },
            body: new URLSearchParams(vehiculo).toString()
```

```
        })
        .then(function (respuesta) {
            return respuesta.json().then(function (datos) {
                if (!respuesta.ok) {
                    throw new Error(datos.error || "No se pudo editar.");
                }
                return datos;
            });
        })
        .then(function (datos) {
            mensaje.className = "alert alert-success";
            mensaje.textContent = datos.mensaje || "Editado correctamente.";

            setTimeout(function () {
                window.location.href = "/Mostrar Vehiculo.html";
            }, 1000);
        })
        .catch(function (error) {
            mensaje.className = "alert alert-danger";
            mensaje.textContent = error.message;
        });
    });
</script>



</body>
</html>
```

y esta es un como fue quedando el proyecto hasta llegar a la culminacion del mismo

PEDRAZA:El día **20/10,** durante un ratito en la hora de laboratorio, realice el código del servidor web en Java, correspondiente a la función de agregar empleados. Utilice una plantilla que ya había realizada previamente en el mismo código, para agregar vehículos. Solo modifique las partes necesarias para adaptarlo a Empleados.

```
servidor.createContext("/api/Empleados/crear", new HttpHandler() {
  @Override
  public void handle(HttpExchange intercambio) throws IOException {
    if (!intercambio.getRequestMethod().equals("POST")) {
      enviarRespuesta(intercambio, 405,
          "{\"error\":\"Metodo no permitido\"}");
      return;
    }
    // Leer los datos que llegan desde el formulario.
    java.io.InputStream entrada = intercambio.getRequestBody();
    java.io.ByteArrayOutputStream buffer =
        new java.io.ByteArrayOutputStream();
    int dato;
    while ((dato = entrada.read()) != -1) {
      buffer.write(dato);
    }
```

```
String cuerpo = new String(
        buffer.toByteArray(),
        StandardCharsets.UTF_8
);
// Separar los datos recibidos.
java.util.Map<String, String> datos =
        new java.util.HashMap<String, String>();
String[] parametros = cuerpo.split("&");
for (String parametro : parametros) {
    String[] partes = parametro.split("=", 2);
    String clave = java.net.URLDecoder.decode(
            partes[0], "UTF-8"
    );
    String valor = "";
    if (partes.length > 1) {
        valor = java.net.URLDecoder.decode(
                partes[1], "UTF-8"
        );
    }
    datos.put(clave, valor);
}
// Obtener los valores del formulario.
String Nombre = datos.get("Nombre");
String Apellido = datos.get("Apellido");
String Dni = datos.get("Dni");
```

```
String Email = datos.get("email");
String Telefono = datos.get("telefono");
String Sector = datos.get("sector");
String Cargo = datos.get("cargo");
String Rol = datos.get("rol");
// Validar los campos.
if (Nombre == null
        || Apellido == null
        || Dni == null
        || Email == null
        || Telefono == null
        || Sector == null
        || Cargo == null
        || Rol == null
        || Nombre.trim().isEmpty()
        || Apellido.trim().isEmpty()
        || Dni.trim().isEmpty()
        || Email.trim().isEmpty()
        || Telefono.trim().isEmpty()
        || Sector.trim().isEmpty()
        || Cargo.trim().isEmpty()
        || Rol.trim().isEmpty()
        ) {
    enviarRespuesta(intercambio, 400,
            "{\"error\":\"Todos los campos son obligatorios\"}");
```

```
                return;
            }
            int dni;
            int telefono;
                    try {
                dni = Integer.parseInt(Dni);
                telefono = Integer.parseInt(Telefono);
            } catch (NumberFormatException e) {
                enviarRespuesta(intercambio, 400,
                        "{\"error\":\"El DNI y telefono deben ser numeros enteros\"}");
                return;
            }
            // Preparar la consulta SQL.
           String sql = "CALL agregar_empleado(?, ?, ?, ?, ?, ?, ?, ?, ?)"; // Mantiene los 7 '?'
try (
    Connection conexion = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/logistica32",
            "root",
            ""
    );
     CallableStatement sentencia = conexion.prepareCall(sql);
) {
    sentencia.setString(1, Nombre);
    sentencia.setString(2, Apellido);
    sentencia.setInt(3, dni);
```

```
    sentencia.setString(4, Email);
    sentencia.setInt(5, telefono);
    sentencia.setString(6, Sector);
    sentencia.setString(7, Cargo);
    sentencia.setString(8, Rol);

    // 1. Corregido: Registrar el séptimo parámetro como SALIDA (Ejemplo si devuelve
Texto)
    sentencia.registerOutParameter(7, java.sql.Types.VARCHAR);

    sentencia.execute();

    // 2. Opcional: Puedes leer lo que te devolvió la base de datos
    String respuestaBD = sentencia.getString(7);

    enviarRespuesta(intercambio, 201,
            "{\"mensaje\":\"Empleado guardado correctamente\", \"detalle\":\"" + respuestaBD +
"\"}");

} catch (SQLException e) {
    e.printStackTrace();
    enviarRespuesta(intercambio, 500,
            "{\"error\":\"Error al guardar en la base de datos\"}");
}
```

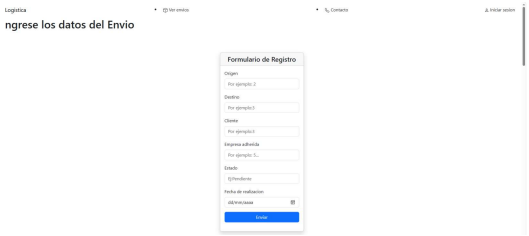
```
        }
    });
```

SARAGUSTI:El 20 de octubre termine la ventana por completo, con todas las tablas funcionales, quedando como 1000 lineas de codigo, tuve que usar para estas la IA de google debido a que el código del profesor me tiraba errores desconocidos, pero solo la implemente para poder incluir los procedimientos en lugar de los selects, y revise antes

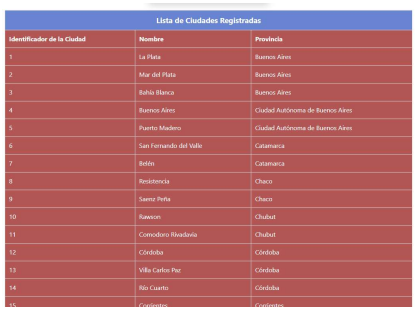
```
    while (resultado.next()) {
        if (!primero) {
            json.append(",");
        }
        json.append("{");
        json.append("\"id\":").append(resultado.getInt("ID_rt")).append(",");

        // CORRECCIÓN: Se agregaron las comillas \" al inicio y al final de los textos

        json.append("\"Ticket\":\"").append(escaparJson(resultado.getString("ID_Ticket"))).append("\
        json.append("\"Reporte\":\"").append(escaparJson(resultado.getString("ID_Reporte"))).append("
        json.append("}");
        primero = false;
    }
    json.append("]");

    // CORRECCIÓN EXTRA: Asegúrate de que enviarRespuesta configure el Content-Type como "applicatio
    intercambio.getResponseHeaders().set("Content-Type", "application/json; charset=UTF-8");

    enviarRespuesta(intercambio, 200, json.toString());
    System.out.println("Llegó hasta aquí con éxito. JSON enviado.");

} catch (SQLException e) {
    e.printStackTrace();
    enviarRespuesta(intercambio, 500, "{\"error\":\"Error al consultar la base de datos\"}");
}
```

(Este fragmento, que se repite por cada tabla es lo que corrigió esta, porque originalmente me tiraba error al querer usar los procedimientos, y fue debido a que me confundi con la ubicación de las comas en el código)

También hoy implemente por completo la funcionalidad de la ventana de crear envíos, en la que también se pueden asignar vehículos y paquetes al envío en cuestión, me tomo bastante y tuve varios errores, tuve que usar inserts en algunas partes en vez de los procedimientos debido a que tiraban error y además porque el profe aclaró que no eran necesarios/obligatorios de usar.



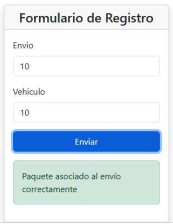
(El formulario para añadir envios, solo añade los envios, nada más, tiene la visualización con bootstrap, implementado con ayuda del profesor Nahuel)



Lista de Ciudades Registradas

Identificador de la Ciudad	Nombre	Provincia
1	La Plata	Buenos Aires
2	Mar del Plata	Buenos Aires
3	Bahía Blanca	Buenos Aires
4	Buenos Aires	Ciudad Autónoma de Buenos Aires
5	Puerto Madero	Ciudad Autónoma de Buenos Aires
6	San Fernando del Valle	Catamarca
7	Belén	Catamarca
8	Resistencia	Chaco
9	Saenz Peña	Chaco
10	Rawson	Chubut
11	Comodoro Rivadavia	Chubut
12	Córdoba	Córdoba
13	Villa Carlos Paz	Córdoba
14	Río Cuarto	Córdoba
15	Corrientes	Corrientes

(En la ventana también está la lista de ciudades para establecer como origen y destino, debido a que para el momento de la realización no sabia como hacer checkboxes y después de que me entere ya no me daba el espacio para incorporarlo)



(Formularios auxiliares con la funcionalidad de agregar paquetes a un envío en específico y agregar vehículos a un envío en específico).

Para la corrección de errores recurrir a la IA debido a que no podía consultarle al profesor, pero el código primero lo plasme yo y después lo corregi, debido a que los errores que tiraban eran solo para la base de datos y no errores tan profundos, como que por ejemplo nombres de variables, o incompatibilidades en el formato de datos.

Por eso es que aparecen // en el documento que son las correcciones hechas, pero en si lei el codigo y no lo copie y pegue.

21/10

ZOWMIR: Una de las partes en las que trabajé durante el proyecto fue el desarrollo de las páginas relacionadas con los clientes. El objetivo era que tanto el administrador como el usuario pudieran realizar distintas operaciones relacionadas con los clientes registrados en nuestra empresa.

Primero me encargué de realizar la página para registrar clientes. Para hacerla tomé como referencia la página que ya teníamos para registrar paquetes, ya que ambas tenían una estructura parecida. Lo que hice fue cambiar los campos del formulario para que coincidieran con los datos que necesitábamos guardar de un cliente, como el nombre, apellido, DNI, dirección, código postal, correo electrónico y contraseña.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title> Registrar Cliente </title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/css/boo
tstrap.min.css" rel="stylesheet"
integrity="sha384-B/GM4XqrwHnWXNOWMbloTmrYXZg10cakYGmpfsR/bbzQ6JAJI4ihu
yADKLnBgrCe" crossorigin="anonymous">

   <script type="module"
src="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/js/boots
trap.bundle.min.js"
integrity="sha384-1a/pXj49ZQ1aHEmrJ+gMw1otqoVsYwlEn1D8mIfY2TV03r20Y0CN7
uqx1tQogjPL" crossorigin="anonymous"></script>
    <link rel="stylesheet" href="general.css">
</head>
<body class="fondo-blanco">

    <div class="fondo-blanco">

     <!-- BARRA DE BUSQUEDA -->
    <nav class="navbar fondo-blanco">
        <div class="container-fluid">
            <a class="navbar-brand texto-negro"
onclick="window.open('index.html')">

            Logistica
            </a>
            <li class="nav-item">
                <button type="button" class="btn hover-lift"
aria-current="page" href="#" onclick="window.open('VerEnvios.html',
'_self');">
```

```
                <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-box" viewBox="0 0 16 16">
                <path d="M8.186 1.113a.5.5 0 0 0-.372 0L1.846 3.5 8
5.961 14.154 3.5zM15 4.239l-6.5 2.6v7.922l6.5-2.6V4.24zM7.5
14.762V6.838L1 4.239v7.923zM7.443.184a1.5 1.5 0 0 1 1.114 0l7.129
2.852A.5.5 0 0 1 16 3.5v8.662a1 1 0 0 1-.629.928l-7.185 2.874a.5.5 0 0
1-.372 0L.63 13.09a1 1 0 0 1-.63-.928V3.5a.5.5 0 0 1 .314-.464z"/>
                </svg>
                Ver Clientes</a>
            </li>
            <li class="nav-item">
                <button type="button" class="btn hover-lift"
onclick="alert('Numero de  telefono : +54 911 6767 6767 \nCorreo
electronico : logistica32@gmail.com  \nNombre de contacto:Logistica 32
Equipo 6')">
                <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-telephone" viewBox="0 0 16
16">
                <path d="M3.654 1.328a.678.678 0 0 0-1.015-.063L1.605
2.3c-.483.484-.661 1.169-.45 1.77a17.6 17.6 0 0 0 4.168 6.608 17.6 17.6
0 0 0 6.608 4.168c.601.211 1.286.033 1.77-.45l1.034-1.034a.678.678 0 0
0-.063-1.015l-2.307-1.794a.68.68 0 0 0-.58-.122l-2.19.547a1.75 1.75 0 0
1-1.657-.459L5.482 8.062a1.75 1.75 0 0 1-.46-1.657l.548-2.19a.68.68 0 0
0-.122-.58zM1.884.511a1.745 1.745 0 0 1 2.612.163L6.29
2.98c.329.423.445.974.315 1.494l-.547 2.19a.68.68 0 0 0 .178.643l2.457
2.457a.68.68 0 0 0 .644.178l2.189-.547a1.75 1.75 0 0 1 1.494.315l2.306
1.794c.829.645.905 1.87.163 2.611l-1.034 1.034c-.74.74-1.846
1.065-2.877.702a18.6 18.6 0 0 1-7.01-4.42 18.6 18.6 0 0
1-4.42-7.009c-.362-1.03-.037-2.137.703-2.877z"/>
                </svg>
                Contacto</button>
            </li>
            <button id="nombre_inicio" type="button" class="btn
hover-lift" aria-current="page"
onclick="window.open('Inicio_sesion.html', '_self');">
              <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-person" viewBox="0 0 16
16">
              <path d="M8 8a3 3 0 1 0 0-6 3 3 0 0 0 0 6m2-3a2 2 0 1 1-4
0 2 2 0 0 1 4 0m4 8c0 1-1 1-1 1H3s-1 0-1-1 1-4 6-4 6 3 6
4m-1-.004c-.001-.246-.154-.986-.832-1.664C11.516 10.68 10.289 10 8
10s-3.516.68-4.168 1.332c-.678.678-.83 1.418-.832 1.664z"/>
              </svg>
```

```
            Iniciar sesion</button>
        </div>
    </nav>

    <div class="separador">

    </div>

    <h1 class="texto-negro"> formulario para registrar clientes</h1>

    <br>

      <div class="container mt-5 ">
      <div class="row justify-content-center">
        <div class="col-md-5 col-lg-3 test_form">
          <div class="card shadow">
            <div class="card-header ">
              <h4 class="texto-formulario mb-0 text-center
texto-blanco" >Formulario de Registro</h4>
            </div>
            <div class="card-body">
              <form>
                <div class="mb-3">
                  <label for="nombre" class="form-label">Nombre</label>
                  <input type="text" class="form-control" id="nombre"
required>
                </div>


                <div class="mb-3">
                  <label for="nombre"
class="form-label">Apellido</label>
                  <input type="text" class="form-control" id="apellido"
required>
                </div>

                <div class="mb-3">
                  <label for="email" class="form-label">Email</label>
                  <input type="text" class="form-control" id="email"
required>
                </div>

                <div class="mb-3">
```

```
                    <label for="contrasenia"
class="form-label">Contrasenia</label>
                    <input type="text" class="form-control"
id="contrasenia" required>
                </div>



                <button type="submit" class="btn btn-primary w-100
hover-lift">Enviar</button>
            </form>
          </div>
        </div>
      </div>
    </div>



    </div>
```

cambié el nombre de los datos que requieren para registrar un paquete a los que se requieren para los clientes. luego cambié el icono que tenia la sección “ver clientes” de una caja que tenia a una especie de icono dentro de un marco, me guié usando bootstrap para cambiar el icono, quedando así:

```
 <li class="nav-item">
                <button type="button" class="btn hover-lift"
aria-current="page" href="#" onclick="window.open('VerEnvios.html',
'_self');">
                    <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-file-person" viewBox="0 0
16 16">
                      <path d="M12 1a1 1 0 0 1 1 1v10.755S12 11 8 11s-5
1.755-5 1.755V2a1 1 0 0 1 1-1zM4 0a2 2 0 0 0-2 2v12a2 2 0 0 0 2 2h8a2 2
0 0 0 2-2V2a2 2 0 0 0-2-2z"/>
                      <path d="M8 10a3 3 0 1 0 0-6 3 3 0 0 0 0 6"/>
                      </svg>
                    Ver Clientes</a>
            </li>
```

SARAGUSTI:El 21 de octubre estuve trabajando en lo correspondiente al servidor HTTP con java, en lo referido al ingreso y a los directorios de algunos de los archivos terminados, además termine la ventana de ver un envío, por parte de un cliente, debido a que no sabemos como usar el método de buscar tuve que hardcodear algunos de los envíos que ya teníamos en un JSON pero en el auxiliar, los pase al archivo principal con la finalidad de no tener que abrir el archivo en el servidor.

(Vista de la ventana de ver envio por parte del cliente, la cual cuenta con 4 registros hardcodeados, debido a que la idea original era poderlo ver con un stored procedure y una tabla creada con triggers pero debido a la incapacidad de usar un método de buscar no hemos podido realizar esa conexión)

```
servidor.createContext("/", new HttpHandler() {

    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        String ruta = intercambio.getRequestURI().getPath();
        System.out.println("Contexto HTML/CSS: " + ruta);
        Path archivo = null;
        if (ruta.equals("/")) {
            archivo = Paths.get("src", "Web", "index.html");
        }else if(ruta.equals("/VerEnvios.html")){
            archivo = Paths.get("src", "Web", "VerEnvios.html");
        }else if(ruta.equals("/Inicio_sesion.html")){
            archivo = Paths.get("src", "Web", "Inicio_sesion.html");
        }else if(ruta.equals("/CrearEnvio.html")){
            archivo = Paths.get("src", "Web", "CrearEnvio.html");
        }else if(ruta.equals("/VerEnvio.html")){
            archivo = Paths.get("src", "Web", "VerEnvio.html");
        }else if(ruta.equals("/Crear.html")){
            archivo = Paths.get("src", "Web", "Crear.html");
        }else if (ruta.equals("/VistaGeneral.html")){
            archivo = Paths.get("src", "Web", "VistaGeneral.html");
        }else if(ruta.equals("/Registrar Paquete.html")){
            archivo = Paths.get("src", "Web", "Registrar Paquete.html");
        }
        System.out.println("Archivo: " + (archivo != null ? archivo.toAbsolutePath() : "No defi
        System.out.println("Existe: " + (archivo != null && Files.exists(archivo)));
```

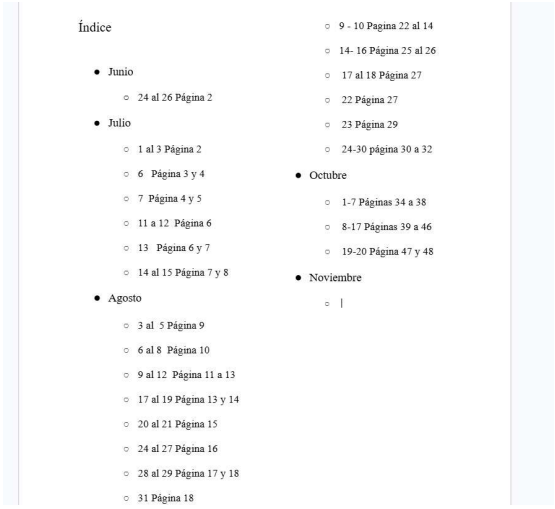
(Ahora el servidor puede acceder a más archivos, algunos no tienen enlaces activos actualmente pero se puede escribir el directorio de forma manual, en cambio los que sí tienen funcionan)

Tuve que ir modificando las direcciones que pegaban debido a que antes por ejemplo en vez de tener la URL de “/” para el índice tenía “index.html”, pero lo bueno es que aunque ahora funcione en el servidor el localStorage sigue funcionando, lo que me pregunto es si después vamos a poder mantener los auxiliares de cuentas o van a ser descartados, debido a que como dije no se puede usar el buscar, y el inicio de sesión corresponde de pegarle a la base de datos que tienen los usuarios el inicio de sesión se deberá mantener en específico a lo referido en la versión anterior.

La idea original era usar el sistema de inicio de sesión con credencial visto previamente en la versión de escritorio pero eso no es posible.

A su vez este día estuve trabajando en lo referido a la estructura de la carpeta de campo, agregando el índice, con algunos de los días trabajados, debido a que seguiré trabajando en el proyecto.

Además hoy plantearon los requerimientos que constaban para la pre entrega y última del año siendo el 28/10, siendo que varios de los aspectos a pedir ya los cumplimos.



Índice

- Junio
  - 24 al 26 Página 2
- Julio
  - 1 al 3 Página 2
  - 6 Página 3 y 4
  - 7 Página 4 y 5
  - 11 a 12 Página 6
  - 13 Página 6 y 7
  - 14 al 15 Página 7 y 8
- Agosto
  - 3 al 5 Página 9
  - 6 al 8 Página 10
  - 9 al 12 Página 11 a 13
  - 17 al 19 Página 13 y 14
  - 20 al 21 Página 15
  - 24 al 27 Página 16
  - 28 al 29 Página 17 y 18
  - 31 Página 18
  - 9 - 10 Pagina 22 al 14
  - 14- 16 Página 25 al 26
  - 17 al 18 Página 27
  - 22 Página 27
  - 23 Página 29
  - 24-30 página 30 a 32
- Octubre
  - 1-7 Páginas 34 a 38
  - 8-17 Páginas 39 a 46
  - 19-20 Página 47 y 48
- Noviembre
  - |

(Le di formato para que se vea de mejor manera y en un futuro entren todas las fechas planeadas por haber)

GOLDFARB:**Fecha: 21/10/26**
Ubicación: Escuela : Aula

Hora: 11:50

Actividades Realizadas:

Hoy Nahuel enseñó el CRUD entero de HTML + JAVA MYSQL. El lunes tuve problemas para entenderlo, asi que intente meterle ganas a entenderlo, después de un rato logre entender como funcionaba la parte de “/api/personas” y las subdivisiones “/api/personas/crear”, “/api/personas/editar”, “/api/personas/eliminar”. Decidí ayudar a mis compañeros de equipo a que entiendan

```
    });
    servidor.createContext("/api/usuarios", new HttpHandler() {
        @Override
        public void handle(HttpExchange intercambio) throws IOException {
String ruta = intercambio.getRequestURI().getPath();
System.out.println("Contexto de usuario: " + ruta);

if (!ruta.equals("/api/usuarios")) {
    enviarRespuesta(intercambio, 404, "{\"error\":\"Ruta no encontrada\"}");
    return;
}

String sql = "SELECT id_usuario, nombre, apellido, email, contrasenia FROM usuarios";
StringBuilder json = new StringBuilder();

try (
    Connection conexion = DriverManager.getConnection("jdbc:mysql://localhost:3306/logistica32","root","");
    PreparedStatement sentencia = conexion.prepareStatement(sql);
    ResultSet resultado = sentencia.executeQuery()
) {
    json.append("[");
    boolean primero = true;
```

Decidí pensarlo de esta forma. “api” es una carpeta y “usuarios” otra. Usuarios está dentro de la carpeta “api”.

Fecha: 21/10/26

Ubicación: Escuela : Aula

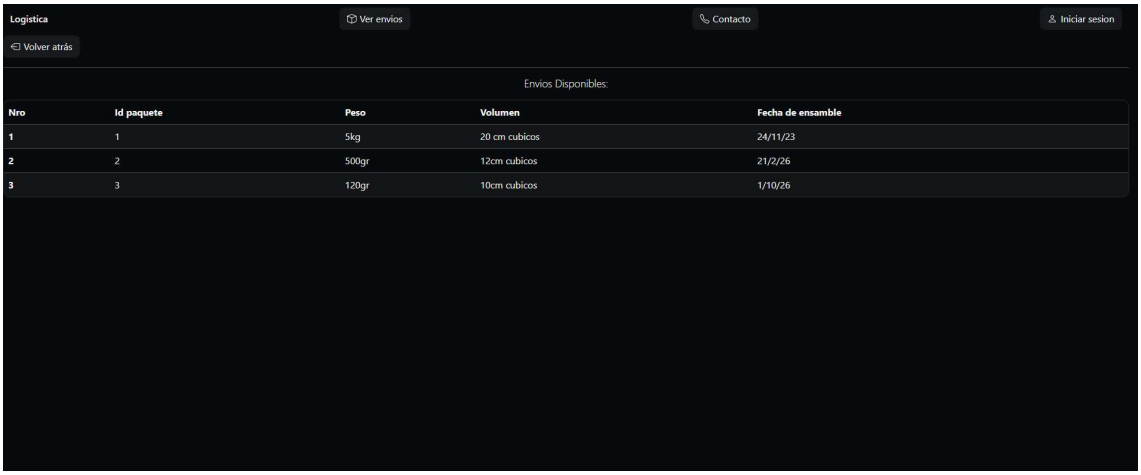
Hora: 11:50

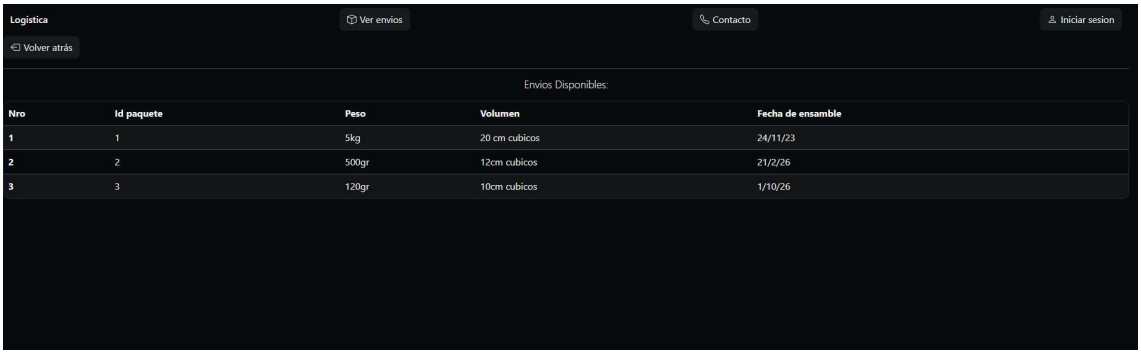
Actividades Realizadas:

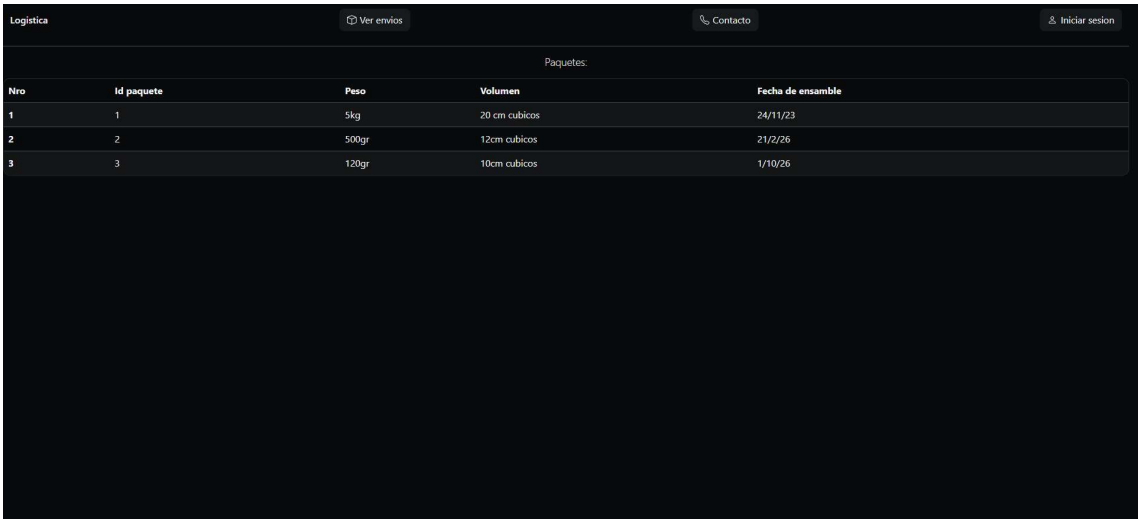
Me asigné una ventana más para hacer en la parte de HTML, la ventana “VerEnvios”. Esta página es la que le permite al administrador ver TODOS los envíos, es simplemente un select + las opciones de editar y eliminar integradas, para que este pueda manejar todos los envíos sin la necesidad de hacer redundancia de páginas.

Segui haciendo el diseño de la página antes de hacer la integración con base de datos, prefiero hacer una cosa a la vez. Le añadí un id a las tablas para cuando haga la conexión con la base de datos, pueda insertar esos valores en la tabla.

Benjamin vino a preguntarme por una duda que tenía. Según él, su página de VerEnvio (Aquella hecha para el cliente y que se busque por el id) necesitaba la opción de “buscar”. Elemento que no se nos fue enseñado, le pregunté a Nahuel y me dijo que “hardcode” los datos.





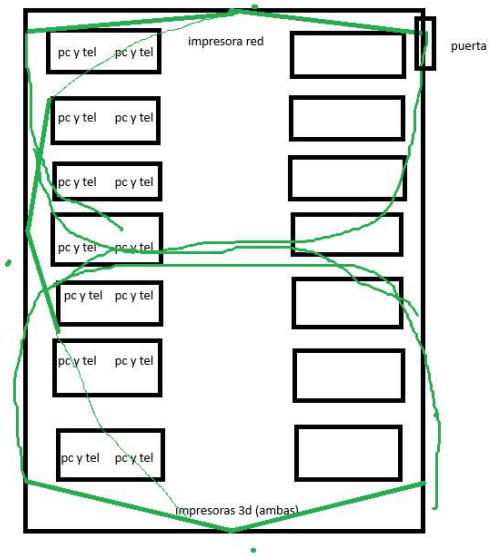


22/10

PEDRAZA:El día **22/10** en mi casa, tras haber terminado finalmente con el proyecto aparte, decidí comenzar con lo que era el diseño de la maqueta virtual. Si bien era consciente de que no era lo más urgente, ya tenía la cabeza quemada de HTML y CSS, por lo que decidí realizar esto otro primero para despejar la mente.

Saragusti me aclaró algunas cosas antes de realizar los diseños: se realizarían en la página Sketchup y que los pisos pares debían llevar router.

Primero diseñe un boceto muy primitivo en Paint, pero que me sirvió bastante (seria la habitacion vista de arriba y sin techo):



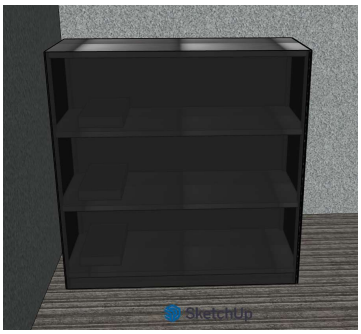
Yo no tenía absolutamente ni idea de cómo usar Sketchup, era la primera vez que escuchaba hablar de él. Por lo tanto, para empezar con el diseño del piso 9 pedí un montón de ayudas a la IA (ChatGPT), ayudas como por ejemplo “cómo mover la cámara”, “cómo hacer paredes y techo”, “cómo buscar componentes”, “cómo hacer puertas” y “cómo achicar y ocultar objetos”.

Con esas ayudas y el boceto, pude realizar todo el piso 9 ese mismo día, quedando así:

Ahi se ve una impresora 3D y PCs.

Una impresora 3D, una impresora comun de red, PCs y la puerta.

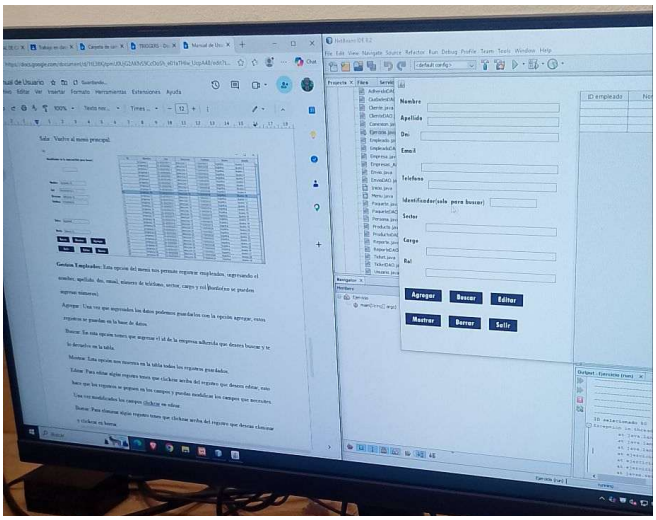
Las capturas simulan ser sacadas desde las 3 cámaras que se encuentran dentro de la habitación.



Acá van los switches (3).

23/10

SARAGUSTI:El 23 de octubre estuve colaborando en la creación del manual de usuario de la edición del programa derivada al sistema de java, debido a que podría no saberse al 100% como funcionaba cada una de las ventanas. La persona a la que le colaboré fue Xiomara Ureña, con la cual llegamos hasta la ventana de paquetes, la cual tiene funcionalidades adicionales que en su momento propuse y que en parte eran similares a las que nos enseñaron los profesores, pero que a su vez se comportan distintos. Fuimos sacando capturas de las ventanas también para dar una parte visual como guía de lo que el usuario iba a ir viendo, describiendo también el proceso del registro de datos.



(Referencia visual de cómo se veía el avance, y la metodología que fuimos usando para el uso de el manual de usuario)

GOLDFARB:**Fecha: 23/10/26**
Ubicación: Escuela : Aula

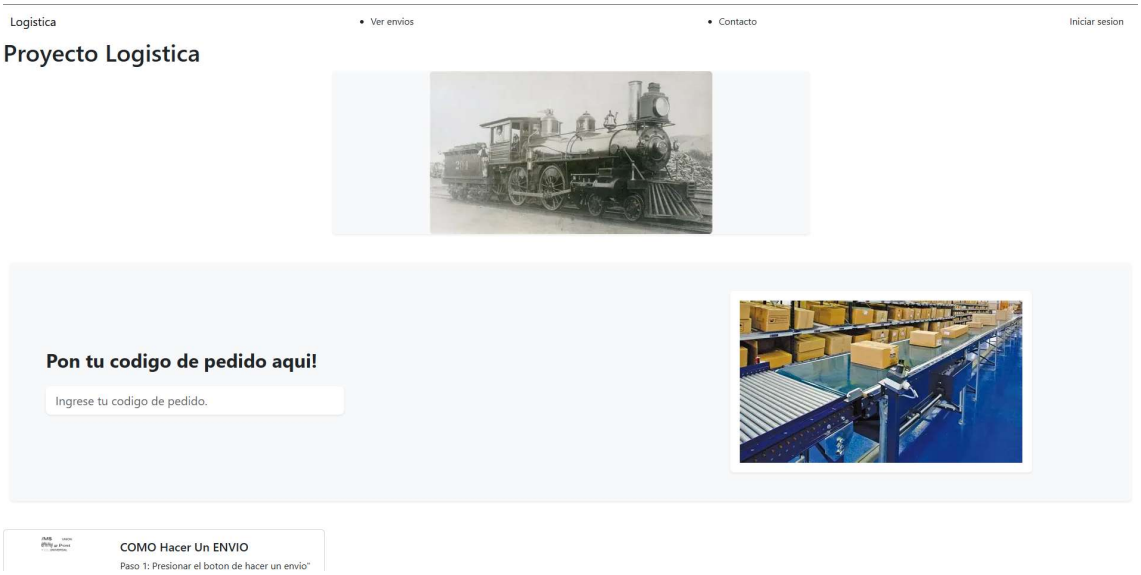
Hora: 10:50

Actividades Realizadas:

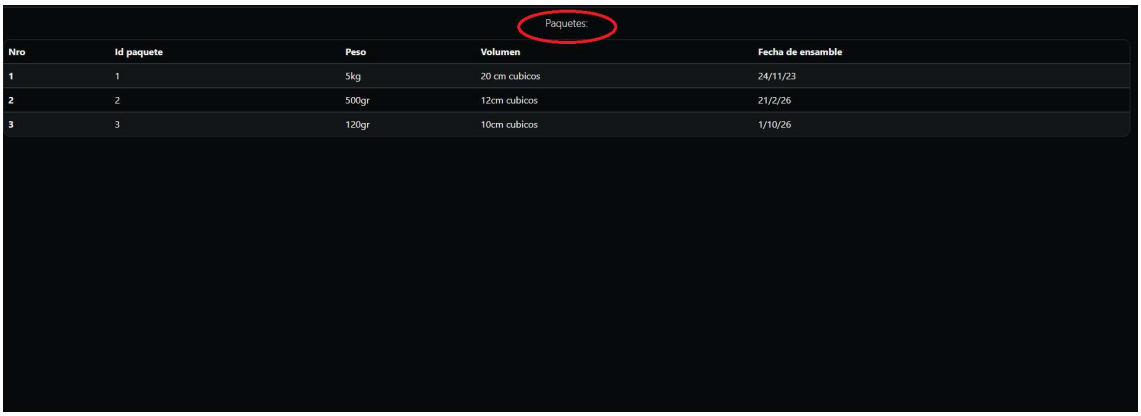
Benjamin me avisó que había que completar los manuales de usuario y de explotación. También, faltaba agregar la página web al Cisco Packet Tracer. Pero, al usar bootstrap, habría que instalarlo localmente. Sin contar que no se vería con el bootstrap aplicado, esto debido a que Cisco Packet Tracer no soporta el css de Bootstrap, así que se verá de una forma distinta a la que se vería en el HTML en sí.

Pedraza me vino a avisar lo que le quedaba pendiente de su parte correspondiente de HTML. De paso, me pregunto en qué cosa podía colaborar de los que nos quedaba pendiente en el proyecto, y después de pensarlo un momento, decidí decirle que haga el diseño del index, ya que al tener poco y nada de conexión de base de datos, no le sería problema. El index de cliente contaba con un buscar envío por ID, pero, al no haber sido enseñado el método de buscar con base de datos, no se hará. Le dije que cambie la tipografía de la página con el “h1” que se encuentra en el css, para que quede con un texto no tan genérico.

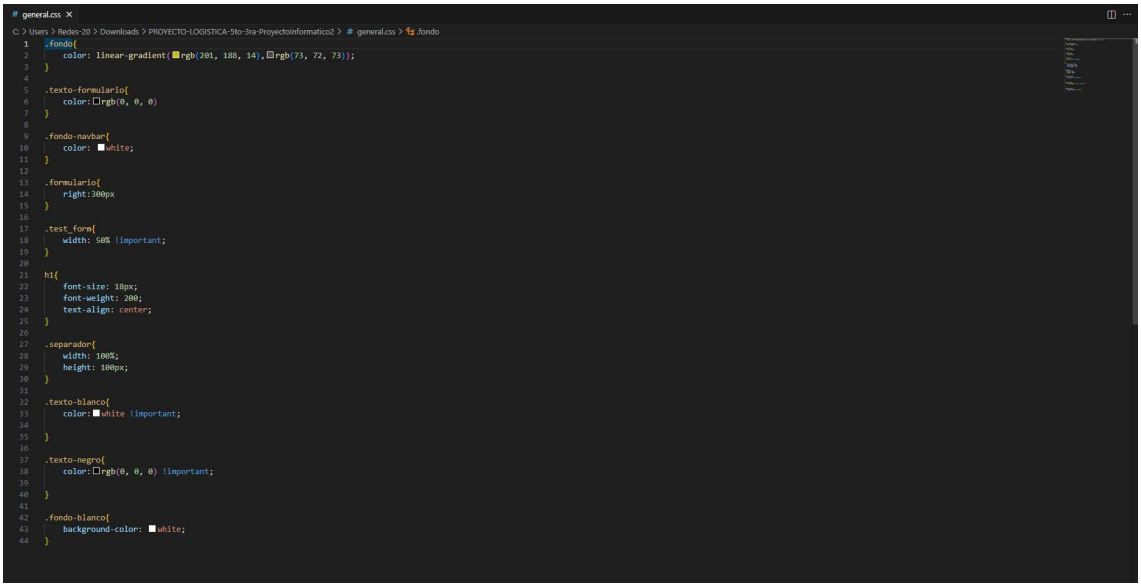
Tipografía genérica:



Tipografía deseada:

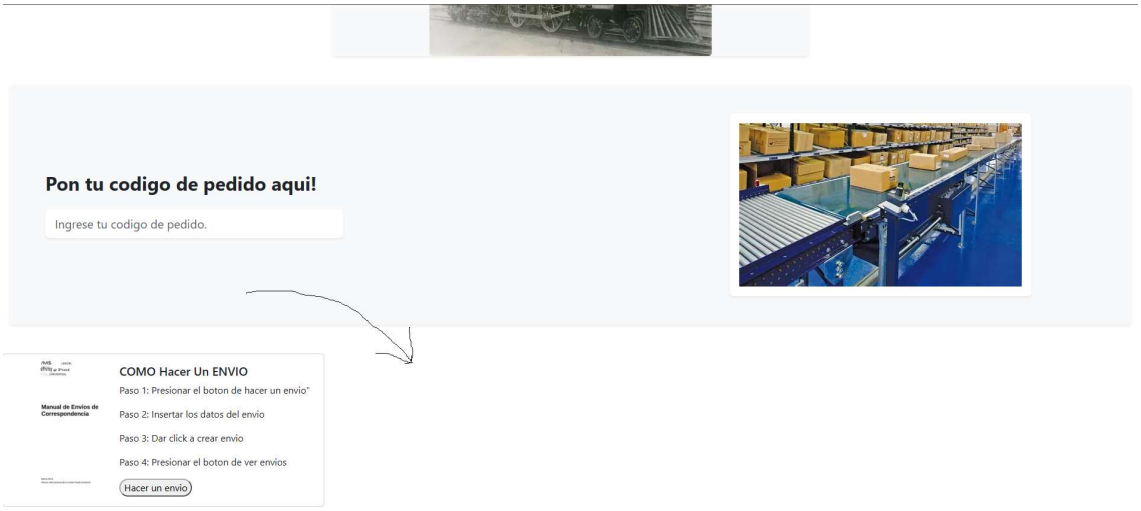


Esto se hace con el método de CSS “h1”:

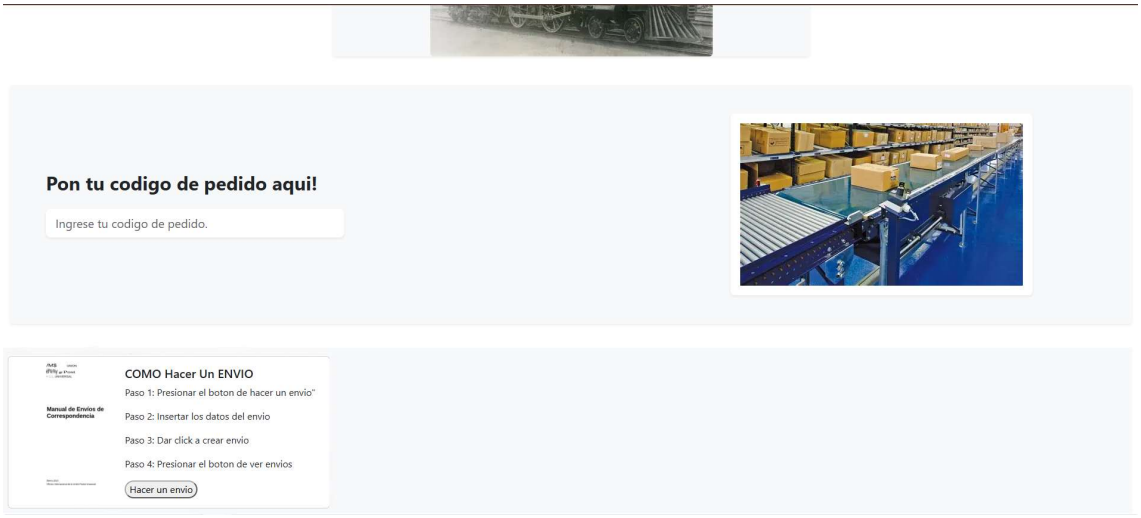


```
.fondo{
    color: linear-gradient(rgb(201, 188, 14),rgb(73, 72, 73));
}

.texto-formulario{
    color:rgb(0, 0, 0)
}

.fondo-navbar{
    color: white;
}

.formulario{
    right:300px
}

.test_form{
    width: 50% !important;
}

h1{
    font-size: 18px;
    font-weight: 200;
    text-align: center;
}

.separador{
    width: 100%;
    height: 100px;
}

.texto-blanco{
    color:white !important;

}

.texto-negro{
    color:rgb(0, 0, 0) !important;

}

.fondo-blanco{
    background-color: white;
}
```

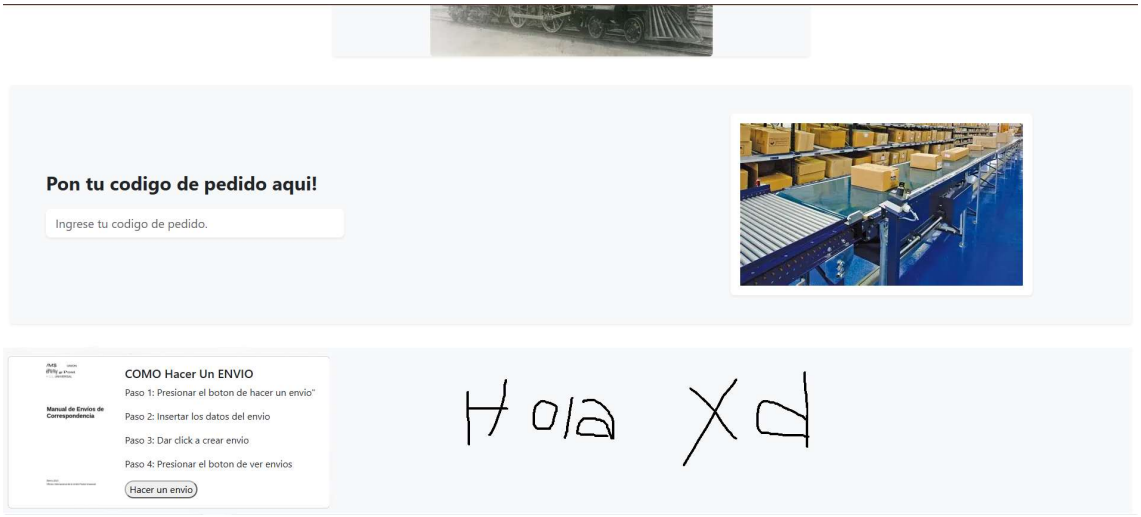
Además, le dije que añada un bloque en el fondo de “COMO Hacer Un ENVÍO”



para que quede así:



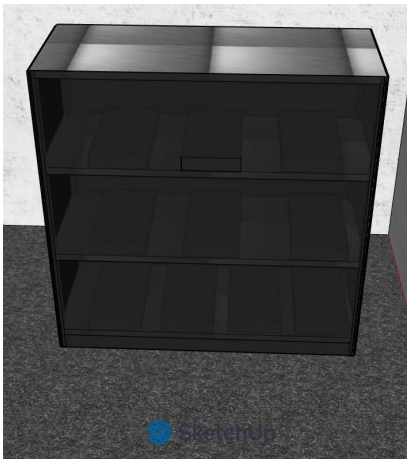
Si no, esta la opcion de ponerle texto a la derecha, de tal manera que quede asi:



PEDRAZA:El día **23/10** realicé los pisos 11 y 10 respectivamente.

Para realizar el piso 11, no tuve dificultades ni necesite de bocetos ni de ayudas de la IA o vídeos.

Piso 11:

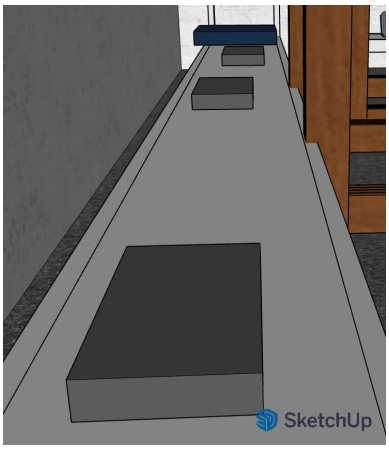

Switches (10)

Tuve una dificultad cuando quise realizar la habitación 10, ya que no sabía cómo meter 23 PCs en una habitación 5x5 mt. Pero después investigue sobre como se veían las habitaciones de seguridad patrimonial, y vi imágenes donde los monitores de las PCs están una encima de otra, en filas y columnas, por lo tanto lo realice de esa manera, usando estanterías para poner las PCs.

Piso 10:

piso 10
Guardado
Compartir
Información de la entidad
Componentes
Instructor
3D Warehouse
Materiales
Estilos
Etiquetas
Sombras
Escenas
Visualización
Suavizar/alisar
Información del modelo
Network Printer
SketchUp

piso 10
Guardado
Compartir
Información de la entidad
Componentes
Instructor
3D Warehouse
Materiales
Estilos
Etiquetas
Sombras
Escenas
Visualización
Suavizar/alisar
Información del modelo
SketchUp



Switches (3) y Router (1)

24/10

SARAGUSTI:El 24 de octubre le recordé/pregunte al líder que me avisasen en el caso de que definiera a alguien para hacer el manual de explotación, debido a que el profesor Nahuel acosta en su momento me aconsejo que apoyará a las personas que lo hicieran, para que entendieran el funcionamiento del sistema.

25/10

ROSATO:Una de mis ultimas tareas fueron realizar los triggers de mis respectivos campos

Codigo :

Devolver Stock si se desarma o elimina un Producto de un Paquete

```
DELIMITER //

CREATE TRIGGER trg_devolver_stock_productos
AFTER DELETE ON Productos_Paquetes
FOR EACH ROW
BEGIN
    UPDATE Productos
    SET stock = stock + OLD.cantidad
    WHERE ID_Producto = OLD.ID_Producto;
```

```
END //

DELIMITER ;
```

Evitar Stocks Negativos y Precios Inválidos

```
DELIMITER //

CREATE TRIGGER trg_productos_antes_insertar
BEFORE INSERT ON Productos
FOR EACH ROW
BEGIN

    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El stock inicial del producto no
puede ser negativo.';
    END IF;

    IF NEW.precio <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El precio del producto debe ser
mayor a cero.';
    END IF;
END //

DELIMITER ;
```

Control de Stock y Precios al Modificar

```
DELIMITER //

CREATE TRIGGER trg_productos_antes_actualizar
BEFORE UPDATE ON Productos
```

```
FOR EACH ROW
BEGIN
    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: La actualización dejaría el stock en
negativo.';
    END IF;
    IF NEW.precio <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El precio modificado debe ser
mayor a cero.';
    END IF;
END //

DELIMITER ;
```

Validar Peso, Volumen y Fecha

```
DELIMITER //

CREATE TRIGGER trg_paquetes_antes_insertar
BEFORE INSERT ON Paquetes
FOR EACH ROW
BEGIN
    IF NEW.peso <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El peso del paquete debe ser mayor
a cero.';
    END IF;

    IF NEW.volumen <= 0 THEN
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Error: El volumen del paquete debe ser
mayor a cero.';
    END IF;

    IF NEW.fecha_ensamble > CURDATE() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: La fecha de ensamble no puede ser
una fecha futura.';
    END IF;
END //

DELIMITER ;
```

Control de Datos al Modificar

```
DELIMITER //

CREATE TRIGGER trg_paquetes_antes_actualizar
BEFORE UPDATE ON Paquetes
FOR EACH ROW
BEGIN
    IF NEW.peso <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El peso modificado debe ser mayor
a cero.';
    END IF;

    IF NEW.volumen <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El volumen modificado debe ser
mayor a cero.';
    END IF;

    IF NEW.fecha_ensamble > CURDATE() THEN
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Error: No se puede asignar una fecha de ensamble futura.';
    END IF;
END //

DELIMITER ;
```

26/10

SARAGUSTI:El 26 de octubre tuvimos la revisión de octubre, en la que nos anunciaron las cosas que faltaban y teníamos que mejorar, que sería terminar la página html(para este punto mis partes ya estaban hechas), terminar la maqueta(mis partes también ya estaban hechas, previamente detallado en las páginas anteriores), arreglar detalles visuales de JAVA, que corresponden al tamaño de los campos y botones, además de la organización de estos últimos debido a que estaban volando en vez de estar organizados.

Por parte de las páginas html, las mias ya estan en el zip con las versiones conectadas a la base de datos, y funcionan mostrando, excepto como había aclarado la de ver envio que tiene los envios hardcodeados, por el no saber de usar el método de buscar.

No me acuerdo si lo siguiente que voy a comentar lo escribí pero por si acaso lo voy a volver a escribir:

Las ventanas de JAVA tienen un tema, con respecto a la función de borrar y sus procedimientos respectivos, debido a que estos últimos tienen una verificación de dependencias, lo que significa que antes de borrar uno van a pegarle a las tablas en las que podría estar este registro y si lo encuentran van a prohibir el borrado. Esto igual no es solo de nuestros procedimientos, naturalmente la base de datos lo hace solo, pero el mensaje que da cuando no se usa procedimientos está en inglés y es más directo al punto que el nuestro.

Lo que significa que en el caso que, por ejemplo quieras borrar un cliente vas a tener que seguir los siguientes pasos:

-Borrar los Paquetes del Envio del cliente

-Borrar los vehículos del Envío

-Borrar el Envío

-Borrar la conexión del Reporte correspondiente al ticket en el que está el cliente

-Borrar el ticket del cliente

Y por último borrar el cliente.

Esto ya viene de fábrica con los procedimientos realizados por cada uno de los integrantes, aunque si usamos consultas directas pasaría lo mismo debido a que SQL hace esa verificación sin necesidad de hacerla manualmente.

Otra opción que se podría haber realizado era que el procedimiento de borrar clientes borrará todos los datos que relacionaban a este con otras tablas, pero no se planteó debido a que iba a ser más complicado, y en el uso real sería un potencial peligro de seguridad borrar tantos campos de una sola vez. Si se borran manualmente hay un tiempo de reacción que puede permitir evitar borrarlo por error, ya que hay que ir siguiendo paso por paso hasta terminar de cerrarlo por completo.

GOLDFARB:**Fecha: 26/10/26**
Ubicación: Escuela : Aula

Hora: 10:50

Actividades Realizadas:

Hoy llamaron a mi grupo y nos hicieron la pre entrega de octubre. Nos corrigieron algunas cosas como por ejemplo el diseño de la página web y un error con la sección de ingresar un código de envío. Ya que este te manda a una página que no debería. Debería mandarte a “VerEnvio” que es la pagina hecha para mostrar envíos por el ID. Sin embargo, esta te manda a “VerEnvios”, que es la pagina hecha para ver TODOS

Fecha: 26/10/26

Ubicación: Escuela : Aula

Hora: 15:00

Actividades Realizadas:

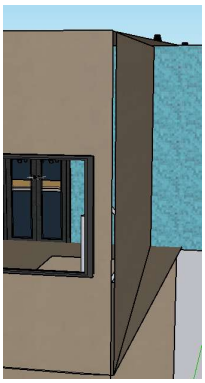
En el turno tarde, me puse a laburar en el diseño de la maqueta, pero al ver que la página no toleraba y se crasheaba, pasé a hacer la conexión con html y sql. Agustin me pidió ayuda de como hacer su conexión, le avise que benja, al haber hecho la consulta general, tenía el código ya hecho. Agustin sólo tenía que cambiar algunas variables.

PEDRAZA: El día **26/10,** mientras miraba como iba quedando el proyecto, me di cuenta que las ventanas del programa en JAVA estaban bastante desprolijas, con los labels, textfields y botones muy desordenados, y con tamaños distintos. Por lo tanto, le consulté al líder Goldfarb y a mi compañero Saragusti si podía realizar modificaciones con el fin de hacer las ventanas más cómodas visualmente y, por lo tanto, más fáciles de utilizar. Me dijeron que sí, y por ende, ese mismo día comencé con eso.
Esto transcurrió durante el horario de Base De Datos. Más tarde, cuando llegué a mi casa quise avanzar, pero tuve un problema: mi versión de NetBeans no tenía incluidas las fuentes de letras utilizadas en las ventanas originales, por lo que se desemparejaron y volvieron a verse desprolijas, asi que decidi trabajar en eso solo con las PCs del colegio.

27/10

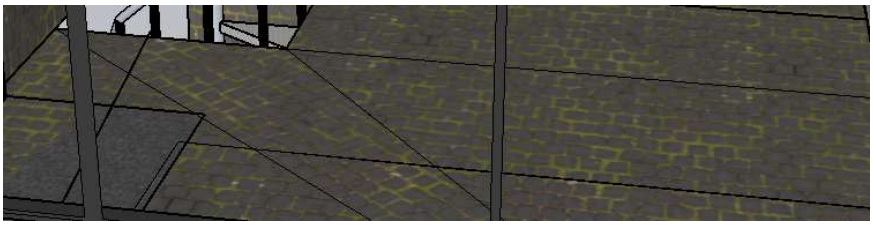
SARAGUSTI: El 27 de octubre estuve centrado en el aspecto de la maqueta, realizando las conexiones entre pisos, y colocando en su lugar las oficinas realizadas hasta el momento, siendo las oficinas 9, 10, 11, 16, 18 y 20.(Las que hice yo y las que hizo Pedraza), en principio la idea era hacer un pasillo que conecta todas las habitaciones, porque parecía más fácil la idea así. Pero debido a un cambio de ideales, por mi parte debido a la cantidad de oficinas realizadas hasta el momento, cambié la modalidad para que quedase con forma de

edificación, que tenía más sentido para la cantidad de oficinas. Tuve dificultades en algunas partes, como al conectar las habitaciones 10 y 11, debido a que tenían un pasillito de suelo, que terminó atravesando el suelo de el conector de las dos habitaciones, osea le piso 6, pero igual no se nota tanto debido a la sobretexturacion. Otro problema fueron algunas paredes las cuales quedaron torcidas y no las pude arreglar, debido a las dimensiones rotas de la maqueta.

 (Ejemplo de la rotura de la línea, se puede apreciar como la pared se desfigura, esto no se puede arreglar debido a que sí intento de corregirlo la pared se va a ir lejos, y reemplazarla tampoco pareció arreglarlo)

Además de este tema con la pared, hubo otro problema con uno de los entrepisos, que hizo que las paredes quedarán divididas por líneas diagonales, no entendí muy bien el problema y cuando trataba de solucionarlo se rompió aún más, por lo que intenté no maniobrar.

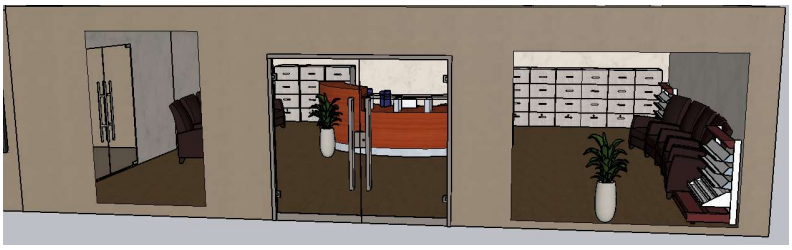
(Esas líneas negras, en diagonales y rectas , no son aproposito, el programa las hizo de manera automática, y al tratar de borrarlas todo el diseño se corrompe, por lo que intente de desistir por esa sección)

 (Lo que decía del pasillo que había quedado, debido a esto mucho no se nota pero se superponen en los planos el piso natural y el pasillo sin poder borrar, intento de borrarlo pero al hacerlo se elimina todo el piso, por lo que no continúe por ese lado y preferí concentrarme en otros aspectos)

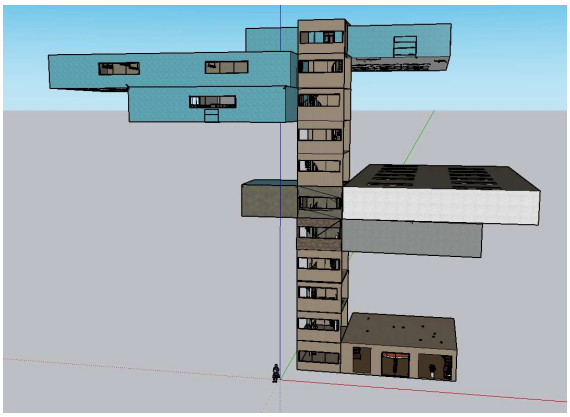
Otro error sucedido sin posibilidad aparente de corrección es el techo y el piso de algunas maquetas, que se debe a que el techo de una superpone al otro, y viceversa, lo que hace que al girar la cámara falle solo, pero no genera tanto problema si no lo miras tan profundamente, y para los hechos de la entrega no presenta algún problema.

(El techo de abajo superpone sobre el piso de arriba, solo en algunos ángulos igualmente, pero no presenta mayor problema que la unión de los dos materias en un degradado, si se hubiera presentado anteriormente se hubiera borrado el techo de todas antes de ver un error del tipo, como no sucedió lo tuve que dejar así).

La creación de la recepción fue idea de goldfarb, la cual hice de 10*10m y 2,8m de altura, con la finalidad de tener una entrada linda en lugar de solo una puerta, esta sala cuenta con flores, revisteros, asientos, archiveros y una pantalla, elementos típicos de cualquier sala de registrarse, además hay tres computadoras, tachos de basura y sillas giratorias, para lograr una buena atención al cliente, las compus son laptops.

 (Vista desde afuera del establecimiento, con las ventanas, lámparas, plantas, revisteros, no se ven desde acá pero hay carpetas compus detrás del mostrador e interruptores para prender las luces).

(Como se visualiza el mostrador de la recepción, con las plantas en el fondo, las carpetas con los datos de los envíos propuestos por los clientes y lo necesario para cumplir con el realizacimineto de todos los procedimientos).

 (Como se ven todas las oficinas colocadas, con él entre oficina con las escaleras de subida y bajada, que son escaleras en caracol, debido a que consumen menos espacio y menos polígonos, el tema es que algunas traspasan un poco las paredes pero en si no se ven mal).

Los modelos de las computadoras varían entre oficina, debido a que cada uno uso uno distinto, pero puede que no se vean de la mejor calidad por la optimización a lo referido a la cantidad de polígonos.

Todo este bloque se trata como un grupo, debido a que si no los volvía un grupo, en el caso de que alguien moviera la mínima parte, lo demás se iba a quedar quieto y eso iba a arruinar el orden por completo, por lo que une todas las partes por grupo, aunque debido a eso se creó un árbol de jerarquía entre cada uno de los artículos, esta se organiza de la siguiente manera.

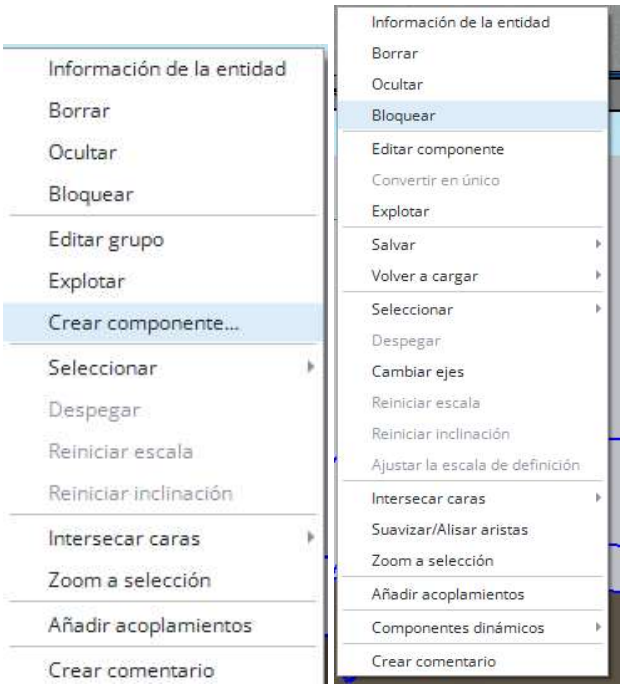
Editar grupo: Entra en la primera esfera, que puede contener algunos de los entrepisos unidos a las oficinas convencionales.

Editar grupo:Entra en la segunda esfera, dependiendo de la sección esta puede ser la última como no.

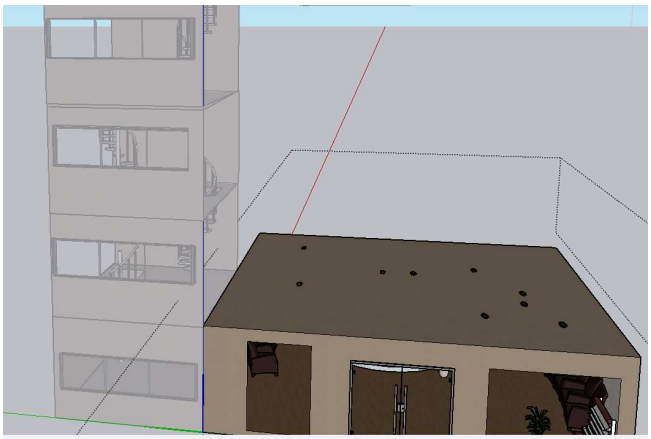
Editar grupo: Si no me confundo hay otra subdivisión, una última antes de lo demás.

Editar componente: Cuando aparece esta opción ya podes confirmar que vas a poder editar la oficina referida.

Si tratas de modificar una oficina sin entrar al componente vas a modificar todo de esa oficina, la ventaja de ir entrando a cada una de las secciones es que las demás se vuelven inaccesibles al modificar, por lo que sabes que parte fue la que seleccionaste.



(A la izquierda se aprecia como se ven los grupos al ser clickeados con el click derecho, a la derecha es cuando ya pasaste las barreras del grupo y puedes acceder directamente al componente)


(Al tocar el botón de editar componente todo lo demás se pone gris y se ve de esta manera)

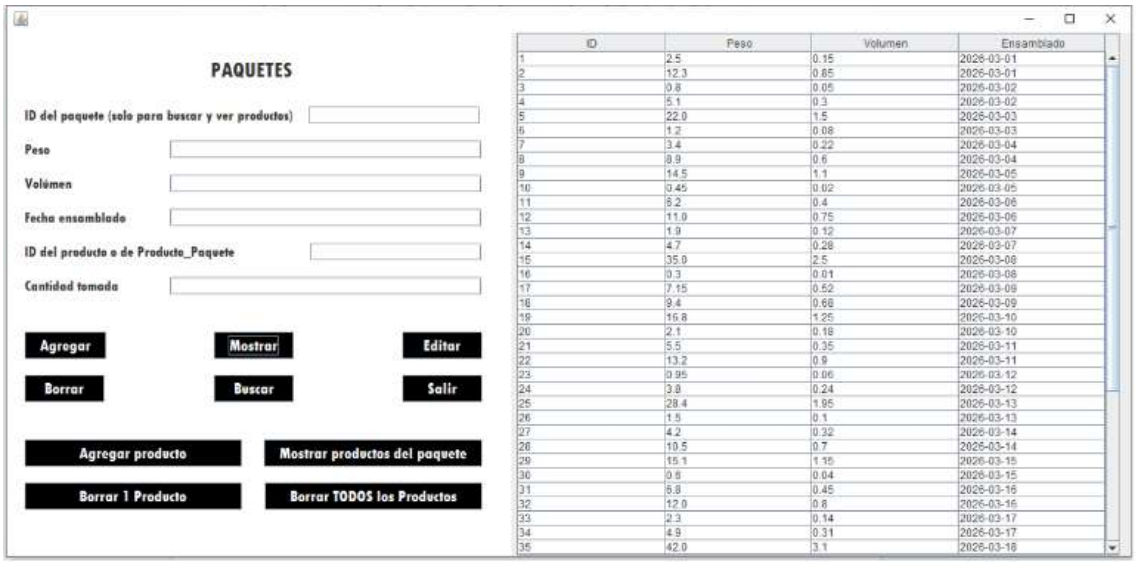
Para volver atrás solo haces click en una sección que no tenga nada y vas haciendo click en las opciones, salir del grupo/componente hasta salir por completo.

Los vacíos de los costados de los entrepisos están para ser rellenados con las paredes de las maquetas de mis compañeros para evitar corrupciones de paredes como las que le pasaron al techo y a lo demás.

Este dia tambien colabore con goldfarb para hacerle las correcciones a su código de editar, borrar,y mostrar, lo que pasaba es que había nombrado a la variable paquete como paquetes cuando ya había un array con ese nombre, además faltaban piezas de código que permiten la conexión con la base de datos de mysql, por lo que tuve que agregarlas/ corregirlas para que quedase bien. Además hay un input de id que queda oculto por defecto que él no lo había puesto por lo que lo tuve que colocar también, y luego de estas modificaciones el código quedo funcional y tambien corria, permitiendo borrar paquetes, editarlos y que despues te devolviera al menú mostrando sus datos actualizados, también el tema que tenía la pagina era que debido al input faltante, los datos viejos no se mostraban en los parámetros. Me explico, la ventana de editar funciona mostrando los datos a editar, con los valores que están actualmente, y ahí vos lo unico que tenes que hacer es modificarlos y darle a Enviar, antes de las revisiones no mostraba ni eso, porque no tomaba 1 paquete si no que los tomaba todos.

PEDRAZA: Al día siguiente, el **27/10,** terminé de mejorar las ventanas que me faltaban. Todas tienen la misma dinámica. Lo que si, Saragusti me dijo que no modificara la ventana inicial (en la que inicias sesión). Supongo que la modificara el.

Ejemplo de ventana corregida por mi:



Posteriormente, una vez que llegué a mi casa, realice los triggers que tenía pendientes desde hacía algunas semanas. No eran urgentes, pero tenía que hacerlos igual.
Realice los triggers (11 en total) correspondientes a las mismas tablas de las cuales había realizado anteriormente los procedures (Empleados, que realicé 6 y Vehículos, que realicé 5).

Si bien las estructuras de los triggers como tal las codifique yo mismo, con ayuda de trabajos realizados por mi anteriormente en la materia de base de datos, las ideas sobre qué funcionamientos hacer, se los pedí a la Inteligencia Artificial (Gemini), ya que no estaba tan lúcido mentalmente en ese momento como para andar pensando en ellos.
Triggers Empleados:

```
-- trigger empleados: verificar que el cargo corresponda al sector transporte
drop trigger if exists verificar_sector_transporte;
DELIMITER //
create trigger verificar_sector_transporte
before insert on empleados
for each row
begin
if new.Cargo != "Transportista" and new.Cargo != "Repartidor" and new.Cargo != "Supervisor de transporte" and new.Sector = "Transporte" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
-- trigger empleados: verificar que el cargo corresponda al sector deposito
drop trigger if exists verificar_sector_deposito;
DELIMITER //
create trigger verificar_sector_deposito
before insert on empleados
for each row
begin
if new.Cargo != "Operario" and new.Cargo != "Repositor" and new.Sector = "Deposito" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
```

```
-- trigger empleados: verificar que el cargo corresponda al sector administracion
drop trigger if exists verificar_sector_administracion;
DELIMITER //
create trigger verificar_sector_administracion
before insert on empleados
for each row
begin
if new.Cargo != "Administrativo" and new.Cargo != "Gerente" and new.Cargo != "Director" and new.Sector = "Administracion" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
-- trigger empleados: verificar que el cargo corresponda al sector atencion al cliente
drop trigger if exists verificar_sector_aac;
DELIMITER //
create trigger verificar_sector_aac
before insert on empleados
for each row
begin
if new.Cargo != "Comunicador" and new.Cargo != "Registrador de reclamos" and new.Sector = "Atencion al cliente" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
```

```
-- trigger empleados: verificar que el cargo corresponda al sector finanzas
drop trigger if exists verificar_sector_finanzas;
DELIMITER //
create trigger verificar_sector_finanzas
before insert on empleados
for each row
begin
if new.Cargo != "Contador" and new.Cargo != "Administrador de ingresos" and new.Sector = "Finanzas" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
-- trigger empleados: verificar que el cargo corresponda al sector mantenimiento
drop trigger if exists verificar_sector_mantenimiento;
DELIMITER //
create trigger verificar_sector_mantenimiento
before insert on empleados
for each row
begin
if new.Cargo != "Mecanico" and new.Cargo != "Tecnico" and new.Sector = "Mantenimiento" then
signal sqlstate '45000'
set message_text = "Error. El cargo no corresponde al sector asignado.";
end if ;
end //
DELIMITER ;
```

Triggers vehículos:

```sql
-- trigger vehiculos: evitar registrar vehiculo con precio negativo
drop trigger if exists precio_negativo;
DELIMITER //
create trigger precio_negativo
before insert on vehiculos
for each row
begin
if new.precio < 0 then
signal sqlstate '45000'
set message_text = "Error. El precio no puede ser negativo";
end if;
end //
DELIMITER ;
-- trigger vehiculos: evitar registrar vehiculo con año imposible
drop trigger if exists año_imposible;
DELIMITER //
create trigger año_imposible
before insert on vehiculos
for each row
begin
if new.anio < 2006 or new.anio > year(curdate()) then
signal sqlstate '45000'
set message_text = "Error. El año de fabricacion no es valido";
end if;
end //
DELIMITER ;
```

```sql
-- trigger vehiculos: impide insertar un vehículo si el empleado asignado no existe.
drop trigger if exists chequear_empleado;
DELIMITER //
create trigger chequear_empleado
before insert on vehiculos
for each row
begin
declare existe_empleado int default null;
if new.ID_Empleado is not null then
    select ID_Empleado into existe_empleado from Empleados
    where ID_Empleado = new.ID_Empleado;

    if existe_empleado is null then
    signal sqlstate '45000'
    set message_text = "Error. El empleado asignado al vehiculo no existe.";
    end if;
end if;
end //
DELIMITER ;
```

```sql
-- trigger vehiculos: impide insertar un vehiculo si el empleado asignado no pertenece al sector "Transporte".
drop trigger if exists chequear_sector;
DELIMITER //
create trigger chequear_sector
before insert on vehiculos
for each row
begin
declare existe_empleado int default null;
declare sector_emp varchar(100);
if new.ID_Empleado is not null then
    select sector into sector_emp from Empleados
    where ID_Empleado = new.ID_Empleado;

    if sector_emp != "Transporte" then
    signal sqlstate '45000'
    set message_text = "Error. El empleado asignado no corresponde al sector 'Transporte'.";
    end if;
end if;
end //
DELIMITER ;
-- Trigger vehículos: impide eliminar un vehiculo que tenga empleados asociados
drop trigger if exists limitar_eliminar_vehiculo;
DELIMITER //
create trigger limitar_eliminar_vehiculo
before delete on Vehiculos
for each row
begin
if old.ID_Empleado is not null then
signal sqlstate '45000'
set message_text = "Error. No se puede eliminar el vehiculo ya que tiene empleados asociados.";
end if ;
end //
DELIMITER ;
```

Hubo un detalle interesante sobre uno de los triggers de vehículos, más precisamente el trigger que impide ingresar vehículos creados en un “año imposible”.

Cuando la Inteligencia Artificial me dio la idea, solo me decía que con año imposible se refería a años posteriores al año actual 2026, pero yo me di cuenta que tambien deberia haber un año maximo de antiguedad en los vehículos de una empresa logística, tanto por temas de seguridad como de modernidad. Por lo que decidí investigarlo, y lo pensé correctamente, ya que según la Ley Nacional de Tránsito N° 24.449 (artículo 53) y su decreto reglamentario N° 779/95, los transportes de cargas generales en el país no pueden superar los 20 años de antigüedad.

Tras conocer acerca de esto, en el trigger puse como año mínimo de fabricación el año 2006.

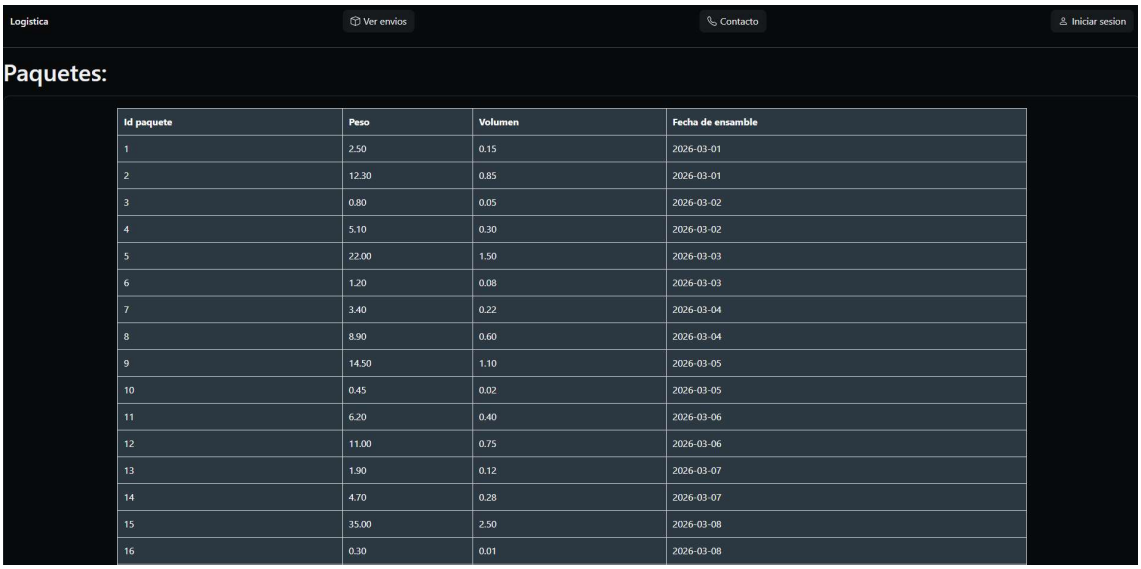
GOLDFARB:Fecha: 27/10/26

Ubicación: Escuela : Aula

Hora: 15:00

Actividades Realizadas:

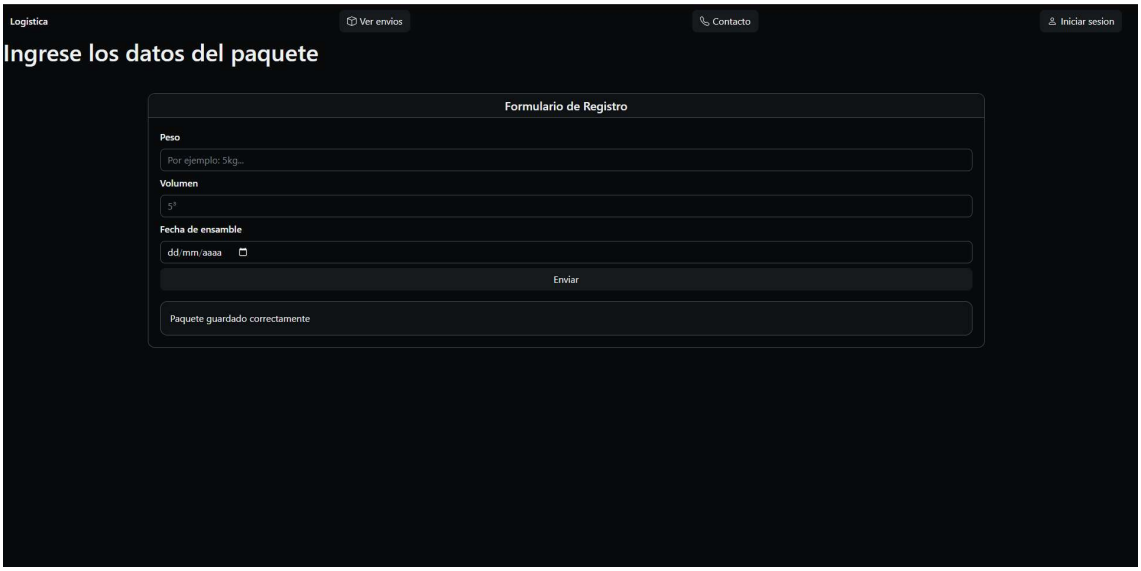
Hoy avance con la pagina de MostrarPaquete y RegistrarPaquete, después de un rato y algunos problemas con el código que copie de los ejemplos; el codigo de mostrar paquete



Id paquete	Peso	Volumen	Fecha de ensamble
1	2.50	0.15	2026-03-01
2	12.30	0.85	2026-03-01
3	0.80	0.05	2026-03-02
4	5.10	0.30	2026-03-02
5	22.00	1.50	2026-03-03
6	1.20	0.08	2026-03-03
7	3.40	0.22	2026-03-04
8	8.90	0.60	2026-03-04
9	14.50	1.10	2026-03-05
10	0.45	0.02	2026-03-05
11	6.20	0.40	2026-03-06
12	11.00	0.75	2026-03-06
13	1.90	0.12	2026-03-07
14	4.70	0.28	2026-03-07
15	35.00	2.50	2026-03-08
16	0.30	0.01	2026-03-08

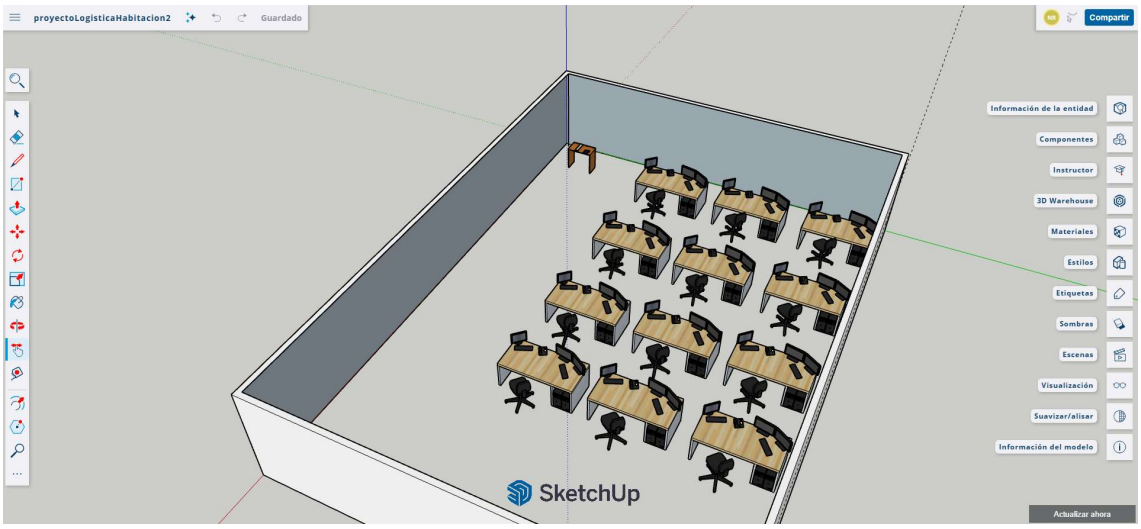
```sql
INSERT INTO Paquetes (peso, volumen, fecha_ensamble) VALUES
(2.50, 0.15, '2026-03-01'),
(12.30, 0.85, '2026-03-01'),
(0.80, 0.05, '2026-03-02'),
(5.10, 0.30, '2026-03-02'),
(22.00, 1.50, '2026-03-03'),
(1.20, 0.08, '2026-03-03'),
(3.40, 0.22, '2026-03-04'),
(8.90, 0.60, '2026-03-04'),
(14.50, 1.10, '2026-03-05'),
(0.45, 0.02, '2026-03-05'),
(6.20, 0.40, '2026-03-06'),
(11.00, 0.75, '2026-03-06'),
(1.90, 0.12, '2026-03-07'),
(4.70, 0.28, '2026-03-07'),
(35.00, 2.50, '2026-03-08'),
(0.30, 0.01, '2026-03-08'),
(7.15, 0.52, '2026-03-09'),
(9.40, 0.68, '2026-03-09'),
(16.80, 1.25, '2026-03-10'),
(2.10, 0.18, '2026-03-10'),
(5.50, 0.35, '2026-03-11'),
(13.20, 0.90, '2026-03-11'),
(0.95, 0.06, '2026-03-12'),
(3.80, 0.24, '2026-03-12'),
(28.40, 1.95, '2026-03-13'),
(1.50, 0.10, '2026-03-13'),
(4.20, 0.32, '2026-03-14'),
```

Después de un rato. Logre que la pagina de Registrar Paquete funcione. Habia un problema de tipo de variables en sql + diferencia de nombres de variables en java. Ahora ya funciona



ROSATO:27/10

Se continuó con el modelado tridimensional del proyecto de infraestructura logística utilizando la herramienta SketchUp. Durante esta jornada, el desarrollo se centró en el diseño y la renderización a escala de la segunda habitación de las instalaciones, optimizando la distribución espacial para el almacenamiento seguro y la correcta circulación de la paquetería dentro del centro de distribución.



28/10

GOLDFARB:

Fecha: 28/10/26

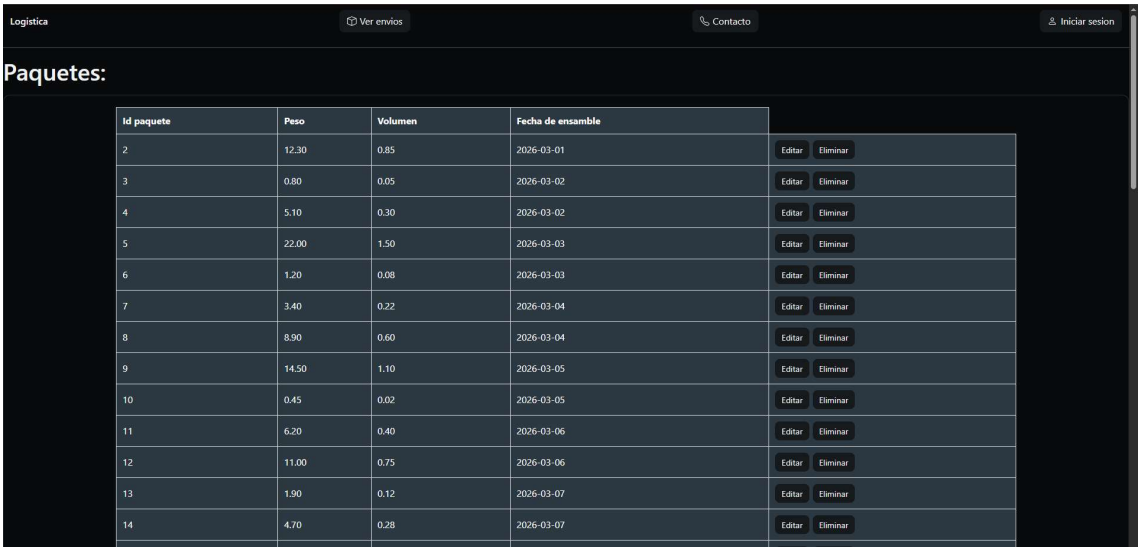
Ubicación: Escuela : Aula

Hora: 15:00

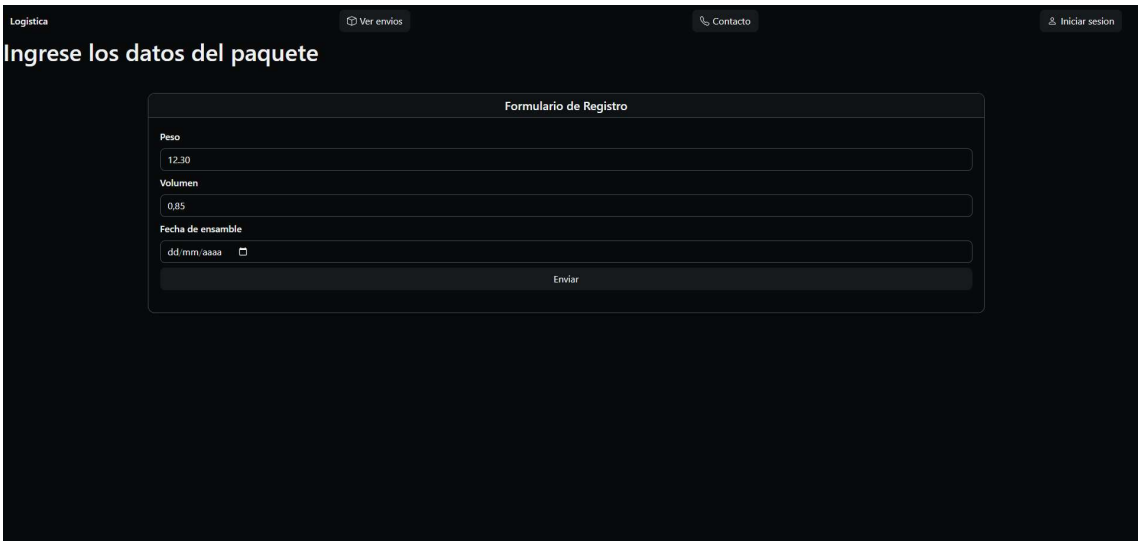
Actividades Realizadas:

Hoy avance con la página de MostrarPaquete, ya que me faltaban los botones de editar y eliminar. Después de estar un buen rato cambiando nombres de variables y viendo

errores junto con Benjamin, logré hacer que funcione.

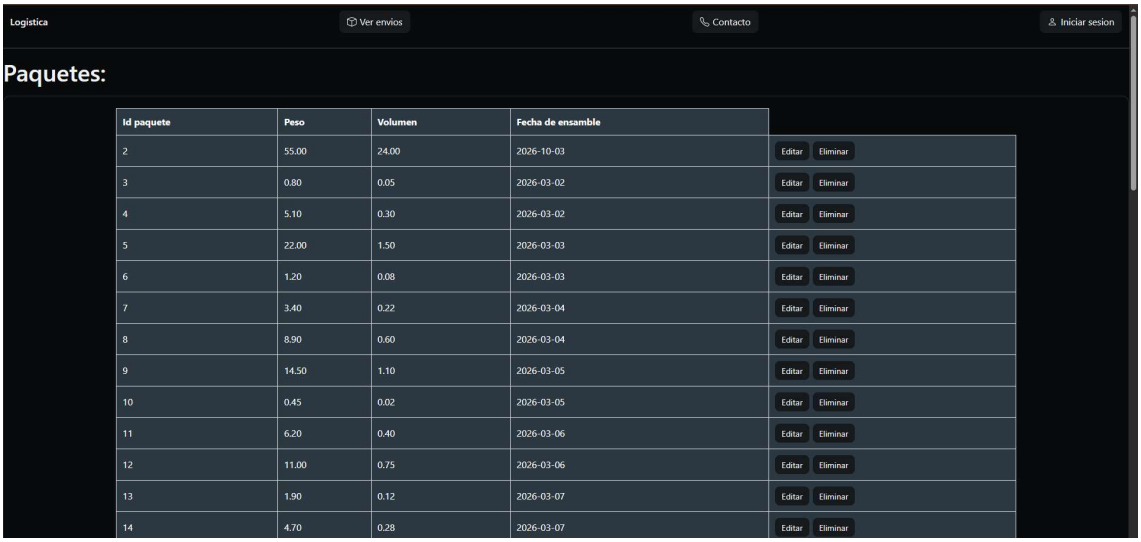


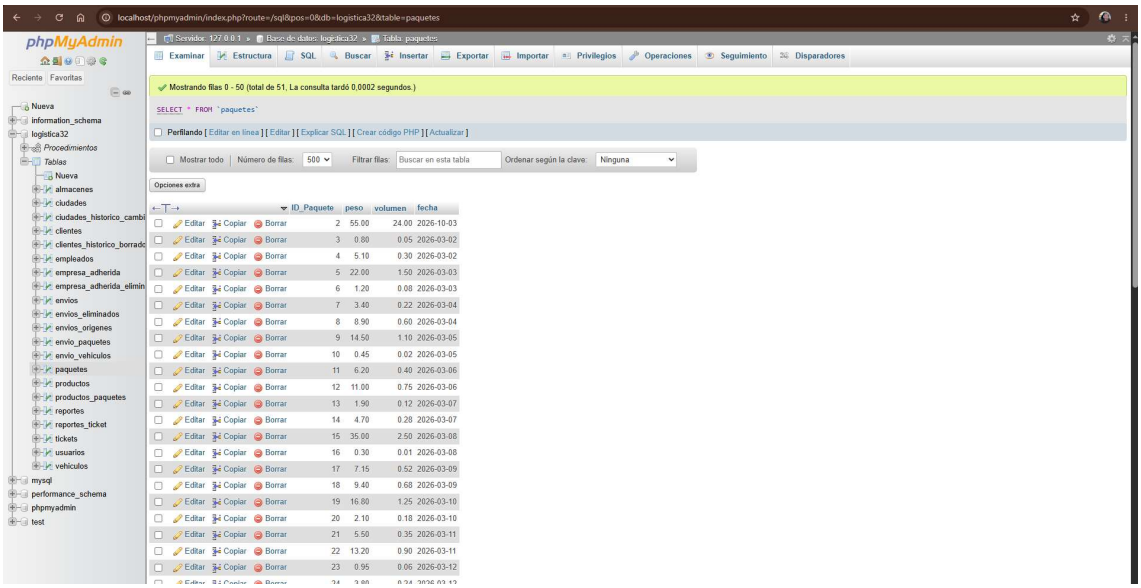
En este caso, toque el botón de editar del 1er paquete de la lista.



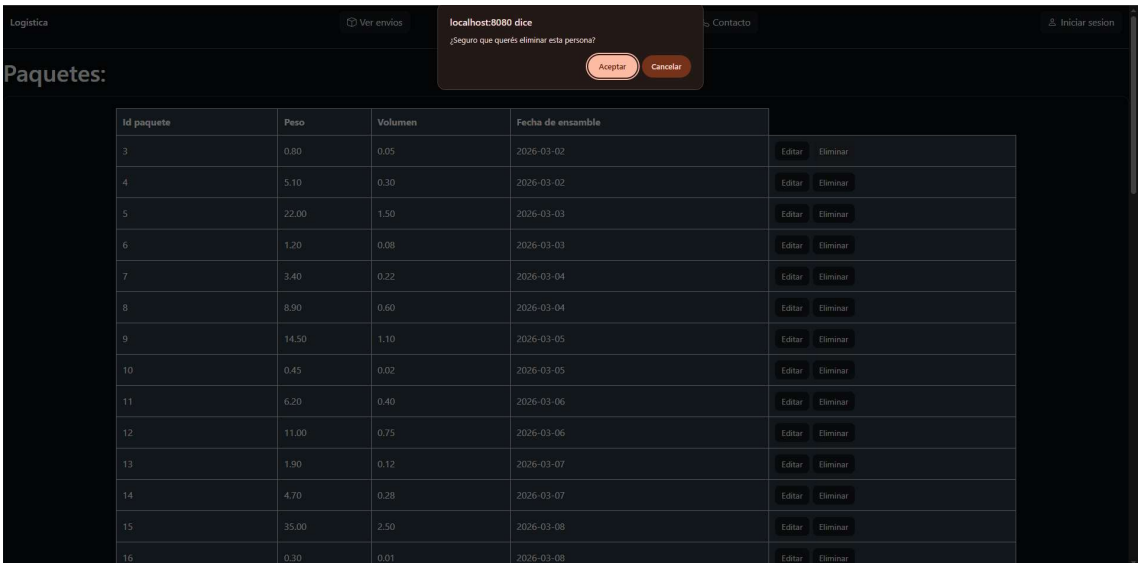
Cambio los datos

Funciono.

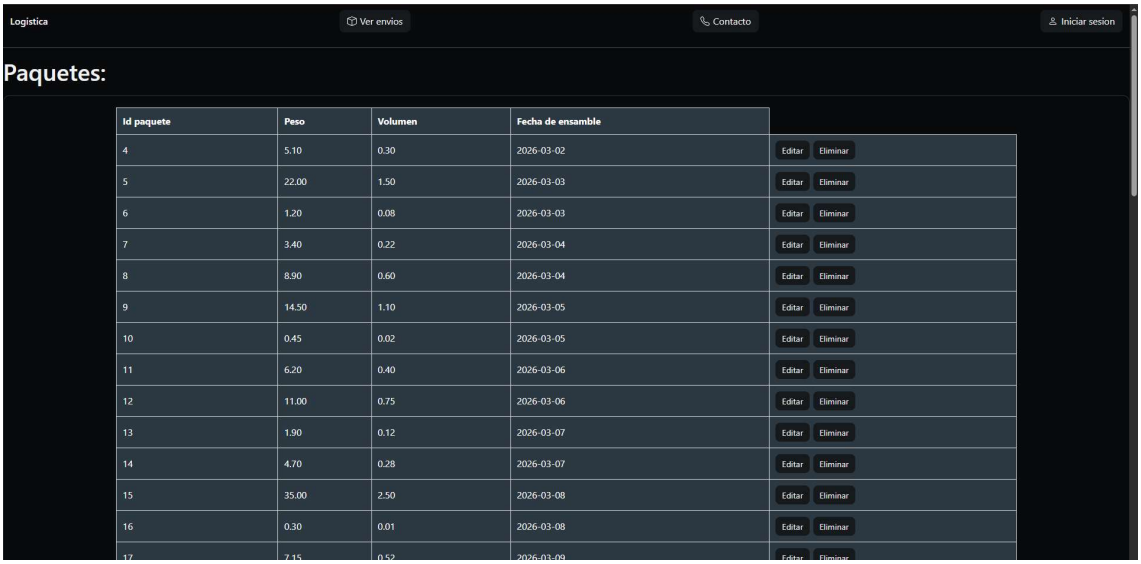


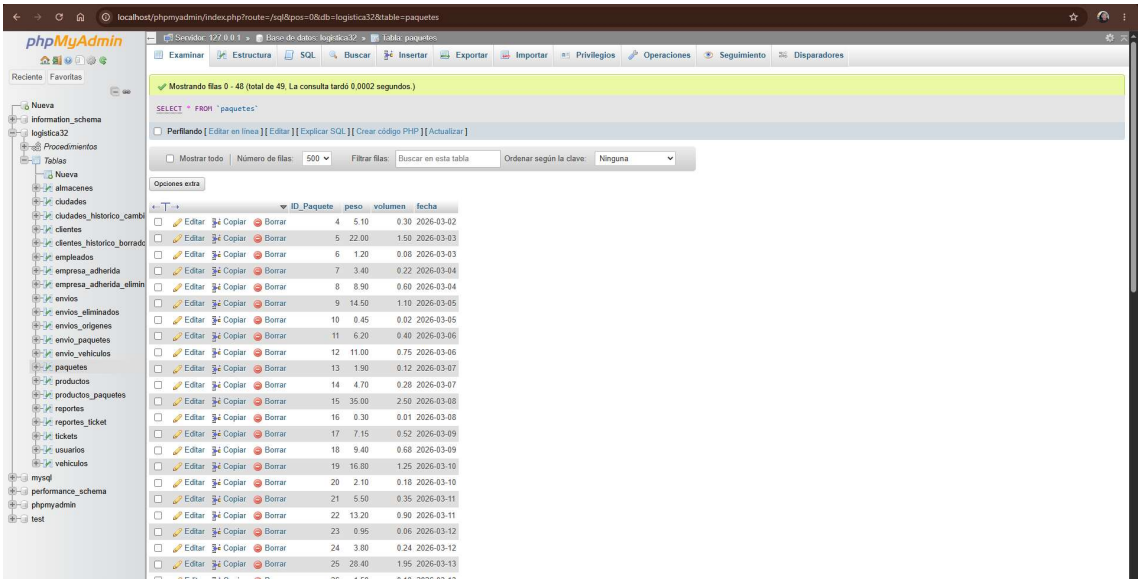


Ahora con el botón de eliminar eliminaré el mismo paquete que edité antes.


Logistica
Ver envios
Contacto
Iniciar sesion
localhost:8080 dice
¿Seguro que querés eliminar esta persona?
Aceptar
Cancelar
Paquetes:
Id paquete
Peso
Volumen
Fecha de ensamble
3 0.80 0.05 2026-03-02 Editar Eliminar
4 5.10 0.30 2026-03-02 Editar Eliminar
5 22.00 1.50 2026-03-03 Editar Eliminar
6 1.20 0.08 2026-03-03 Editar Eliminar
7 3.40 0.22 2026-03-04 Editar Eliminar
8 8.90 0.60 2026-03-04 Editar Eliminar
9 14.50 1.10 2026-03-05 Editar Eliminar
10 0.45 0.02 2026-03-05 Editar Eliminar
11 6.20 0.40 2026-03-06 Editar Eliminar
12 11.00 0.75 2026-03-06 Editar Eliminar
13 1.90 0.12 2026-03-07 Editar Eliminar
14 4.70 0.28 2026-03-07 Editar Eliminar
15 35.00 2.50 2026-03-08 Editar Eliminar
16 0.30 0.01 2026-03-08 Editar Eliminar

Funciono.



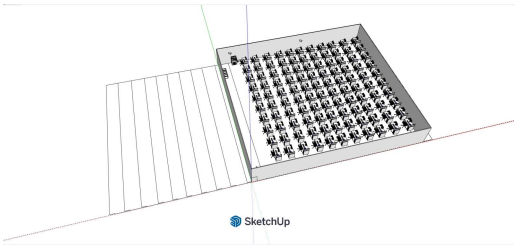


30/10

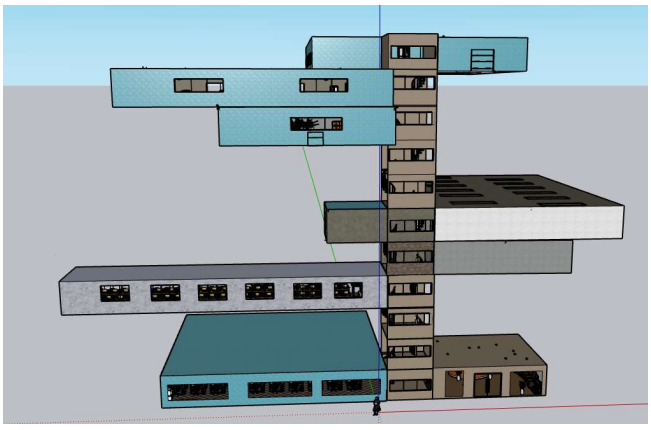
SARAGUSTI:

El 30 de octubre continue con lo referido a la maqueta, recibí la parte hecha por Rosato, que solo me había mandado la oficina número 1, y la coloque en su lugar correspondiente, luego de hacerle correcciones, debido a que tenía una doble pared no intencional, lo que culmina en una incapacidad de colocar ventanas en el lugar indicado. Por lo que las tuve que borrar a las

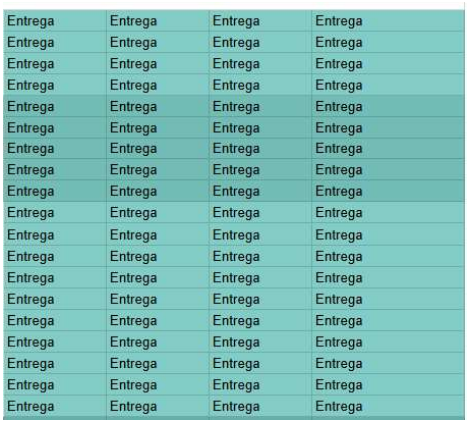
paredes internas para poder armar agujeros que dejan espacio a las ventanas, también estuvo el tema de que no habían switches, siendo que cada una de las oficinas debe contar con una cantidad de switches que cubran las necesidades de puertos necesario, siendo la fórmula. Can't. Switch=Cant. Dispositivos/22. El número 22 sale debido a que aunque un switch tenga 24 puertos, dos se reservan como mínimo para poderse conectar a los otros switches.
Otros problemas que tuve a la hora de la recepción, fueron que el piso se extendía demasiado por fuera de la zona de trabajo, necesitando borrar el piso y rehacerlo, con las dimensiones pactadas. Además las paredes quedaban con algunas marcas, de medida o aberturas.



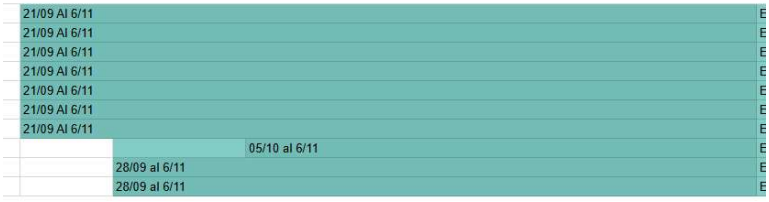
(A la izquierda está lo que me envió Nicolas y a la derecha está lo que modifica y corrige, estando actualmente ya colocado en el archivo de todas las edificaciones).


(Como va quedando la maqueta, faltando sólo las oficinas: 2,3,4,5,7,8,12,13,14,15,17,19. Que tengo que esperar a que mis compañeros las terminen para poderlas implementar en la maqueta unificada)
Este día también actualice el Gant, aunque nadie me lo pidió, debido a la necesidad de mantenerlo al margen de cómo llevamos la organización de los tiempos, ya que a partir de

dentro de 2 semanas empiezan las correcciones de todo en general, y vamos a tener que tener todo hecho.



(Como se ve la sección final ahora, le coloque entrega a las líneas del final, ya que antes lo tomábamos como el tiempo disponible hasta que se terminara el proyecto, pero ahora sabemos que no contamos con eso)



(Actualización de las partes de proyecto informático en las cuales estamos trabajando, aunque yo mis partes ya las termine y no tengo nada más para hacer).

PEDRAZA: El dia **30/10**, durante las horas de análisis de sistemas, le consulté a mi compañero Saragusti, quien realizo la mayoria del codigo correspondiente a la conexión de JAVA con el servidor HTTP para que me explique a detalle cómo funcionaba cómo funcionaba el mismo, ya que es en lo que a mi me falta trabajar y que más se me complica comprender.

Me explico el funcionamiento de cada bloque de código, si bien el concepto lo entendí, voy a estudiarlo a detalle para el día de la defensa individual.

Además, aproveché el momento para hablar con él sobre cuestiones de organización del grupo y el reparto de tareas a lo largo del proyecto, de igual manera lo consulte con el líder Goldfarb.

31/10

ROSATO:Se ejecutaron los ajustes finales y las modificaciones de depuración sobre las interfaces maquetadas en HTML. Con estas tareas de optimización de las plantillas visuales, se dio por concluida formalmente la etapa de integración y establecimiento de conexiones entre la capa de presentación y la lógica de control desarrollada en el backend de Java.

Noviembre:

2 de noviembre:

Pedraza:El día **2/11** realice los diseños CSS correspondientes a las páginas de iniciar sesión y crear usuario.

Por el lado de Iniciar Sesión, primero modifiqué el HTML debido a que el section que incluye el código para ingresar los datos, estaba como campos input. Para mayor comodidad visual y dado que ya había aprendido a implementarlos en el proyecto aparte de Tuinyla, lo cambié a un <form>. Posteriormente, modifique el JavaScript para adaptarlo.

Luego, para el CSS, me inspiré en un formulario similar que había realizado para otro proyecto, adaptando los datos a ingresar a los requeridos. También modifique el aspecto de todos los botones. El resultado final es el siguiente:

Luego para la parte de Crear usuario, a la cual se accede con el tercer botón del formulario, tuve que configurar prácticamente todo el HTML, JavaScript y CSS de cero (de por si CSS no tenia).

El codigo HTML + JavaScript solo constaba de una plantilla primitiva, no funcional y con datos faltantes. Aprovechando la plantilla y con algunas ayudas de Saragusti, deje el codigo asi:

```html
<!DOCTYPE html>
<html lang = "en">
<head>
    <meta charset = "UTF-8">
    <meta name = "viewport" content = "width=device-width, initial-scale=1.0">
    <title>Crear cuenta</title>
   <link rel="stylesheet" href="Crear.css">
</head>
<body>
    <button class="atras" onclick="history.back()">Volver atrás</button>
    <section class="centrar">
        <form class="crearCuenta">
            <h1>Crear cuenta de cliente: </h1>
            <label class="textoUsuario" for="Usuario">Nombre:</label>
            <input id="Usuario"  type="text" placeholder="Nombre">
            <label class="textoApellido" for="apellido">Apellido:</label>
            <input id = "apellido" type="text" placeholder="Apellido">
            <label class="textoDNI" for="dni">DNI:</label>
            <input id="dni" type="text" placeholder="DNI">
            <label class="textoDireccion" for="dire">Direccion:</label>
            <input id="dire" type="text" placeholder="Direccion">
            <label class="textoTelefono" for="tel">Telefono:</label>
            <input id="tel" type="text" placeholder="Telefono">
            <label class="textoEmail" for="email">E-Mail:</label>
            <input id="email" type="text" placeholder="E-Mail">
            <label class="textoCodigoP" for="cp">Codigo Postal:</label>
            <input id="cp" type="text" placeholder="Codigo Postal">
            <label class="textoContraseña" for="Contra">Contraseña:</label>
            <input id="Contra" type="text" placeholder="Contraseña">
            <button class="agregar" onclick="agregar_cuenta()">Crear Cuenta</button>
        </form>
    </section>
```

HTML

```
    <script>
        var listaa=JSON.parse(localStorage.getItem("lista"));
        function agregar_cuenta (){
            let ids=document.getElementById("Usuario").value;
            let apellido= document.getElementById("apellido").value;
            let dni= document.getElementById("dni").value;
            let dire= document.getElementById("dire").value;
            let tel= document.getElementById("tel").value;
            let email= document.getElementById("email").value;
            let cp= document.getElementById("cp").value;
            let contras=document.getElementById("Contra").value;

            let auxiliar = {
            Nombre: ids,
            Apellido: apellido,
            DNI: dni,
            Direccion: dire,
            telefono: tel,
            email: email,
            Codigo_postal: cp,
            Contrasenia: contras
            };
            listaa.push(auxiliar);
            window.open('Inicio_sesion.html', '_self');
            localStorage.setItem("lista", JSON.stringify(listaa));
        }

    </script>
</body>
```

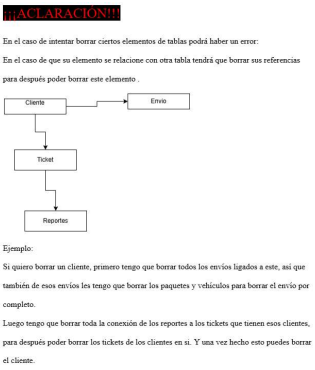
JavaScript (por medio de <script>)

Y sumandole el CSS, el cual tiene la misma plantilla que el utilizado para iniciar sesión, el resultado final de Crear cuenta es el siguiente:

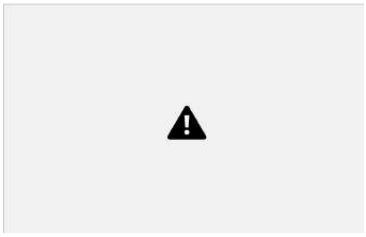
Saragusti:El 2 de Noviembre estuve colaborando de vuelta con Xiomara, en la realización del manual de usuario de la edición de Java, en el sentido de explicando todas las funciones requeridas de la ventana, debido a que algunas de las ventanas, las cuales se conectan con una tabla intermedia, por lo que tienen botones como mostrar, agregar y borrar, no como los normales si no con funciones específicas. Por lo que hoy terminamos el manual de usuario, subiéndolo al github, añadiendo la aclaración de que para eliminar las cosas añadidas, lo que significa que hay que eliminar todos los rastros de referencias a ese registro previo a su eliminación.

Además de esto también tuve que hacer algunas revisiones al sql, debido a que goldfarb para la pagina habia hecho algunas modificaciones que no me convencian, por lo que en paquetes le cambie el nombre a la variable fecha por fecha de ensamble, como estaba antes, para poder seguir usando la versión de java de normal, pero después tuve que en la versión html, cambiar

cómo se realizaban las llamadas, siendo que tuve que cambiar cada vez que se llamaba a fecha por fecha de ensamble. Otra cosa que tuve que modificar fueron que el diccionario de paquetes llama a la fecha detalle, y tuve que cambiarla a fecha para que se pudieran mostrar las tablas los datos de fecha, en editar ahora se muestra correctamente los parámetros que antes no aparecían.



(Como se ve la advertencia, del final aclaro por si no se ve al inicio, no puedo meter foto de todo porque sería innecesario y reiterar lo que se puede ver en el github)


(Debido a que muy bien no me acuerdo de donde se guardaron los archivos, no pude tomar la foto del código de esto, por lo que dejó el cuadro abierto por si en un futuro encuentro donde había hecho esas modificaciones como reemplazo)
Además agregó que al tocar una imagen te abra la página del index de administrador, para que no todo el mundo pueda acceder de manera única. Otra de las imágenes permite acceso a la creación de las auxiliares, que serían la lista de las cuentas previamente disponibles en el sistema. Además ayuda con la creación de que las listas ahora tengan todos los datos en vez de solo el nombre, y que cuenten con contraseña escrita por la persona en vez de usar el mero mail como hacían antes.

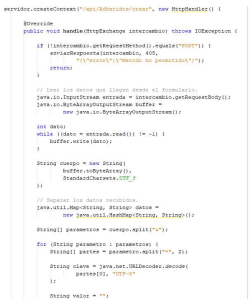
Estuve colaborando con los chicos, más específicamente Pedraza, Rosato y Ureña en explicarles cómo es que funciona la conexión html a java usando una parte de código en el html y otra parte de código en el servidor web. También estuve teniendo que ser el portavoz de lo que me comentaba Goldfarb debido a que faltó y hubo todo un ida y vuelta al respecto de quien hacía el manual de explotación.

Otra de las cosas que hice fue la revisión de la ventana de agregar adheridos y la de vehículos, siendo que la de adheridos estaba bien, pero a la de vehículos le faltaban muchos datos, así que se lo tuve que comentar. Debido a que la tabla de sql cuenta con ID(Que igual no cuenta porque es auto_increment), precio, anio, empleado, marca y modelo. Solo apareciendo precio y año en la página de mis compañeros.

Después de hacer explicaciones termine por hacerle a Rosato una parte del codigo de conexion para que se pudieran crear los vehículos y además las empresas

```
servidor.createContext("/api/Adheridos/crear", new HttpHandler() {

    @Override
    public void handle(HttpExchange intercambio) throws IOException {

        if (!intercambio.getRequestMethod().equals("POST")) {
            enviarRespuesta(intercambio, 405,
                    "{\"error\":\"Método no permitido\"}");
            return;
        }

        // Leer los datos que llegan desde el formulario.
        java.io.InputStream entrada = intercambio.getRequestBody();
        java.io.ByteArrayOutputStream buffer =
                new java.io.ByteArrayOutputStream();

        int dato;
        while ((dato = entrada.read()) != -1) {
            buffer.write(dato);
        }

        String cuerpo = new String(
                buffer.toByteArray(),
                StandardCharsets.UTF_8
        );

        // Separar los datos recibidos.
        java.util.Map<String, String> datos =
                new java.util.HashMap<String, String>();

        String[] parametros = cuerpo.split("&");

        for (String parametro : parametros) {
            String[] partes = parametro.split("=", 2);

            String clave = java.net.URLDecoder.decode(
                    partes[0], "UTF-8"
            );

            String valor = "";
```

```
const formulario = document.getElementById("formulario_ad");
const mensaje = document.getElementById("mensaje");

formulario.addEventListener("submit", function (evento) {
    evento.preventDefault();

    const adherido = {
        Nombre : document.getElementById("Nombre").value,
        Cuil : document.getElementById("Cuil").value,
        direccion : document.getElementById("Direccion").value,
        telefono : document.getElementById("Telefono").value,
        rubro : document.getElementById("Rubro").value,
        dominio : document.getElementById("Dueño").value
    };

    fetch("/api/Adheridos/crear", {
        method: "POST",
        headers: {
            "Content-Type": "application/x-www-form-urlencoded"
        },
        body: new URLSearchParams(adherido).toString()
    })
    .then(function (respuesta) {
        return respuesta.json().then(function (datos) {
            if (!respuesta.ok) {
                throw new Error(datos.error || "No se pudo guardar el vehiculo.");
            }

            return datos;
        });
    })
    .then(function (datos) {
        mensaje.className = "alert alert-success mt-3";
        mensaje.textContent = datos.mensaje;

        formulario.reset();
    })
    .catch(function (error) {
        console.error(error);

        mensaje.className = "alert alert-danger mt-3";
        mensaje.textContent = error.message;
    });
});
```



```
servidor.createContext("/api/Adheridos/crear", new HttpHandler() {

    @Override
    public void handle(HttpExchange intercambio) throws IOException {

        if (!intercambio.getRequestMethod().equals("POST")) {
            enviarRespuesta(intercambio, 405,
                    "{\"error\":\"Método no permitido\"}");
            return;
        }

        // Leer los datos que llegan desde el formulario.
        java.io.InputStream entrada = intercambio.getRequestBody();
        java.io.ByteArrayOutputStream buffer =
                new java.io.ByteArrayOutputStream();

        int dato;
        while ((dato = entrada.read()) != -1) {
            buffer.write(dato);
        }

        String cuerpo = new String(
                buffer.toByteArray(),
                StandardCharsets.UTF_8
        );

        // Separar los datos recibidos.
        java.util.Map<String, String> datos =
                new java.util.HashMap<String, String>();

        String[] parametros = cuerpo.split("&");

        for (String parametro : parametros) {
            String[] partes = parametro.split("=", 2);

            String clave = java.net.URLDecoder.decode(
                    partes[0], "UTF-8"
            );

            String valor = "";
```

```
<!DOCTYPE>
<script>
    const formulario = document.getElementById("formulario vehiculo");
    const mensaje = document.getElementById("mensaje");

    formulario.addEventListener("submit", function (evento) {
        evento.preventDefault();

        const vehiculo = {
            precio : document.getElementById("precio").value,
            año : document.getElementById("año").value,
            empleado : document.getElementById("id_empleado").value,
            modelo : document.getElementById("modelo").value,
            Marca : document.getElementById("Marca").value
        };

        fetch("/api/Vehiculos/crear", {
            method: "POST",
            headers: {
                "Content-Type": "application/x-www-form-urlencoded"
            },
            body: new URLSearchParams(vehiculo).toString()
        })
        .then(function (respuesta) {
            return respuesta.json().then(function (datos) {
                if (!respuesta.ok) {
                    throw new Error(datos.error || "No se pudo guardar el vehiculo.");
                }

                return datos;
            });
        })
        .then(function (datos) {
            mensaje.className = "alert alert-success mt-3";
            mensaje.textContent = datos.mensaje;

            formulario.reset();
        })
        .catch(function (error) {
            console.error(error);

            mensaje.className = "alert alert-danger mt-3";
            mensaje.textContent = error.message;
        });
    });
```

(Visualización del código, hecho para permitir que los datos de los formularios de html lleguen a java y se manden a la base de datos, funcionan y las testee. Se añaden los datos y se muestran de manera correcta en ambas páginas)

Rosato:2/11

Se llevó a cabo una sesión de desarrollo conjunto con el compañero Benjamin Saragusti con el propósito de resolver las últimas tareas pendientes en la capa de código. Durante esta jornada, se revisaron las funciones de consistencia y se realizaron las pruebas unitarias finales, alcanzando el cierre técnico de la programación de los módulos asignados y asegurando la estabilidad integral del software.

3 de noviembre:

Pedraza:

> El día martes **3/11**, en mi casa, retome con los códigos que tenía pendientes desde mi semana de inactividad (10/10 - 19/10 aprox.).
>
> Hasta el momento, solo tenía realizada una primera versión del código de Java para agregar empleados (del servidor web). Este día, finalicé del todo el código para agregar empleados, comencé y complete los de editar empleado y eliminar empleado, todo a partir de plantillas ya realizadas dentro de la clase, adaptándola a lo solicitado.
>
> Como en mi casa tengo una version de NetBeans distinta a la usada en las PCs del colegio, las conexiones entre Java, HTTPS y la base de datos no me andaban correctamente, por lo que la unica manera en el momento que tenia para verificar que esten bien los codigos fue preguntarselo a la Inteligencia Artificial (ChatGPT). Le pase toda la clase completa y le pregunte si, a partir de ello, mis codigos estaban bien.

Tenian la estructura correcta, solo me habia equivocado en un par de detalles menores, que en 10 segundos los corregi.

Los errores fueron:

En Agregar Empleado:

sentencia.setString(6, Sector);

sentencia.setString(7, Cargo);

sentencia.setString(8, Rol);

sentencia.registerOutParameter(7, java.sql.Types.VARCHAR); ←— ACÁ IBA 9, NO 7

En editar empleado:

String sql = "UPDATE empleados SET nombre = ?, apellido = ?, "

+ "dni = ?, email = ?, telefono = ?, sector = ?" ←— ME FALTO UNA COMA

+ "cargo = ?, rol = ? WHERE ID_Empleado = ?";

Los códigos finales, quedaron de la siguiente manera:

CREAR EMPLEADO (agregar empleado):

```
1858        });
            servidor.createContext("/api/Empleados/crear", new HttpHandler() {
1860
1861            @Override
                public void handle(HttpExchange intercambio) throws IOException {
1863
1864                if (!intercambio.getRequestMethod().equals("POST")) {
1865                    enviarRespuesta(intercambio, 405,
1866                            "{\"error\":\"Metodo no permitido\"}");
1867                    return;
1868                }
1869
1870                // Leer los datos que llegan desde el formulario.
1871                java.io.InputStream entrada = intercambio.getRequestBody();
1872                java.io.ByteArrayOutputStream buffer =
1873                        new java.io.ByteArrayOutputStream();
1874
1875                int dato;
1876                while ((dato = entrada.read()) != -1) {
1877                    buffer.write(dato);
1878                }
1879
1880                String cuerpo = new String(
1881                        buffer.toByteArray(),
1882                        StandardCharsets.UTF_8
1883                );
1884
1885                // Separar los datos recibidos.
1886                java.util.Map<String, String> datos =
                        new java.util.HashMap<String, String>();
1888
1889                String[] parametros = cuerpo.split("&");
1890
1891                for (String parametro : parametros) {
1892                    String[] partes = parametro.split("=", 2);
1893
1894                    String clave = java.net.URLDecoder.decode(
1895                            partes[0], "UTF-8"
1896                    );
1897
1898                    String valor = "";
```

```
1900                    if (partes.length > 1) {
1901                        valor = java.net.URLDecoder.decode(
1902                                partes[1], "UTF-8"
1903                        );
1904                    }
1905
1906                    datos.put(clave, valor);
1907                }
1908
1909                // Obtener los valores del formulario.
1910                String Nombre = datos.get("Nombre");
1911                String Apellido = datos.get("Apellido");
1912                String Dni = datos.get("Dni");
1913                String Email = datos.get("email");
1914                String Telefono = datos.get("telefono");
1915                String Sector = datos.get("sector");
1916                String Cargo = datos.get("cargo");
1917                String Rol = datos.get("rol");
1918                // Validar los campos.
1919                if (Nombre == null
1920                        || Apellido == null
1921                        || Dni == null
1922                        || Email == null
1923                        || Telefono == null
1924                        || Sector == null
1925                        || Cargo == null
1926                        || Rol == null
1927                        || Nombre.trim().isEmpty()
1928                        || Apellido.trim().isEmpty()
1929                        || Dni.trim().isEmpty()
1930                        || Email.trim().isEmpty()
1931                        || Telefono.trim().isEmpty()
1932                        || Sector.trim().isEmpty()
1933                        || Cargo.trim().isEmpty()
1934                        || Rol.trim().isEmpty()
1935                        ) {
1936
1937                    enviarRespuesta(intercambio, 400,
1938                            "{\"error\":\"Todos los campos son obligatorios\"}");
1939                    return;
1940                }
```

```
                int dni;
                int telefono;

                try {
                    dni = Integer.parseInt(Dni);
                    telefono = Integer.parseInt(Telefono);

                } catch (NumberFormatException e) {
                    enviarRespuesta(intercambio, 400,
                            "{\"error\":\"El DNI y telefono deben ser numeros enteros\"}");
                    return;
                }

                // Preparar la consulta SQL.
                String sql = "CALL agregar_empleados(?, ?, ?, ?, ?, ?, ?, ?, ?)"; // Mantiene los 7 '?'

try (
    Connection conexion = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/logistica32",
            "root",
            ""
    );
     CallableStatement sentencia = conexion.prepareCall(sql);
) {
    sentencia.setString(1, Nombre);
    sentencia.setString(2, Apellido);
    sentencia.setInt(3, dni);
    sentencia.setString(4, Email);
    sentencia.setInt(5, telefono);
    sentencia.setString(6, Sector);
    sentencia.setString(7, Cargo);
    sentencia.setString(8, Rol);

    // 1. Corregido: Registrar el séptimo parámetro como SALIDA (Ejemplo si devuelve Texto)
    sentencia.registerOutParameter(9, java.sql.Types.VARCHAR);

    sentencia.execute();

    // 2. Opcional: Puedes leer lo que te devolvió la base de datos
    String respuestaBD = sentencia.getString(9);

    enviarRespuesta(intercambio, 201,
            "{\"mensaje\":\"Empleado guardado correctamente\", \"detalle\":\"" + respuestaBD + "\"}");
```

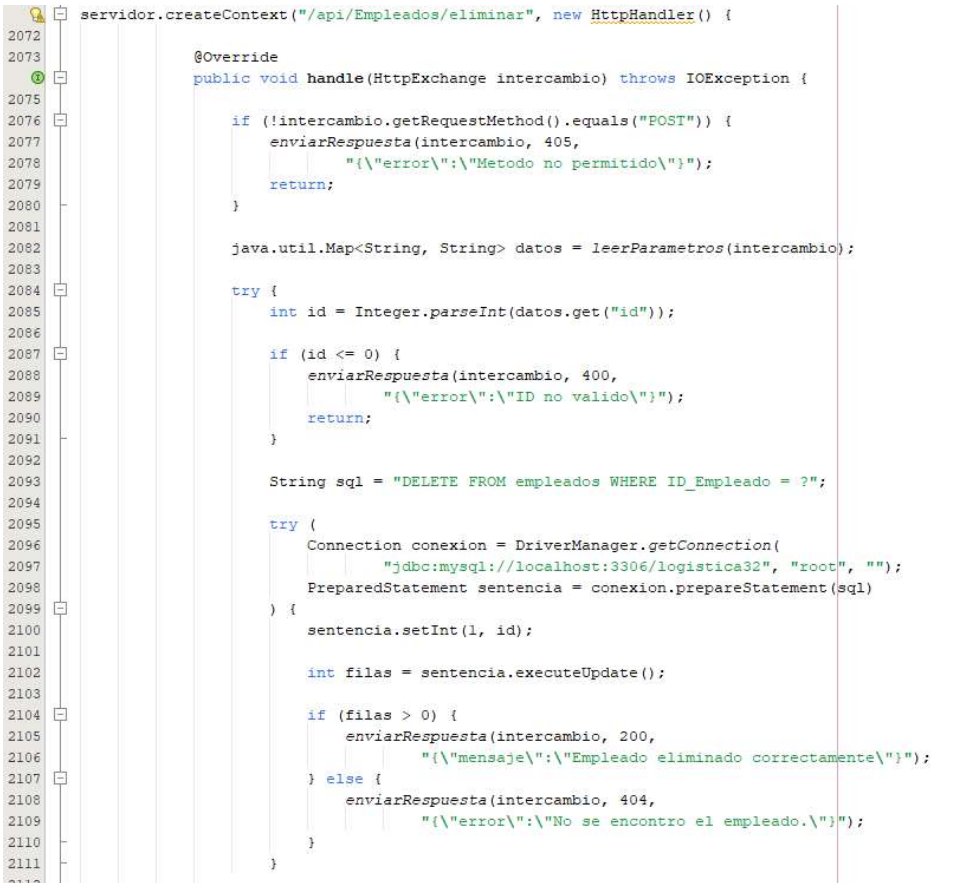
```
} catch (SQLException e) {
    e.printStackTrace();
    enviarRespuesta(intercambio, 500,
            "{\"error\":\"Error al guardar en la base de datos\"}");
}


        }
    });

// --------------------------------------------------------------------------
```

EDITAR EMPLEADO:

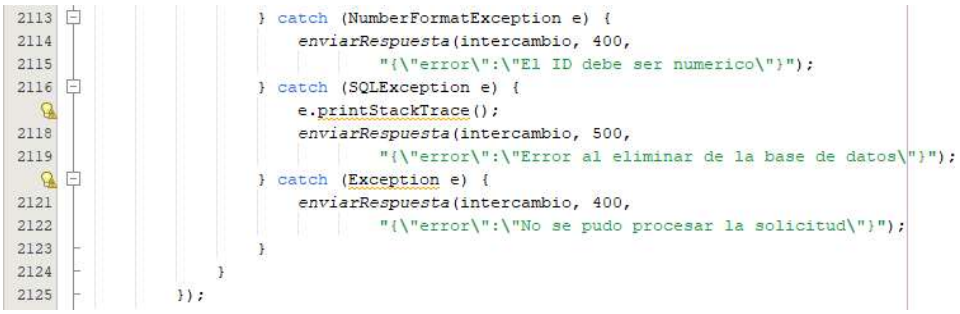
```
servidor.createContext("/api/Empleados/editar", new HttpHandler() {
1997
1998            @Override
                public void handle(HttpExchange intercambio) throws IOException {
2000
2001            if (!intercambio.getRequestMethod().equals("POST")) {
2002                enviarRespuesta(intercambio, 405,
2003                        "{\"error\":\"Metodo no permitido\"}");
2004                return;
2005            }
2006
2007            java.util.Map<String, String> datos = leerParametros(intercambio);
2008
2009            try {
2010                int id = Integer.parseInt(datos.get("id"));
2011                String nombre = datos.get("nombre");
2012                String apellido = datos.get("apellido");
2013                int dni = Integer.parseInt(datos.get("Dni"));
2014                String email = datos.get("email");
2015                int telefono = Integer.parseInt(datos.get("telefono"));
2016                String sector = datos.get("sector");
2017                String cargo = datos.get("cargo");
2018                String rol = datos.get("rol");
2019
2020
2021                if (dni <= 0 || telefono <= 0) {
2022
2023                    enviarRespuesta(intercambio, 400,
2024                            "{\"error\":\"Revisa los datos ingresados\"}");
2025                    return;
2026                }
2027
2028                String sql = "UPDATE empleados SET nombre = ?, apellido = ?, " + "dni = ?, email = ?, telefono = ?, sector = ?," + "cargo = ?, rol = ? WHERE ID_Empleado = ?";
2029
2030                try (
2031                    Connection conexion = DriverManager.getConnection(
2032                            "jdbc:mysql://localhost:3306/logistica32", "root", "");
2033                    PreparedStatement sentencia = conexion.prepareStatement(sql)
2034                ) {
2035
2036                    sentencia.setString(1, nombre);
2037                    sentencia.setString(2, apellido);
2038                    sentencia.setInt(3, dni);
```

```
2038                    sentencia.setInt(3, dni);
2039                    sentencia.setString(4, email);
2040                    sentencia.setInt(5, telefono);
2041                    sentencia.setString(6, sector);
2042                    sentencia.setString(7, cargo);
2043                    sentencia.setString(8, rol);
2044                    sentencia.setInt(9, id);
2045
2046                    int filas = sentencia.executeUpdate();
2047
2048                    if (filas > 0) {
2049                        enviarRespuesta(intercambio, 200,
2050                                "{\"mensaje\":\"Empleado actualizado correctamente\"}");
2051                    } else {
2052                        enviarRespuesta(intercambio, 404,
2053                                "{\"error\":\"No se encontro al empleado\"}");
2054                    }
2055                }
2056
2057                } catch (NumberFormatException e) {
2058                    enviarRespuesta(intercambio, 400,
2059                            "{\"error\":\"El DNI y el Telefono deben ser numeros validos\"}");
2060                } catch (SQLException e) {
2062                    e.printStackTrace();
2063                    enviarRespuesta(intercambio, 500,
2065                            "{\"error\":\"Error al actualizar la base de datos\"}");
2066                } catch (Exception e) {
2067                    enviarRespuesta(intercambio, 400,
2068                            "{\"error\":\"Faltan datos o son invalidos\"}");
2069                }
            }
        });
```

ELIMINAR EMPLEADO:



```
servidor.createContext("/api/Empleados/eliminar", new HttpHandler() {

    @Override
    public void handle(HttpExchange intercambio) throws IOException {

        if (!intercambio.getRequestMethod().equals("POST")) {
            enviarRespuesta(intercambio, 405,
                    "{\"error\":\"Metodo no permitido\"}");
            return;
        }

        java.util.Map<String, String> datos = leerParametros(intercambio);

        try {
            int id = Integer.parseInt(datos.get("id"));

            if (id <= 0) {
                enviarRespuesta(intercambio, 400,
                        "{\"error\":\"ID no valido\"}");
                return;
            }

            String sql = "DELETE FROM empleados WHERE ID_Empleado = ?";

            try (
                Connection conexion = DriverManager.getConnection(
                        "jdbc:mysql://localhost:3306/logistica32", "root", "");
                PreparedStatement sentencia = conexion.prepareStatement(sql)
            ) {
                sentencia.setInt(1, id);

                int filas = sentencia.executeUpdate();

                if (filas > 0) {
                    enviarRespuesta(intercambio, 200,
                            "{\"mensaje\":\"Empleado eliminado correctamente\"}");
                } else {
                    enviarRespuesta(intercambio, 404,
                            "{\"error\":\"No se encontro el empleado.\"}");
                }
            }
```



```
            } catch (NumberFormatException e) {
                enviarRespuesta(intercambio, 400,
                        "{\"error\":\"El ID debe ser numerico\"}");
            } catch (SQLException e) {
                e.printStackTrace();
                enviarRespuesta(intercambio, 500,
                        "{\"error\":\"Error al eliminar de la base de datos\"}");
            } catch (Exception e) {
                enviarRespuesta(intercambio, 400,
                        "{\"error\":\"No se pudo procesar la solicitud\"}");
            }
        }
    });
```

Lamentablemente, todavía no entiendo el funcionamiento de estos códigos más que por deducción, ya que solo me base en las plantillas realizadas previamente. Por lo que tendré que estudiar bien sus funcionamientos para el día de las defensas personales.

Saragusti:El 3 de noviembre hice algunas correcciones al código de xiomara, debido a que lo estaba ejecutando en el archivo y no en el Localhost, y tenía algunos problemas con el fetch, por lo que la estuve ayudando con eso. Además de ayudar a Xiomara me encargue de meter el segundo piso, hecho por Rosato, en la maqueta, quedando correctamente puesto. Tuve que modificarle las paredes porque no tenían textura pero más que ello nada porque estaba bien y además porque no quería cambiarla por completo. Además de eso estuve normalizando las

versiones del modelo de http porque al hacerlo grupal tengo que estar cambiando de lugar algunos datos, copiando y pegando htmls, y pegando bloques de código del servidor para mantener una dirección clara y que no quede ningún código roto o trabado.

Además de eso le puse botones para poder registrar vehículos, empresas adheridas y poder ver la vista general sin necesitar de escribir la dirección, el tema es que los botones no tienen un css correcto por lo que se ven feos y estirados, pero esa parte se encargará uno de mis compañeros para que yo no realize todo.

4 de noviembre:

Pedraza:Al día siguiente, el miércoles **4/11** y durante las horas de Redes y Proyecto Informático 2,

mi compañero Benjamin Saragusti y yo corregimos juntos los códigos del servidor web en Java que realicé el día anterior.

Una vez que los corroboramos bien, me puse a hacer los códigos de mismo fin, pero correspondientes a la página en HTML.

De igual manera que con los códigos de Java, me base en plantillas reutilizadas por otros integrantes, modificándolas y adaptándolas para que funcionen con empleados.

No voy a agregar capturas de los códigos, ya que son los mismos que quedaron en la versión final de la página, y se pueden encontrar en el zip de la misma. Esto con el fin de no irradiar la carpeta de muchas capturas.

Lo mismo con el funcionamiento, si bien esto fue porque no saque capturas en el momento y, como mencione anteriormente, en mi PC se me dificulta probar las conexiones por el tema de versiones, no subire tampoco porque se vera despues como funciona a la hora de presentar el proyecto y utilizar la página en cualquier momento.

Los probamos en la página y funcionaban perfectamente, los empleados creados se guardaban en la base de datos y se visibilizaban en las tablas de la página, así como también los podías editar y eliminar. Lamentablemente, cree los códigos html de agregar, editar y eliminar empleado y funcionan

Saragusti:El 4 de noviembre estuve colaborando en codigo HTML, para que mis compañeros pudieran terminar sus partes, este dia ayude a que terminaran por completo sus partes a Pedraza ya Zownir que terminaron sus partes este dia, a Xiomara y Rosato también los ayude pero no terminaron sus partes este dia. Y además de estar ayudando con el código estuve sincronizando las versiones porque al ser un zip, cuando uno terminaba una parte tenía que juntar específicamente las partes que habían modificado para que los archivos no se sobrelapen.Además como solicitado estuve explicando a la mañana toda la construcción de la red, debido a que como me encargue de la mayoría tenía la obligación de que todos entendieran lo que había hecho. Además les pase el link del instalador de la última versión de cisco Packet Tracer para que pudieran ver cómo quedó la red y hacer pruebas desde sus casas.

Además avance con una parte de la carpeta de campo grupal, aunque no mucho, solo llegue hasta el 11 de agosto, y me terminaron destruyendo de esa tarea para pasarlas a mis compañeros Pedraza y Xiomara.

5 de noviembre:

Goldfarb:

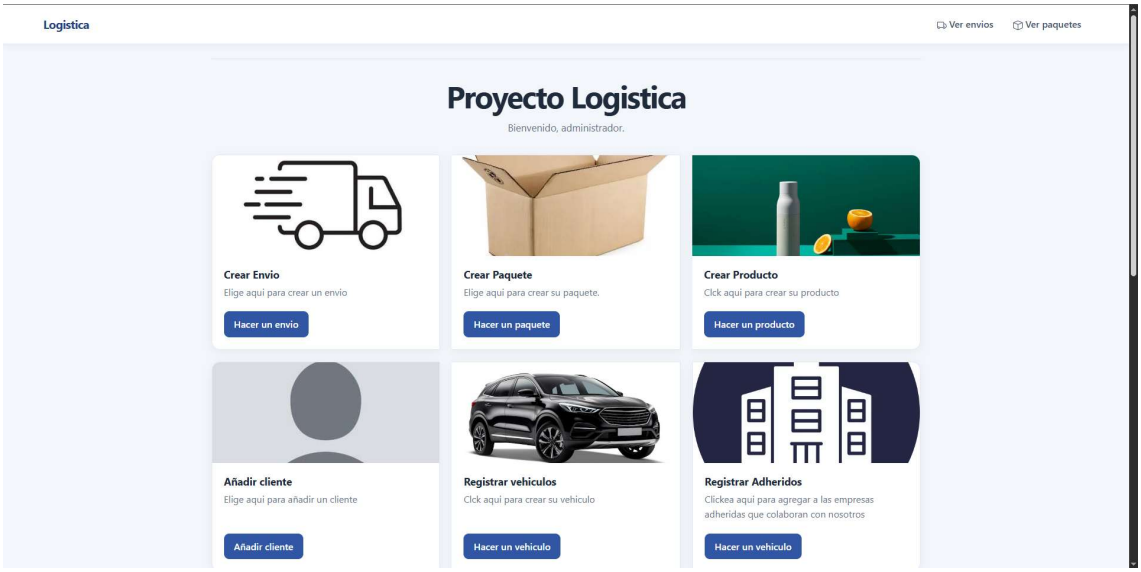
Fecha: 5/11/26

Ubicación: Escuela : Aula

Hora: 15:00

Actividades Realizadas:

Estos últimos días, estuve haciendo todo el diseño del index de la parte del administrador. Gracias a la ayuda de la IA del Visual Studio Code, pude darle un diseño mucho mas renovado y moderno.


Logistica
Ver envios
Ver paquetes
Proyecto Logistica
Bienvenido, administrador.
Crear Envio
Elige aqui para crear un envio
Hacer un envio
Crear Paquete
Elige aqui para crear su paquete.
Hacer un paquete
Crear Producto
Clck aqui para crear su producto
Hacer un producto
Añadir cliente
Elige aqui para añadir un cliente
Añadir cliente
Registrar vehiculos
Clck aqui para crear su vehiculo
Hacer un vehiculo
Registrar Adheridos
Clickea aqui para agregar a las empresas adheridas que colaboran con nosotros
Hacer un vehiculo

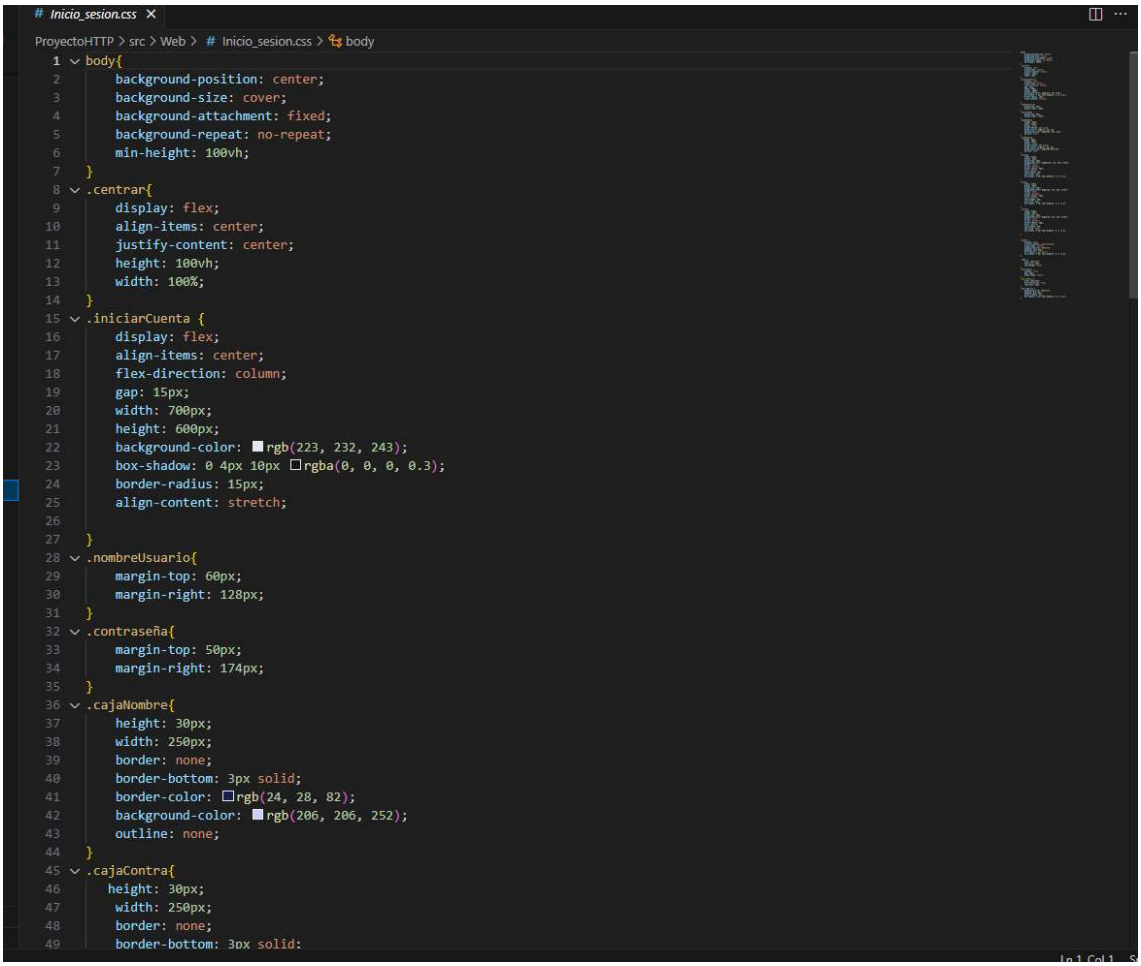
Seccion de Consultas
Ver Envio
Elige aqui para ver los envios
Ver envios
Ver Paquete
Elige aqui para ver los paquetes.
Ver paquetes
Ver Productos
Clck aqui para ver los productos
Ver productos
Ver cliente
Elige aqui para ver los clientes
Ver cliente
Ver todas las tablas
Consulta general de la base de datos.
Ver
Registrar vehiculos
Clck aqui para crear su vehiculo
Hacer un vehiculo
Registrar Adheridos
Clickea aqui para agregar a las empresas adheridas que colaboran con nosotros
Hacer un vehiculo

Saragusti:

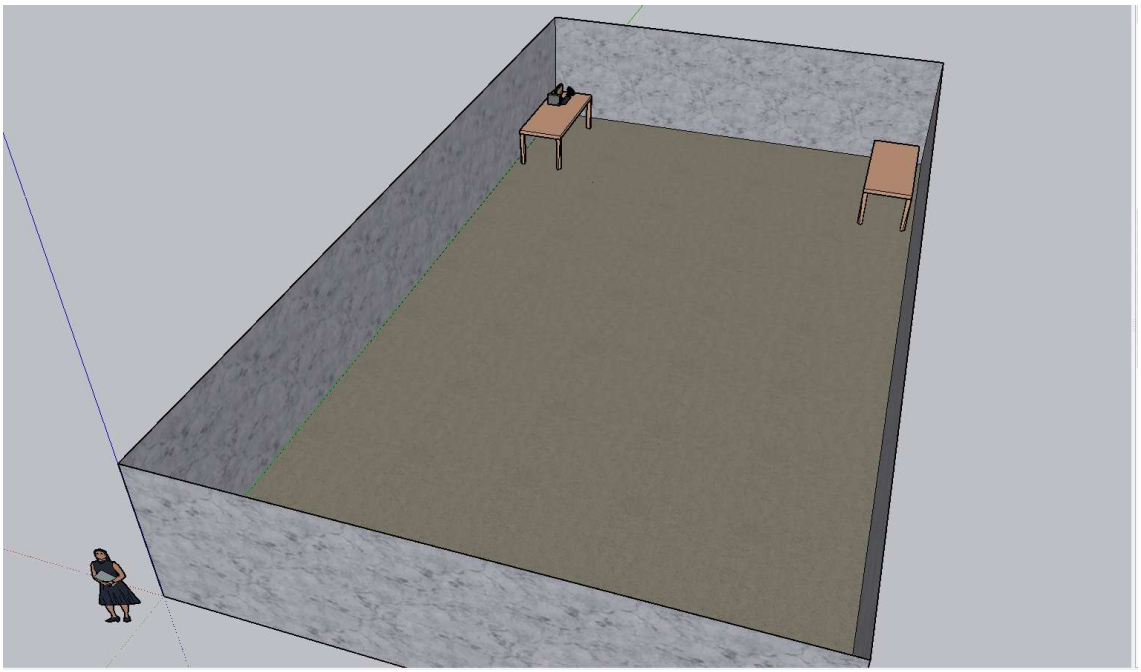
El 5 de noviembre estuve ayudando específicamente a Zownir con sus consultas al respecto de la realización de la maqueta, debido a como tenía que hacer borrados de paredes. Además de esto estuve colaborando en la elaboración del código de las partes correspondientes a Xiomara y a Rosato logrando así ayudarlos en el testing para que sus partes funcionan, siendo que el único error que muestran es uno prestado por la base de datos que es el error clave foránea que ocurre al intentar borrar un registro que está relacionado con otro en otra tabla. Además de haber colaborado con este código realice, con la base de Goldfarb, la ventana de ver envíos, editar envíos y los métodos de borrar envíos. Debido a que los envíos tienen paquetes y vehículos, cuando los vas a borrar se borran estos también, para hacerlo más sencillo para el usuario común. Los métodos que hice están en la sección de envíos también, y el de borrar tiene 3 sentencias, la primera para borrar todos los paquetes del envío, otra para borrar todos los vehículos del envío y por último para borrar el envío en sí.
No lo pude testear hoy al borrar envio por lo que esto lo haré mañana, pero por el editar y mostrar si los probe y funcionan por completo, por lo que lo unico que quedaria por hacer del proyecto será: Manual explotación, digitalizar foda, volver pdf todas las carpetas de campo, poner los html en el cisco y nada más creo.

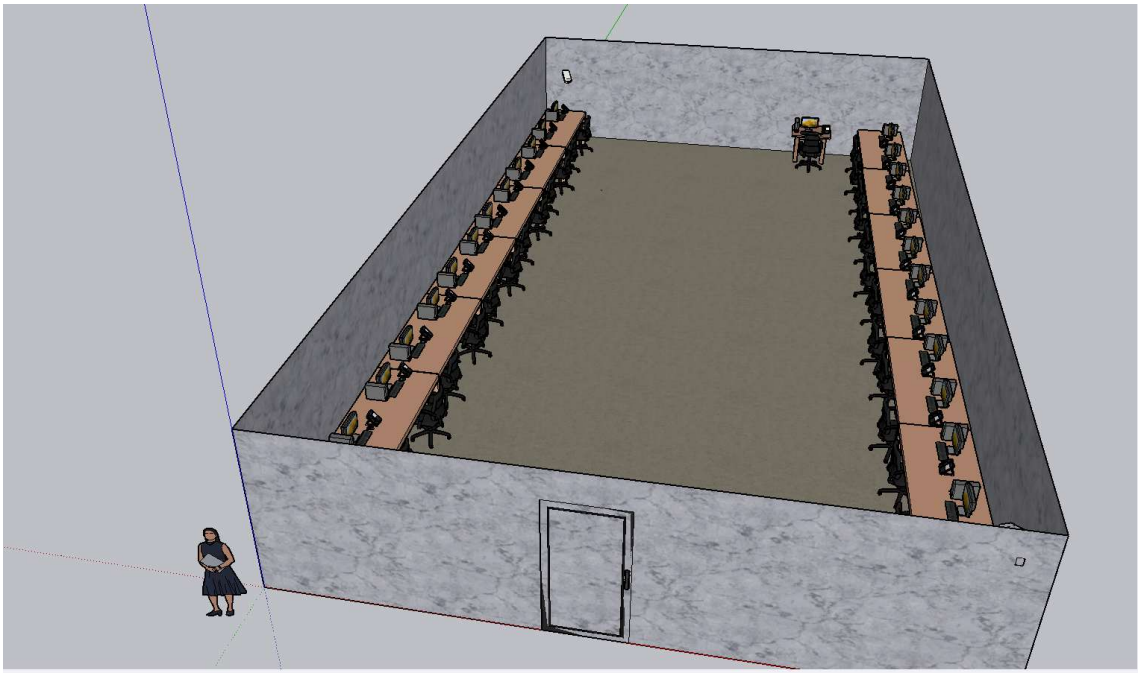
Xiomara:**05/11/2026**

Finalmente hoy jueves 5 de noviembre terminé de realizar la conexión con el servidor web. También me puse a modificar el css de las páginas de inicio de sesión y la de crear ya que tenían colores que no iban con las páginas realizadas. Además hoy nos juntamos todos los integrantes del grupo para ponernos de acuerdo que parte va a presentar cada uno. A mi me tocó presentar la parte de POO.

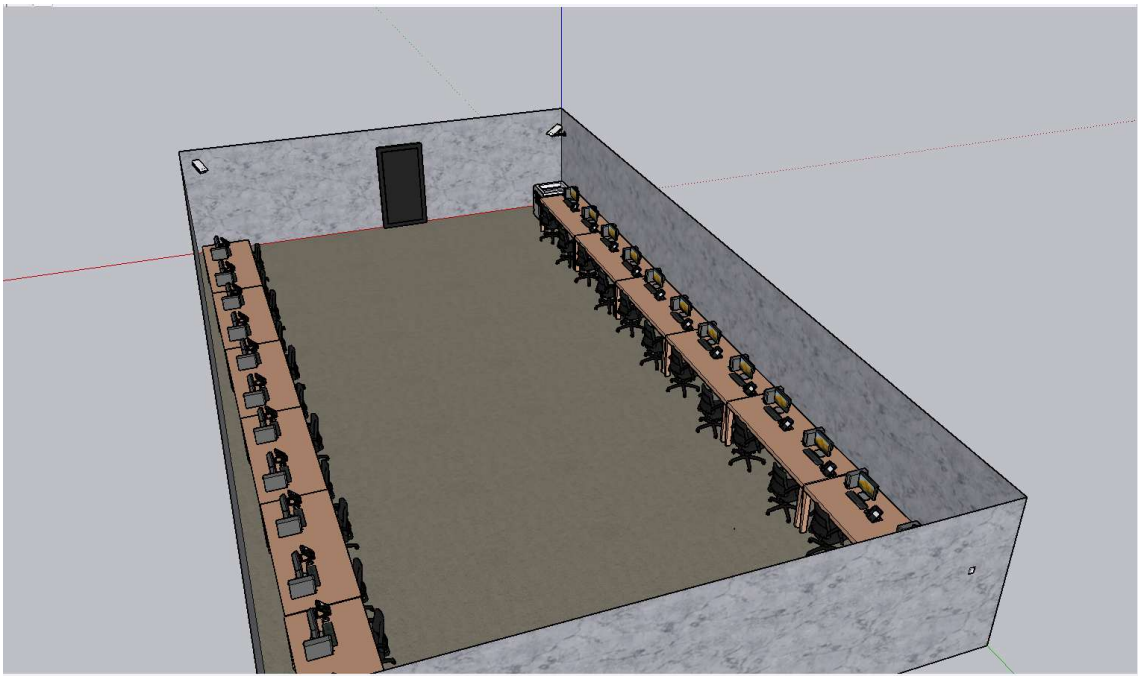


Zowmir:5/11 empezé a hacer el piso 7, en el cual requeria una habitacion de 18.6m de largo x 10.2m de ancho, con 2.8m de alto. hice la habitacion en sketch up y luego busque en las eccion de 3d warehouse de la aplicación una computadora, un telefono, una mesa, una camara y una impresora de red.





como se puede apreciar. son 2 filas de mesas. 6 mesas en cada fila. Cada mesa tiene 2 computadoras junto con 2 teléfonos ip y el respectivo asiento. Como me faltó 1 computadora y 1 teléfono más, descargué una mesa pequeña para ponerla al lado de una de las filas y ahí agregué la pc y teléfono faltante. en 1 de las filas al lado de la puerta se encuentra la impresora de red. también habiendo 3 cámaras donde asigne 2 en las esquinas donde se encuentra la puerta y la otra la coloqué en el fondo del lado izquierdo



(lo de la puerta lo soluciono despues)

6 de noviembre:

Goldfarb:

Fecha: 6/11/26

Ubicación: Escuela : Aula

Hora: 15:00

Actividades Realizadas:

Hoy era el último día para entregar el proyecto. Estuve ayudando a mis compañeros de equipo a realizar su laburo. También, actualice TODA mi carpeta de campo, ya que me faltaba todo el material del día 26/6 hasta el 8/10. También pasé los archivos html al cisco (Al usar bootstrap, elemento que cisco no entiende, la página se verá distinta. Aun así, Nahuel dijo que estaba bien y que no pasaba nada.

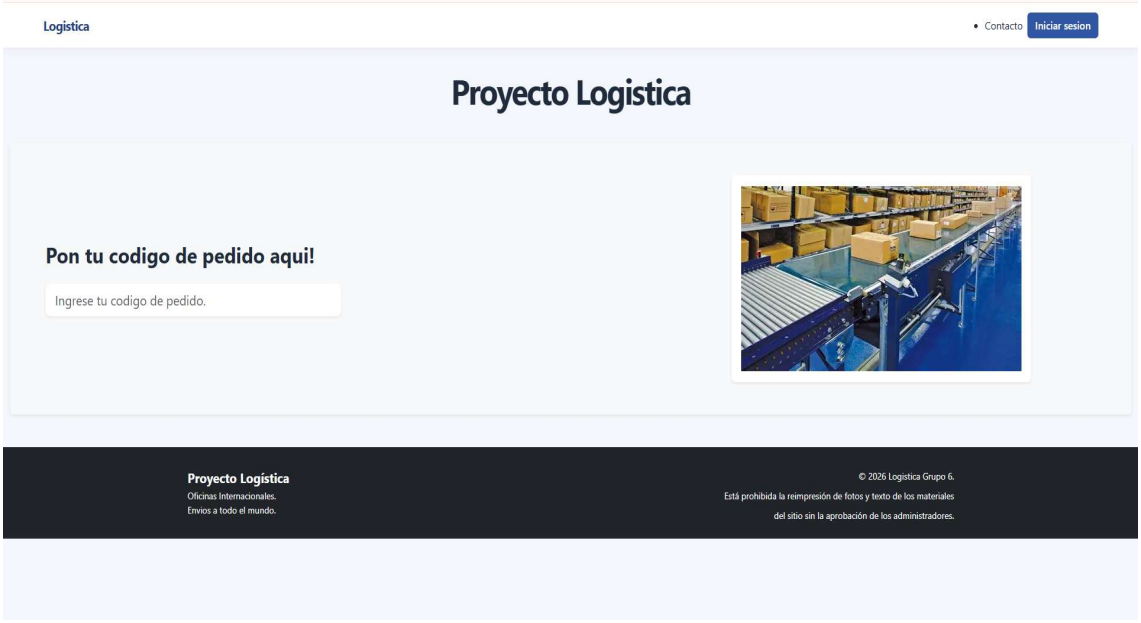
Pedraza:

El día viernes **6/11** realicé la última actividad que me asignaron referente a lo que es códigos: modificar el aspecto del Index (que incluía header, body y footer) y el footer del Index de admin.

Por el lado del header del index normal, copie el formato del index de admin. Las cosas distintas que tenia este nuevo header eran un efecto hover en el texto “Logistica” (el cual refresca la pagina), y lo mismo con el boton de tu usuario.

En el body, adapte el titulo “Proyecto Logistica” como en el index de admin, y en el footer modifique los textos de copyright, ya que la version anterior contenia unos textos muy chistosos, y elimine los botones para ver envios y paquetes, por orden del lider Goldfarb.

Resultado final del index normal:



También había modificado el footer del index de admin, me lo pidió mi compañero Saragusti para corroborarlo y subirlo él, pero no aparece el cambio. Me dijo que ya no se puede cambiar porque ya se plasmaron en el Cisco.

Osea que en la presentación, tendremos una situación muy ‘’chistosa’’ cuando los profesores vean la seriedad del texto que se encuentra en el footer del index de admins.

Durante la tarde, en mi casa, realice parte de la carpeta de campo grupal. Una carpeta muy improvisada en el momento, pero bueno, a mi se me asignó completarla únicamente con imágenes subidas por cada integrante en sus carpetas o el grupo de Whatsapp y en orden cronológico, pero me parecía que se vería muy vacío, por lo que decidí hacer el siguiente formato: escribir la fecha, escribir el apellido de un integrante que trabajó, pegar parte que explica lo que hizo, sacado de su carpeta personal, y fotos de sus trabajos.
Pude completar el mes de Agosto e inicios de Septiembre, hasta que tuve que salir de casa, por lo que le solicité a mis compañeros si alguien podía terminarla.

Rosato:

6/11

En la fecha actual, se procedió a la elaboración y redacción técnica del Manual de Explotación del sistema informático logístico, contando con la colaboración del compañero Benjamin Saragusti. Este documento formal describe detalladamente los requisitos de infraestructura, la arquitectura del sistema, el proceso de despliegue y los pasos secuenciales necesarios para realizar la correcta instalación y puesta en marcha del programa en el entorno operativo.

Saragusti:El 6 de noviembre estuve colaborando con los últimos detalles de la página html, siendo unos pequeños retoques a algunas llamadas a directorios y específicamente a cómo se tratan algunas de las tablas, debido a que no se iba a la ventana de mostrar productos por un nombre incorrecto, el nombre en el index admin tenia mostrar productos con un espacio en vez de todo pegado como el que había hecho Xiomara, por otro lado probé el eliminar envios y borra bien por lo que las tres sentencias sql no tiran algún error en el medio.
También colaboré en la elaboración del manual de usuario de html, y en el manual de explotación general de java y html.

Fortalezas: - Flexibilidad operativa - Rapidez de respuesta - Equipo operativo capacitado - Buen nivel de servicio al cliente	Oportunidades: - Optimización de rutas - Integración más eficiente entre sistemas - Capacitación continua del personal
Debilidades: - Diferencias de stock causadas por errores humanos - Falta de integración entre distintos sistemas de clientes - Reclamos por entregas fallidas o incompletas	Amenazas: - Dependencia tecnológica - Retrasos externos en el transporte - Mayor competencia en el mercado - Picos de demanda en fechas especiales - Costos crecientes de

(Vistazo de cómo quedó el análisis Foda, es exactamente el mismo texto que tenía antes solamente que ahora en formato pdf, antes siendo solamente una foto).

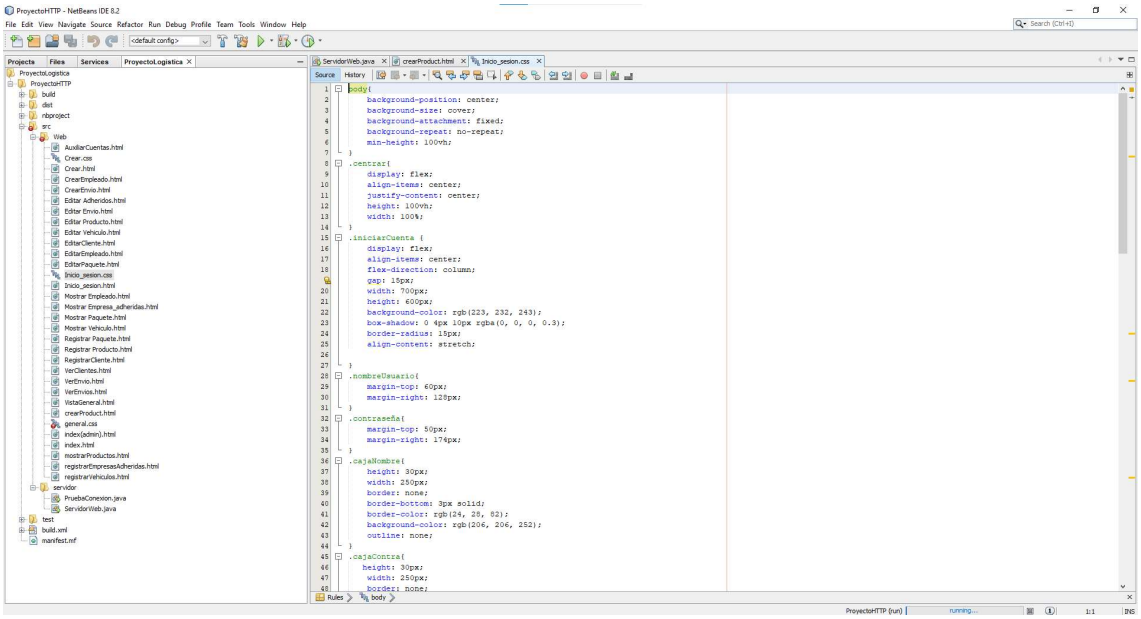
Reorganice los archivos, subiendo lo requerido al github, para que ya quedará la última versión de la página establecida en el documento, terminando así lo que me correspondía.

Solo me falta terminar de conectar las maquetas restantes y no mucho más.

También estuve subiendo los pdf de todos los archivos generales, exceptuando carpetas de campo, para que los profesores después no nos reten por tener links a documentos en la nube en vez de tenerlos como pdfs.

Xiomara:**06/11/2026**

Hoy es el día de la entrega, todavía nos faltan cosas por terminar. Entonces me puse a seguir con las actividades que me quedaban. Como ayer terminé de modificar el css, mis compañeros me pidieron que les pase lo que hice para así poder unirlo a la conexión de Html x Java x MySQL.

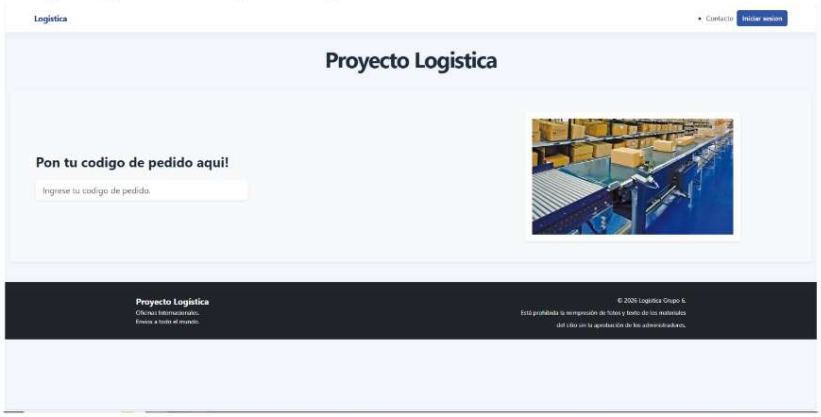


Zowmir:

6/11
Hice una de las cosas que faltaba por terminar que era hacer el manual de usuario para las páginas de cliente y administrador. Al momento de hacerlo, estuve viendo imperfecciones en el CSS que fueron corregidos y un error en la página del administrador. específicamente en el ver clientes. ya que al clickear no encontraba el archivo, no hubo problema en solucionarlo ya que solo cambiamos el nombre del link de “mostrar cliente.html” a “VerClientes:html” que así es como se llama la pagina para ver los clientes registrados

MANUAL DE USUARIO:
allí explique el uso de cada pagina. tanto del lado del cliente como el del administrador. las explicaciones fueron bastante cortas y claras de entender.
EJ:

1) Página Main (cliente)

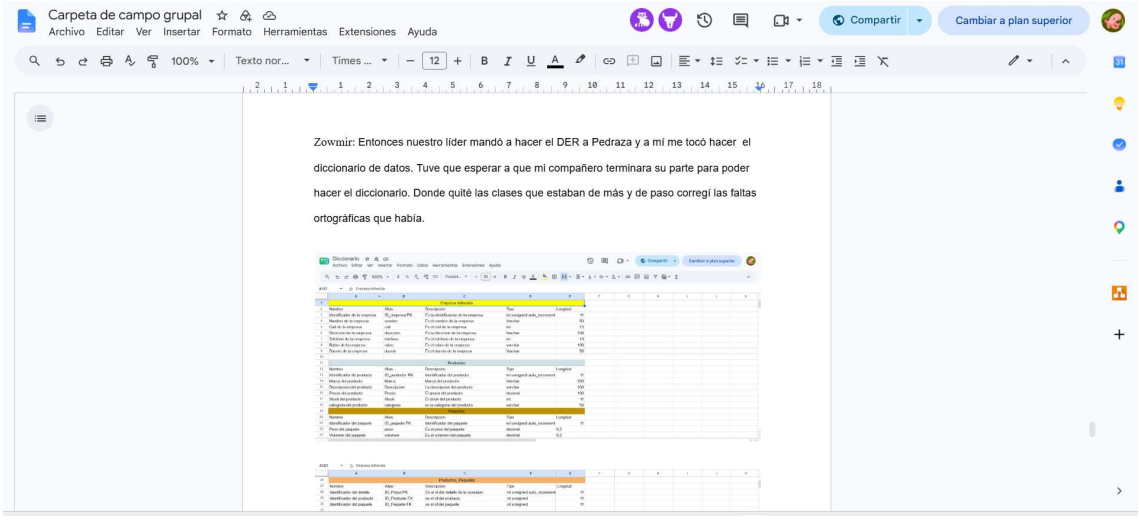


en la página main el cliente tiene las opciones de poner el código de su pedido para ver los datos (Destino,Origen,Estado del pedido, etc.), tiene el botón de contactos para que se pueda contactar con el equipo de soporte por si hay algún problema y tiene el botón de iniciar sesion para que pueda abrir su cuenta(o crear una) en la página.

explicacion de como se usa la pagina main del cliente

Además, a último momento me encargué de terminar la carpeta de campo grupal que había comenzado mi compañero Santiago Pedraza. El objetivo de este trabajo era reunir en un solo documento PDF todas las actividades y tareas que fuimos realizando durante el desarrollo del proyecto, para dejar registrado el trabajo de cada integrante y el avance que tuvimos como grupo.

Para completarla, tuve que recopilar y organizar la información y los trabajos realizados, siguiendo como referencia el ejemplo que se muestra a continuación. De esta manera, buscamos que quedara un registro ordenado de todo el proceso del proyecto.



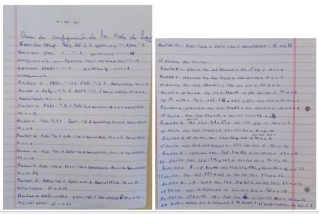
Carpeta de campo grupal

Archivo Editar Ver Insertar Formato Herramientas Extensiones Ayuda

Compartir | Cambiar a plan superior

Saragusti: además de la revisión yo termine de configurar y conectar las últimas 10 redes que faltaban con dhcp,dns y servidor http funcional, que igual no tuve que modificar los datos de los routers ya que previamente los había dejado preparados para la extensión de la red.

También hice una hoja escrita en la que deje la estructura plasmada de la configuración de los equipos para que mis compañeros pudieran entender la estructura y como estaba armada la red, ya que una explicación oral no servirá tanto como el código plasmado en texto de una manera sencilla para fácil deducción de los ideales.



30 de 37